

DOCUMENTAÇÃO TÉCNICA COMPLETA

SUMMIT

O Livro do Sistema — arquitetura, banco de dados, client WPF, backend e cada feature explicada de ponta a ponta

Gerado a partir do código-fonte real do projeto (Summit.csproj + Summit.Api) · docs/book/

SUMMIT — O Livro do Sistema

Documentação técnica completa da plataforma de campeonatos de CS2

Este livro documenta o sistema Summit (ex-Wallbang) de ponta a ponta: o client desktop WPF, a API ASP.NET Core, o banco de dados MySQL, e a lógica de cada feature — para que um desenvolvedor intermediário que nunca viu o projeto consiga, lendo isto, ganhar domínio real sobre como o sistema é construído e por quê. Não cobre configuração de infraestrutura AWS (isso já está em [docs/pLano-aws.md](#)) — mas cobre a lógica de como o pool de servidores funciona, porque essa lógica é parte do desenho do sistema, não da infraestrutura em si.

Todo código mostrado aqui é o código real do projeto, com caminho de arquivo e trecho exato, a menos que esteja explicitamente marcado como "exemplo genérico" para ilustrar um conceito.

Como este livro está organizado

O livro tem seis partes. As Partes I-IV constroem, camada por camada, o entendimento de como o sistema é montado (arquitetura → banco → client → API). A Parte V pega cada *feature* do produto e explica, de ponta a ponta, como ela atravessa todas essas camadas — é a parte mais longa e a mais importante para quem vai *trabalhar* no sistema no dia a dia. A Parte VI é referência pura: um dicionário de toda classe do projeto, para consulta rápida.

Parte I — Visão Geral e Fundamentos

- Capítulo 1 — O que é o Summit
- Capítulo 2 — Arquitetura Geral
- Capítulo 3 — Padrões de Projeto Usados

Parte II — Banco de Dados

- Capítulo 4 — Modelo de Dados Completo

Parte III — Client WPF

- Capítulo 5 — MVVM na Prática
- Capítulo 6 — Navegação e Comunicação com a API
- Capítulo 7 — Modelos do Client
- Capítulo 8 — Services e Repositórios do Client

Parte IV — Backend (Summit.Api)

- [Capítulo 9 — Program.cs e o Bootstrap da API](#)
- [Capítulo 10 — Endpoints por Domínio](#)
- [Capítulo 11 — Services e Background Workers](#)

Parte V — Features de Ponta a Ponta

- [Capítulo 12 — Conta, Login e Perfil](#)
- [Capítulo 13 — Times](#)
- [Capítulo 14 — Amizades](#)
- [Capítulo 15 — Auditoria](#)
- [Capítulo 16 — Campeonatos: Inscrição e Check-in](#)
- [Capítulo 17 — Escalação \(Lineup\)](#)
- [Capítulo 18 — Chave \(Bracket\)](#)
- [Capítulo 19 — Veto de Mapas e Sala da Partida](#)
- [Capítulo 20 — Pool de Servidores CS2 \(a lógica, não a infra\)](#)
- [Capítulo 21 — Pós-Partida: o Que Ainda Não Existe](#)

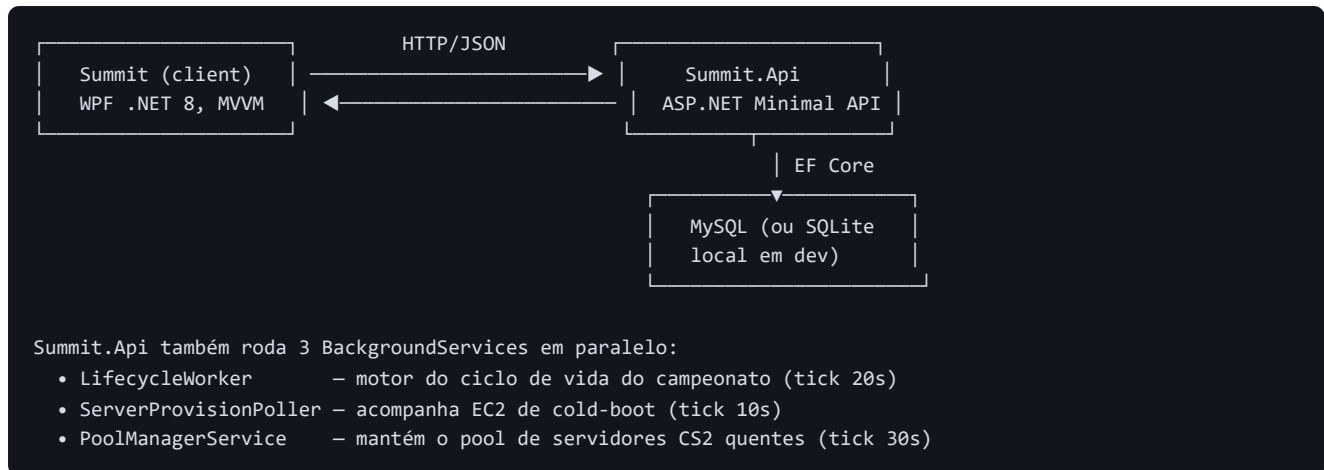
Parte VI — Referência de Classes

- [Capítulo 22 — Referência: Client \(Models, ViewModels, Views, Services\)](#)
- [Capítulo 23 — Referência: API \(Endpoints, Services, DbContext\)](#)

Apêndices

- [Apêndice A — Glossário](#)
 - [Apêndice B — Convenções de Código](#)
 - [Apêndice C — Roadmap Consolidado](#)
-

Mapa mental rápido (para quem vai ler correndo)



Os dois projetos (`Summit.csproj` e `Summit.Api/Summit.Api.csproj`) vivem no mesmo repositório e **compartilham a pasta `Models/`** via link direto de arquivos (`<Compile Include="..\Models\*.cs" LinkBase="Models" />`; no `.csproj` da API) — não é um pacote NuGet separado, é literalmente o mesmo arquivo `.cs` compilado nos dois projetos. Isso significa que uma classe como `Tournament` tem exatamente o mesmo formato dos dois lados da rede, o que elimina uma classe inteira de bugs de serialização/desserialização divergente que normalmente aparece quando client e servidor têm modelos "parecidos, mas não iguais".

Capítulo 1 — O que é o Summit

1.1 O produto

Summit é uma plataforma de campeonatos de Counter-Strike 2. Na prática, ela resolve o mesmo problema que sites como FACEIT ou ESEA resolvem: dar a jogadores e times um lugar para se organizar (perfil, time, amigos), se inscrever em campeonatos, disputar uma chave eliminatória com vetos de mapa no estilo profissional, e jogar a partida num servidor dedicado que a própria plataforma sobe automaticamente — sem que o jogador precise alugar ou configurar nada.

O sistema tem duas metades que conversam por HTTP:

1. **Um client desktop** (`Summit.csproj`), feito em WPF/.NET 8, que é a interface que o jogador realmente usa — telas de time, campeonatos, perfil, ranking, sala de partida.

1. **Uma API** (`Summit.Api/Summit.Api.csproj`), ASP.NET Core Minimal API, que guarda todo o estado real do mundo (banco de dados) e aplica todas as regras de negócio. O client nunca decide nada sozinho — ele só *pede* coisas à API e mostra o que ela responde.

Não existe (ainda) versão web ou mobile; o único jeito de usar o Summit hoje é rodando o executável WPF em uma máquina Windows, apontando para uma API que pode estar rodando localmente (`localhost:5180` , o padrão) ou em qualquer outro host via a variável de ambiente `SUMMIT_API_URL` .

1.2 De onde veio o nome

O projeto começou com o nome "Wallbang" (um termo de CS2 — atirar através de uma parede) e foi rebatizado para "Summit" no meio do desenvolvimento. Por isso o diretório do repositório no disco ainda se chama `Wallbang` , mas todo o código, namespaces (`Summit.*`) e a marca já são "Summit". Isso não é um bug nem uma inconsistência a corrigir — é só histórico, documentado aqui para que ninguém estranhe o descompasso entre o nome da pasta e o nome do produto.

1.3 O que o sistema já faz de ponta a ponta (visão de produto)

Se você seguir a jornada completa de um jogador, hoje ela funciona assim:

1. Ele abre o client e faz login com a conta Steam (OpenID) — ou usa um modo demo sem Steam.
2. No primeiro login, um onboarding mínimo pede país e função principal (rifler, AWPer, IGL...).
3. Ele cria um time (vira dono automaticamente) ou é convidado/solicita entrada em um existente.
4. Dentro do time, dono e sublíder podem promover, rebaixar, transferir propriedade, remover jogador, editar dados do time ou excluí-lo.

1. O dono (ou sublíder) inscreve o time em um campeonato aberto, escolhendo os 5 jogadores da

escalação e quem é o capitão da escalação daquela competição específica.

1. Quando o check-in abre (1h antes do início), o capitão confirma presença — quem não confirma é removido automaticamente 30 minutos antes do início.

1. A chave é gerada (eliminação simples ou dupla, qualquer quantidade de times) e as partidas da primeira rodada abrem o veto de mapas automaticamente, no formato configurado (MD1/MD3/MD5).

1. Ao fim do veto, a plataforma prepara uma sala com IP, senha e mapa — hoje já testado com um servidor CS2 + MatchZy real rodando na AWS, e com um "pool" de servidor sempre quente para que isso não demore minutos.

1. O jogador entra no servidor e joga.

O que **ainda não existe** é o que acontece depois disso: não há como o resultado da partida voltar para a plataforma automaticamente, então a chave não avança sozinha, badges não são concedidas por desempenho real, e o campeonato nunca se "encerra" sozinho. Isso está detalhado no [Capítulo 21](#) — é o maior buraco conhecido do sistema hoje, e vale entender isso cedo porque explica por que várias telas do produto (estatísticas, ranking, histórico de partidas) hoje só mostram dados de *seed* (dados de demonstração), não dados reais gerados pelo uso.

1.4 Como ler este livro se você é novo no projeto

Se seu objetivo é só entender uma feature específica (ex.: "como funciona o veto de mapas?"), vá direto para a Parte V — cada capítulo ali é autocontido e te manda de volta para os capítulos de fundação (Parte II-IV) só quando precisa de um conceito que ainda não foi explicado.

Se seu objetivo é ganhar domínio geral do sistema para poder trabalhar nele com confiança, leia as Partes I-IV em ordem primeiro — elas constroem o vocabulário (o que é um `BaseViewModel`, como o `ApiDbContext` mapeia os modelos, como o `ApiClient` fala com a API) que todos os capítulos de feature vão assumir que você já tem.

Capítulo 2 — Arquitetura Geral

2.1 Os dois projetos

O repositório tem dois projetos .NET lado a lado:

Projeto	Arquivo	SDK	Tipo	Roda onde
Client	Summit.csproj	Microsoft.NET.Sdk + UseWPF	Executável Windows (WinExe)	Máquina do jogador
API	Summit.Api/Summit.Api.csproj	Microsoft.NET.Sdk.Web	Web API (Kestrel)	Servidor (ou localhost em dev)

Ambos miram .NET 8 (o client em net8.0-windows porque usa WPF, que é Windows-only; a API em net8.0 puro, porque não tem nenhuma dependência de UI).

2.1.1 Como eles compartilham código

Olhando o Summit.Api.csproj:

```
<!-- Modelos compartilhados com o client WPF -->
<ItemGroup>
  <Compile Include="..\Models\*.cs" LinkBase="Models" />
</ItemGroup>
```

Isso não importa um pacote — ele **inclui os mesmos arquivos-fonte** da pasta Models/ do client diretamente na compilação da API, via link (LinkBase, sem copiar fisicamente o arquivo). Na prática, isso quer dizer que existe *um* arquivo Models/Tournament.cs, e tanto Summit.exe quanto Summit.Api.exe o compilam para dentro de si. Não existe risco de o client ter um Tournament com um campo a mais ou a menos do que a API espera — é fisicamente o mesmo tipo.

A implicação prática mais importante disso: **quando você adiciona um campo em Models/, os dois lados enxergam automaticamente** (depois de recompilar os dois). Não existe um passo de "gerar DTO" ou "atualizar contrato" — o contrato é o código.

2.2 Como os dois se comunicam

Só por HTTP/JSON, simples assim. Não há gRPC, SignalR ou WebSocket em lugar nenhum do sistema — mesmo telas que parecem "tempo real" (como a sala de partida durante o veto) são implementadas com **polling**: o client pergunta de novo a cada poucos segundos.

- O client fala com a API inteiramente através de Services/ApiClient.cs — uma classe estática

com um único HttpClient compartilhado, que aponta para SUMMIT_API_URL (ou http://localhost:5180 por padrão). Ver Capítulo 6 para os detalhes desse cliente.

- A API expõe tudo via **Minimal API** do ASP.NET Core — não há Controllers, não há MVC, é tudo

`app.MapGet(...)` / `app.MapPost(...)` direto no `Program.cs` (e em `Summit.Api/CompetitionEndpoints.cs`, que é um segundo arquivo de rotas organizado por método de extensão `MapCompetitionEndpoints(this WebApplication app)`).

Um exemplo real do padrão (client → API) para pegar um campeonato por id:

```
// Data/TournamentRepository.cs
public Task<Tournament?> GetByIdAsync(string id)
    => ApiClient.GetAsync<Tournament>($"api/tournaments/{id}");
```

```
// Summit.Api/Program.cs
app.MapGet("/api/tournaments/{id}", async (ApiDbContext db, string id) =>
{
    var t = await db.Tournaments
        .Include(x => x.TournamentTeams).ThenInclude(tt => tt.Team).ThenInclude(tm => tm!.Members)
        .Include(x => x.Bracket).ThenInclude(r => r.Matches)
        .FirstOrDefaultAsync(x => x.Id == id);
    return t == null ? Results.NotFound() : Results.Ok(t);
});
```

O client nunca monta SQL, nunca sabe que existe MySQL por trás — ele só sabe que existe um caminho HTTP que devolve um `Tournament` em JSON.

2.3 Persistência

A API usa **Entity Framework Core** com dois provedores possíveis, escolhidos em runtime pelo `Program.cs`:

```
var mysql = Environment.GetEnvironmentVariable("SUMMIT_DB")
    ?? builder.Configuration.GetConnectionString("MySQL");

builder.Services.AddDbContext<ApiDbContext>(o =>
{
    if (!string.IsNullOrEmpty(mysql))
        o.UseMySQL(mysql, ServerVersion.AutoDetect(mysql));
    else
        o.UseSqlite($"Data Source={Path.Combine(builder.Environment.ContentRootPath, "summit-api.db")}");
});
```

Se a variável de ambiente `SUMMIT_DB` (ou a connection string `MySQL` do `appsettings.json`) estiver definida, usa **MySQL** (via o provedor Pomelo) — esse é o modo "de verdade", usado no dia a dia deste projeto. Se não estiver definida, cai automaticamente para um arquivo **SQLite** local (`summit-api.db`) — um modo de conveniência para rodar a API sem precisar instalar MySQL (por exemplo, em uma máquina nova ou em CI). Os dois caminhos usam exatamente o mesmo `ApiDbContext` e o mesmo conjunto de entidades — só troca o motor por baixo.

Importante: **não existe sistema de migrations** neste projeto. O schema é criado com `db.Database.EnsureCreated()` no startup, uma única vez (se as tabelas já existem, não faz nada). Isso significa que qualquer mudança de schema depois que o banco já existe precisa de um `ALTER TABLE`

manual — ver o [Capítulo 4](#) para a explicação completa dessa decisão e como aplicar mudanças na prática.

2.4 Os três processos de fundo da API

Além de responder requisições HTTP, o processo da API roda três `BackgroundService` em paralelo, registrados no `Program.cs`:

```
builder.Services.AddHostedService<LifecycleWorker>();
builder.Services.AddSingleton<MatchServerService>();
builder.Services.AddHostedService<ServerProvisionPoller>();
builder.Services.AddHostedService<PoolManagerService>();
```

Worker	Tick	Responsabilidade
<code>LifecycleWorker</code>	20s	Fecha check-in, remove ausentes, gera a chave, inicia o campeonato na hora certa, abre vetos, roda bots de veto para contas demo
<code>ServerProvisionPoller</code>	10s	Acompanha instâncias EC2 criadas sob demanda ("cold boot"), grava o IP assim que fica pronta
<code>PoolManagerService</code>	30s	Mantém N servidores CS2 sempre ligados e prontos, confirma via RCON que estão de fato utilizáveis, libera automaticamente os que ficaram vazios

Cada um roda em loop infinito (`while (!ct.IsCancellationRequested)`) com um `try/catch` que nunca deixa uma exceção matar o worker — só loga e tenta de novo no próximo tick. Isso é uma decisão deliberada: é preferível que um tick falhe silenciosamente e tente de novo em 10-30s do que a falha de uma verificação (ex.: uma chamada à AWS que deu timeout) derrubar o processo inteiro da API.

Esses workers são detalhados no [Capítulo 11](#) e sua lógica de produto (por que existem, o que resolvem) está espalhada pelos capítulos de feature correspondentes (Chave no [Capítulo 18](#), Pool de servidores no [Capítulo 20](#)).

2.5 Estrutura de pastas

```
Wallbang/                                     (nome da pasta no disco – o produto se chama Summit)
├─ Summit.csproj                             ← projeto do client WPF
├─ App.xaml / App.xaml.cs                   ← bootstrap do client, monta os services estáticos
├─ Commands/RelayCommand.cs                 ← implementação de ICommand para binding de botões
├─ Helpers/                                 ← IValueConverter's usados no XAML
├─ Components/                              ← UserControls reutilizáveis (ex. LevelBadge)
├─ Models/                                  ← COMPARTILHADO com a API (ver 2.1.1)
├─ Data/                                    ← Repositórios do client (uma classe por área, fala com ApiClient)
├─ Services/                                ← Services do client (regra de UI/orquestração) + auth Steam
│   └─ Interfaces/                          ← interfaces dos services principais
├─ ViewModels/                              ← um ViewModel por tela (ou por sub-fluxo de tela)
├─ Views/                                   ← XAML + code-behind mínimo (1:1 com ViewModels)
├─ Resources/                               ← design system: cores, tipografia, estilos de botão/input/card
├─
├─ Summit.Api/                              ← projeto da API (Sdk.Web)
│   └─ Summit.Api.csproj
│   └─ Program.cs                           ← bootstrap + a maioria dos endpoints REST
│   └─ ApiDbContext.cs                      ← mapeamento EF Core de todas as entidades
│   └─ CompetitionEndpoints.cs              ← endpoints das especificações de time/campeonato + helpers de regra
│   └─ LifecycleWorker.cs                  ← motor do ciclo de vida do campeonato
│   └─ MatchServerService.cs                ← provisionamento AWS (cold-boot e pool) + RCON de alto nível
│   └─ PoolManagerService.cs                ← mantém o pool de servidores quentes
│   └─ ServerProvisionPoller.cs             ← acompanha cold-boot
│   └─ RconClient.cs                       ← cliente do protocolo Source RCON, escrito na mão
│   └─ SeedData.cs                         ← dados de demonstração (usuários, times, campeonatos, partidas)
├─ database/
│   └─ schema.sql                          ← dump do schema MySQL (referência, não é aplicado automaticamente)
│   └─ start-mysql.ps1                     ← sobe o MySQL local de dev
├─ docs/
│   └─ espec-times.md                      ← especificação funcional de times/perfis/amizades
│   └─ espec-campeonatos.md                ← especificação funcional do fluxo de campeonato
│   └─ plano-aws.md                        ← plano + histórico de infraestrutura AWS (fora do escopo deste livro)
│   └─ pendencias.md                       ← lista viva do que falta/precisa melhorar
│   └─ book/                               ← você está aqui
```

Note a simetria: para quase toda pasta do client (`Data/` , `Services/` , `ViewModels/`) existe uma contraparte conceitual do lado da API, mas a API não replica a mesma divisão em pastas — ela é pequena o suficiente para caber em poucos arquivos por domínio (`Program.cs` para tudo que é CRUD simples, `CompetitionEndpoints.cs` para a lógica de regras mais elaborada).

2.6 Ambientes e configuração

Tudo que muda entre "minha máquina" e "produção" é lido de variáveis de ambiente — não existe `appsettings.Production.json` customizado neste projeto. As principais:

Variável	Lido por	Efeito
<code>SUMMIT_API_URL</code>	client (<code>ApiClient</code>)	endereço da API; padrão <code>http://localhost:5180</code>
<code>SUMMIT_DB</code>	API (<code>Program.cs</code>)	connection string MySQL; ausente = usa SQLite local

Variável	Lido por	Efeito
<code>SUMMIT_STEAM_API_KEY</code>	client (<code>SteamConfig</code>)	chave da Steam Web API para buscar nick/avatar reais
<code>AWS_ACCESS_KEY_ID</code> / <code>AWS_REGION</code> / <code>SUMMIT_AMI_ID</code> / ...	API (<code>MatchServerService</code>)	credenciais e parâmetros AWS (fora do escopo deste livro — ver <code>docs/plano-aws.md</code>)
<code>SUMMIT_POOL_SIZE</code>	API (<code>PoolManagerService</code>)	quantos servidores CS2 manter sempre quentes (padrão <code>1</code>)

Essa é uma escolha intencional de simplicidade: qualquer pessoa consegue rodar o sistema inteiro localmente sem tocar em nenhum arquivo de configuração — só definindo (ou não) essas variáveis antes de iniciar os dois executáveis.

Capítulo 3 — Padrões de Projeto Usados

Este capítulo cataloga os padrões arquiteturais recorrentes no sistema. A ideia é que, depois de lê-lo, você reconheça a "forma" de qualquer arquivo novo que abrir no projeto, porque ele vai seguir um desses moldes.

3.1 MVVM no client (Model-View-ViewModel)

O client é WPF clássico com MVVM manual (sem framework como Prism ou CommunityToolkit.Mvvm — tudo escrito à mão, poucas classes de suporte). As três camadas:

- **Model** (`Models/*.cs`): dados puros, compartilhados com a API (ver Capítulo 2). Podem ter

propriedades computadas em C# (getters sem setter) para lógica de exibição que não depende de estado de UI — ex. `Tournament.StatusLabel`, `Tournament.CountdownLabel`.

- **View** (`Views/*.xaml` + `.xaml.cs`): XAML puro de layout/estilo, com o `.xaml.cs` quase

vazio — normalmente só o construtor chamando `InitializeComponent()`. Toda lógica de apresentação mora no ViewModel, nunca no code-behind.

- **ViewModel** (`ViewModels/*.cs`): estado da tela + comandos. Toda classe herda de

`BaseViewModel` e expõe propriedades com `SetProperty` (dispara `INotifyPropertyChanged`) e comandos como `RelayCommand`.

A base de tudo:

```
// ViewModels/BaseViewModel.cs
public abstract class BaseViewModel : INotifyPropertyChanged
{
    public event PropertyChangedEventHandler? PropertyChanged;

    protected void OnPropertyChanged([CallerMemberName] string? name = null)
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(name));

    protected bool SetProperty<T>(ref T field, T value, [CallerMemberName] string? name = null)
    {
        if (EqualityComparer<T>.Default.Equals(field, value)) return false;
        field = value;
        OnPropertyChanged(name);
        return true;
    }
}
```

`SetProperty` é o idioma que se repete em toda propriedade de todo ViewModel do projeto:

```
private bool _isLoading;
public bool IsLoading { get => _isLoading; set => SetProperty(ref _isLoading, value); }
```

Quando uma propriedade depende de outra (ex. `HasNone` depende de `IsLoading` e `Logs.Count`), o padrão é disparar manualmente o `OnPropertyChanged` da propriedade derivada dentro do setter da propriedade base:

```
// ViewModels/AuditLogViewModel.cs
public List<AuditLog> Logs { get => _logs; set { SetProperty(ref _logs, value);
OnPropertyChanged(nameof(HasNone)); } }
public bool IsLoading { get => _isLoading; set { SetProperty(ref _isLoading, value);
OnPropertyChanged(nameof(HasNone)); } }
public bool HasNone => !IsLoading && Logs.Count == 0;
```

Não existe nenhum mecanismo automático de dependência entre propriedades (como teria um framework reativo) — é tudo explícito, o que é mais verboso mas também mais fácil de seguir sem "mágica": você lê o setter e vê exatamente quais outras propriedades ele afeta.

3.1.1 Comandos: `RelayCommand`

```
// Commands/RelayCommand.cs
public class RelayCommand : ICommand
{
    private readonly Action<object?> _execute;
    private readonly Func<object?, bool>? _canExecute;

    public RelayCommand(Action<object?> execute, Func<object?, bool>? canExecute = null) { ... }
    public RelayCommand(Action execute, Func<bool>? canExecute = null)
        : this(_ => execute(), canExecute == null ? null : _ => canExecute()) { }

    public bool CanExecute(object? parameter) => _canExecute?.Invoke(parameter) ?? true;
    public void Execute(object? parameter) => _execute(parameter);
}
```

Toda ação clicável no XAML (botão, item de lista) vira uma propriedade `RelayCommand` no ViewModel, inicializada no construtor. O padrão mais comum é uma lambda `async` chamando um método privado:

```
SaveCommand = new RelayCommand(async _ => await SaveAsync());
```

O segundo construtor (que recebe `Action / Func<bool>` sem parâmetro) existe só para não precisar escrever `_ =>`; toda vez que o comando não precisa do parâmetro do clique — é açúcar sintático, mas usado nos dois estilos dependendo se o comando precisa saber *qual* item foi clicado (ex. `KickCommand` recebe o `userId` como parâmetro) ou não (ex. `SaveCommand`).

3.1.2 Ligação View ↔ ViewModel: `DataTemplates` implícitos

Não existe navegação de `Window` para `Window`, nem `Frame.Navigate`. A troca de tela dentro do shell principal funciona porque `App.xaml` registra um `DataTemplate` para cada tipo de ViewModel, mapeando para a View correspondente:

```
<DataTemplate DataType="{x:Type vm:TeamViewModel}">
    <views:TeamView/>
</DataTemplate>
```

Quando um `ContentControl` (no shell principal) recebe um objeto `TeamViewModel` como `Content / DataContext`, o WPF olha essa tabela e renderiza automaticamente a `TeamView` — sem nenhum código de "qual view mostrar" em lugar nenhum. Ver [Capítulo 6](#) para o mecanismo completo de navegação que usa isso.

3.2 Repository Pattern (client) — uma classe HTTP por área

Cada arquivo em `Data/*.cs` é um repositório fino: só sabe montar a URL e o corpo da requisição, chamando `ApiClient`. Não tem lógica de negócio nenhuma — é tradução pura entre "o que o ViewModel quer" e "qual endpoint HTTP chamar".

```
// Data/TeamRepository.cs
public class TeamRepository
{
    public Task<Team?> GetByIdAsync(string teamId)
        => ApiClient.GetAsync<Team>($"api/teams/{teamId}");

    public Task<bool> KickAsync(string teamId, string userId, string byUserId)
        => ApiClient.PostBoolAsync($"api/teams/{teamId}/kick", new { userId, byUserId });
}
```

Todo repositório é instanciado com `new()` direto onde é usado (não há injeção de dependência do lado do client) — normalmente como campo `private readonly` no ViewModel ou no Service que o usa:

```
private readonly TeamRepository _repo = new();
```

3.3 Service Layer (client) — regra de aplicação acima do repositório

Acima dos repositórios existe uma camada de `Services/*.cs` que adiciona a lógica que depende do *usuário atual* ou combina mais de uma chamada. Por exemplo, `TeamService.PromoteAsync` não apenas chama o repositório — ele primeiro confere se quem está pedindo é o capitão:

```
// Services/TeamService.cs
public async Task<bool> PromoteAsync(string teamId, string userId)
{
    var me = App.UserService.CurrentUser;
    if (me?.TeamId == null || !me.IsCaptain) return false;
    return await _repo.PromoteAsync(teamId, userId, me.Id);
}
```

Isso é uma **conveniência de UI**, não uma **barreira de segurança** — o texto da especificação ([docs/espec-times.md §43](#)) é explícito: "todas as permissões validadas no backend. A UI nunca é a única barreira." A API repete essa mesma checagem de forma independente (`CompetitionEndpoints.IsOwner`). Se um client malicioso pular o `TeamService` e chamar a API direto, a API ainda recusa. Essa checagem do lado do client existe só para não fazer uma requisição HTTP destinada a falhar, e para desabilitar o botão na UI antes mesmo do clique.

Alguns Services têm interface (`Services/Interfaces/ITeamService.cs` , `ITournamentService.cs` , etc.) e outros não (`RankingService` , por exemplo, não tem interface). Não há um critério rígido documentado sobre quando criar interface — na prática, os que têm interface são os que existiam desde as fases iniciais do projeto; os mais novos foram adicionados diretamente como classe concreta.

3.4 Instâncias estáticas ao invés de injeção de dependência (client)

O client **não usa nenhum container de DI**. Todos os services e repositórios "globais" (um por tipo de dado, compartilhados pelo app inteiro) são criados uma vez em `App.xaml.cs` e expostos como propriedades estáticas da classe `App` :

```
// App.xaml.cs
public static NavigationService  Navigation      { get; private set; } = null!;
public static UserService        UserService     { get; private set; } = null!;
public static TeamService        TeamService     { get; private set; } = null!;
public static TournamentService  TournamentService { get; private set; } = null!;
// ...

protected override void OnStartup(StartupEventArgs e)
{
    Navigation      = new NavigationService();
    UserService     = new UserService();
    TeamService     = new TeamService();
    TournamentService = new TournamentService();
    // ...
    new SplashView().Show();
}
```

Qualquer ViewModel acessa esses services simplesmente escrevendo `App.TeamService.XyzAsync(...)` . Isso é o equivalente funcional de um container de DI com tudo registrado como *singleton*, só que sem framework — o "container" é a própria classe `App` . Para um projeto deste tamanho (um client desktop de usuário único, sem necessidade de trocar implementações em testes automatizados), isso é suficiente e evita a complexidade extra de configurar um container real. Se o projeto crescer a ponto de precisar testar ViewModels com mocks, essa é a primeira coisa que precisaria mudar.

Exemplo genérico de como isso se compara a um DI container tradicional (não é código do projeto, é só para contraste):

```
// Com um container de DI (não é assim no Summit):
services.AddSingleton<ITeamService, TeamService>();
// ... e no ViewModel: construtor recebe ITeamService via injeção

// Como é de fato no Summit — acesso direto ao estático:
await App.TeamService.CreateTeamAsync(name, tag);
```

3.5 Minimal API (backend) — rotas como funções, não Controllers

A API não usa o padrão MVC com Controllers/Actions. Cada rota é uma chamada `app.MapGet/MapPost/MapPut/MapDelete` com uma lambda (ou método local) que recebe os serviços que precisa via *parameter injection* automático do ASP.NET Core:

```
app.MapPut("/api/teams/{id}", async (ApiDbContext db, string id, UpdateTeamRequest req) =>
{
    if (!await CompetitionEndpoints.IsOwner(db, id, req.ByUserId)) return Results.Forbid();
    var team = await db.Teams.FirstOrDefaultAsync(t => t.Id == id);
    if (team == null) return Results.NotFound();
    // ...
    return Results.Ok(team);
});
```

O ASP.NET Core resolve `ApiDbContext` do container de DI, `id` da rota (`{id}`), e `req` do corpo JSON — tudo por convenção de tipo/nome, sem atributos explícitos na maioria dos casos. Esse é o motivo de `Program.cs` ter quase 900 linhas: não há divisão em Controllers separados, é tudo sequencial no mesmo arquivo (mais `CompetitionEndpoints.cs` para o segundo bloco de rotas). Isso é uma escolha de simplicidade típica de Minimal API em projetos deste porte — funciona bem até o arquivo ficar grande demais para navegar confortavelmente, ponto em que normalmente se quebra em múltiplos arquivos de extensão como já foi feito com `CompetitionEndpoints`.

3.5.1 O padrão dos DTOs (`record`)

Toda requisição com corpo JSON define um `record` posicional logo no fim do arquivo:

```
record UpdateTeamRequest(string Name, string? Description, string? LogoUrl, string? Country, string ByUserId);
```

`record` é usado (em vez de `class`) porque esses tipos são puramente dados de transporte — igualdade estrutural e imutabilidade fazem sentido para um corpo de requisição, e a sintaxe posicional deixa a declaração de uma linha só.

3.6 Cliente HTTP tolerante a falha (client → API)

`Services/ApiClient.cs` estabelece um padrão importante: a maioria dos métodos **engole exceção e devolve um valor default** em vez de propagar o erro:

```
public static async Task<T?> GetAsync<T>(string path)
{
    try
    {
        using var resp = await Http.GetAsync(path);
        if (!resp.IsSuccessStatusCode) return default;
        return await resp.Content.ReadFromJsonAsync<T>(Json);
    }
    catch
    {
        return default;
    }
}
```

Isso significa que se a API estiver fora do ar, uma tela que faz `GetAllAsync()` simplesmente recebe uma lista vazia — a tela abre "vazia", sem crash, sem mensagem de erro genérica travando o fluxo. É uma escolha deliberada de robustez para um app desktop: melhor UX degradada (tela vazia) do que uma exceção não tratada derrubando a janela.

A exceção a essa regra é o login: `PostRequiredAsync` **propaga** a falha, porque ali o usuário *precisa* saber que algo deu errado (não faz sentido "logar silenciosamente com sucesso vazio"):

```
public static async Task<T> PostRequiredAsync<T>(string path, object? body)
{
    HttpResponseMessage resp;
    try { resp = await Http.PostAsJsonAsync(path, body, Json); }
    catch (Exception ex)
    {
        throw new InvalidOperationException(
            $"Não foi possível conectar à Summit API em {BaseUrl}. A API está rodando?", ex);
    }
    // ...
}
```

E `PutWithMessageAsync` é um meio-termo: nunca lança exceção, mas **carrega a mensagem de erro real da API** de volta para quem chamou, porque a API por vezes retorna um `BadRequest(string)` com um motivo específico que vale a pena mostrar ao usuário (ex. "A escalação precisa de exatamente 5 jogadores."):

```
public static async Task<(bool Ok, string? Message)> PutWithMessageAsync(string path, object? body)
{
    using var resp = await Http.PutAsJsonAsync(path, body, Json);
    var text = await resp.Content.ReadAsStringAsync();
    if (resp.IsSuccessStatusCode) return (true, null);
    return (false, string.IsNullOrEmpty(text) ? null : text.Trim().Trim(' '));
}
```

Regra geral para saber qual usar ao escrever código novo: se a falha é "normal" (rede instável, API momentaneamente fora) e a tela pode se recuperar mostrando vazio, use `GetAsync` / `PostBoolAsync`. Se o usuário precisa ser bloqueado e informado, use `PostRequiredAsync`. Se existe uma mensagem de validação específica da API que vale a pena mostrar, use `PutWithMessageAsync`.

3.7 Validação sempre no backend, nunca só no client

Este é citado explicitamente na especificação (`docs/espec-times.md §43`) como "regra central de segurança", e o código segue isso à risca: toda rota que muda estado (`POST` / `PUT` / `DELETE`) revalida identidade, cargo e elegibilidade, mesmo que o client já tenha feito uma checagem equivalente antes de habilitar o botão. Exemplos de helpers reutilizados para isso:

```
// Summit.Api/CompetitionEndpoints.cs
public static async Task<bool> IsOwner(ApiDbContext db, string teamId, string userId)
    => await db.Users.AnyAsync(u => u.Id == userId && u.TeamId == teamId && u.TeamRole == TeamRole.Captain);

public static async Task<bool> IsOwnerOrSub(ApiDbContext db, string teamId, string userId)
    => await db.Users.AnyAsync(u => u.Id == userId && u.TeamId == teamId &&
        (u.TeamRole == TeamRole.Captain || u.TeamRole == TeamRole.ViceCaptain));
```

Praticamente toda rota sensível do sistema começa com uma dessas duas linhas.

3.8 Auditoria como efeito colateral padronizado

Toda ação administrativa relevante (promover, remover, editar time, trocar escalação, etc.) termina com uma chamada ao helper `Audit`:

```
public static Task Audit(ApiDbContext db, string action, string? actor, string? target,
    string? teamId, string? tournamentId, string? oldValue, string? newValue, string? reason)
{
    db.AuditLogs.Add(new AuditLog { Id = $"aud_{Guid.NewGuid():N}", Action = action, ... });
    return Task.CompletedTask;
}
```

Note que `Audit` só *adiciona* a entidade ao `DbContext` — não chama `SaveChangesAsync()` sozinho. Isso é intencional: o registro de auditoria entra na **mesma transação implícita** da alteração principal (o próximo `await db.SaveChangesAsync()` do endpoint grava os dois juntos). Isso importa porque significa que uma auditoria nunca fica "órfã" de uma mudança que falhou antes de salvar — os dois sempre ou salvam juntos, ou nenhum salva.

3.9 Propriedades computadas nos Models compartilhados

Os `Models/*.cs` (compartilhados entre client e API) frequentemente têm propriedades derivadas (getter-only, sem campo de apoio) que calculam algo a partir de outros campos:

```
// Models/Tournament.cs
public bool IsRegistrationOpen =>
    Status == TournamentStatus.Open && DateTime.UtcNow < RegistrationClosesAt;

public string CountdownLabel { get { /* ... */ } }
```

Do lado da API, o `ApiDbContext.OnModelCreating` explicitamente instrui o EF Core a **ignorar** essas propriedades no mapeamento do banco (`tour.Ignore(t => t.IsRegistrationOpen);` etc.) — elas não viram coluna, só existem em memória, calculadas sob demanda toda vez que são lidas. Isso permite que a mesma lógica de "está aberto para inscrição?" seja calculada de forma idêntica tanto quando o client recebe o objeto via JSON quanto quando a própria API o usa internamente (por exemplo, no `LifecycleWorker`), sem duplicar a fórmula em dois lugares.

3.10 Padrão de endpoint de debug (dev-only, deliberado)

Espalhados pelo `Program.cs` existem vários `app.MapGet/MapPost("/api/debug/...")` — inspeção de instâncias EC2, volumes, snapshots, estado do pool, um endpoint para mandar comando RCON ad-hoc, um para gerar a chave na hora sem esperar o horário automático. Isso é um padrão deliberado deste ambiente de desenvolvimento: em vez de abrir um console de banco ou entrar na AWS toda vez que for preciso inspecionar algo, um endpoint HTTP simples resolve com um `curl`. **Esses endpoints não têm autenticação nem checagem de permissão** — isso é aceitável precisamente porque são ferramentas de operação/depuração local, não superfície de produto; não devem ser expostos numa API voltada à internet pública sem revisão.

Capítulo 4 — Modelo de Dados Completo

4.1 Visão geral do schema

O banco (`database/schema.sql` , MySQL 8.x, charset `utf8mb4`) tem 17 tabelas no dump de referência (mais `poolservers` , criada depois via `ALTER / CREATE TABLE` manual e ainda não refletida nesse dump — ver a ressalva em §4.7 — o que dá 18 no total, batendo com os 18 `DbSet` s do `ApiDbContext` , ver §23.2). Todas usam chave primária `varchar(255)` com um prefixo legível por tipo (`usr_` , `team_` , `trn_` , `bm_` , `rnd_` , `m_` , `mp_` , `fr_` , `inv_` , `jrq_` , `lp_` , `veto_` , `vst_` , `aud_` , `pool_`) gerado em código como `$"usr_{Guid.NewGuid():N}"` — nunca um `int IDENTITY` . Isso é uma escolha simples que evita colisão entre ambientes (dá para gerar um id sem tocar o banco) e torna qualquer id imediatamente legível em um log (`usr_a1b2c3...` já diz "isso é um usuário").

Diagrama de relacionamento (setas = chave estrangeira apontando para quem "possui" a linha):



4.2 A decisão consciente de não usar migrations

O `Program.cs` cria o schema assim:

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<ApiDbContext>();
    db.Database.EnsureCreated();
    await SeedData.EnsureSeededAsync(db);
}
```

`EnsureCreated()` olha se o banco de dados de destino já tem tabelas; se não tiver nenhuma, cria o schema inteiro a partir do que está mapeado em `ApiDbContext.OnModelCreating` . **Se o banco já existe** (mesmo que desatualizado em relação ao código atual), `EnsureCreated()` não faz nada — ele não sabe comparar

"schema atual" com "schema desejado" e aplicar a diferença; essa capacidade de diff é exatamente o que o EF Core Migrations resolveria, e este projeto optou por não usar Migrations.

Por quê? Migrations exigem manter um histórico de arquivos de migração versionados e rodar `dotnet ef database update` a cada mudança — overhead real para um projeto em fase de desenvolvimento rápido, onde o schema muda com frequência e o banco de dev é recriado do zero sem custo. A troca é simples: **em desenvolvimento**, se o schema mudou, dropar e recriar o banco de teste é mais rápido que escrever uma migration. **Contra um banco que já tem dados que você quer preservar** (o caso mais comum neste projeto, onde o MySQL de dev acumula dados de teste ao longo de sessões), a mudança de schema precisa ser aplicada manualmente com `ALTER TABLE`, do jeito que já foi feito várias vezes ao longo deste projeto — por exemplo, quando o campo `Side` foi adicionado a `BracketRound`:

```
ALTER TABLE bracketrounds ADD COLUMN Side INT NOT NULL DEFAULT 0;
```

Regra prática para quem for mexer no schema: sempre que você adicionar um campo a uma classe em `Models/*.cs` que precisa ser persistido, você precisa fazer *duas* coisas, não uma:

1. Mapear o campo (ou confirmar que o EF Core vai inferir corretamente) em

```
ApiDbContext.OnModelCreating.
```

1. Rodar o `ALTER TABLE` correspondente contra qualquer banco MySQL que já exista e que você

queira continuar usando (local de dev, ou qualquer outro).

Esquecer o passo 2 resulta em uma exceção do EF Core em runtime (`Unknown column '...' in 'field list'`) na primeira query que tocar esse campo — não em um erro de compilação, porque o C# compila normalmente.

4.3 `ApiDbContext` — como o mapeamento funciona

`Summit.Api/ApiDbContext.cs` é o único lugar que sabe a forma exata do banco. Alguns padrões que se repetem para cada entidade:

4.3.1 Enums viram `int`

```
user.Property(u => u.TeamRole).HasConversion<int>();
tour.Property(t => t.Status).HasConversion<int>();
tt.Property(x => x.CheckIn).HasConversion<int>();
round.Property(r => r.Side).HasConversion<int>();
```

Por padrão, o EF Core já mapeia `enum` para `int` automaticamente — essas linhas são explícitas principalmente por clareza/documentação e para blindar contra qualquer mudança futura de comportamento padrão. Sem essa conversão (ou se alguém trocasse para `HasConversion<string>()`), o valor gravado mudaria de tipo e quebraria dados já existentes — outro motivo para nunca reordenar os membros de um `enum` que já tem dados gravados (ver §4.6).

4.3.2 Propriedades computadas são explicitamente ignoradas

```
tour.Ignore(t => t.IsRegistered);
tour.Ignore(t => t.MapPool);
tour.Ignore(t => t.RegistrationClosesAt);
// ... (uma dezena de outras)
```

Toda propriedade *getter-only* dos Models (ver §3.9) precisa aparecer aqui, ou o EF Core tenta mapeá-la como coluna e falha na criação do schema (porque não tem setter). Isso é um ponto de atenção real ao adicionar uma propriedade computada nova em qualquer Model: **se você esquecer de adicionar o `.Ignore(...)` correspondente, o `EnsureCreated()` da próxima vez que o banco for recriado do zero vai falhar** (em um banco já existente pode nem dar erro imediato, dependendo do tipo, o que torna esse esquecimento ainda mais traiçoeiro de detectar).

4.3.3 Relacionamentos: `HasOne / WithMany` + `OnDelete`

```
user.HasOne(u => u.Team)
    .WithMany(t => t.Members)
    .HasForeignKey(u => u.TeamId)
    .OnDelete(DeleteBehavior.SetNull);
```

`DeleteBehavior.SetNull` aqui significa: se um `Team` for excluído, os `User.TeamId` dos membros viram `null` automaticamente (em vez de o banco recusar o delete, ou de apagar os usuários em cascata — as duas alternativas seriam erradas: excluir um time não deveria excluir contas de jogador). Compare com:

```
inv.HasOne(i => i.Team)
    .WithMany(t => t.Invitations)
    .HasForeignKey(i => i.TeamId)
    .OnDelete(DeleteBehavior.Cascade);
```

Aqui, excluir um `Team` **apaga em cascata** seus convites pendentes — faz sentido, um convite para um time que não existe mais não tem porque existir. Cada relacionamento do `OnModelCreating` foi pensado individualmente sobre qual `DeleteBehavior` é semanticamente correto; não existe uma regra única aplicada cegamente. Uma tabela-resumo do que está configurado hoje:

Relacionamento	Comportamento ao apagar o "pai"
<code>Team</code> → <code>User.TeamId</code>	<code>SetNull</code> (usuário sobrevive, sem time)
<code>Team</code> → <code>TeamInvitation</code>	<code>Cascade</code>
<code>Team</code> → <code>TeamJoinRequest</code>	<code>Cascade</code>
<code>Tournament</code> → <code>TournamentTeam</code>	<code>Cascade</code>
<code>Tournament</code> → <code>BracketRound</code>	<code>Cascade</code>
<code>BracketRound</code> → <code>BracketMatch</code>	<code>Cascade</code>
<code>Match</code> → <code>MatchPlayer</code>	<code>Cascade</code>
<code>User</code> → <code>Friendship</code> (como requester)	<code>Cascade</code>
<code>User</code> → <code>Friendship</code> (como addressee)	<code>NoAction</code>

Relacionamento	Comportamento ao apagar o "pai"
TournamentTeam → TournamentLineupPlayer	Cascade
VetoSession → VetoStep	Cascade
Badge / User → UserBadge	Cascade nos dois lados

O caso de `Friendship` merece nota: os dois lados do relacionamento apontam para `User` (`RequesterId` e `AddresseeId`), e o EF Core/MySQL não permite dois `ON DELETE CASCADE` na mesma tabela apontando ambos para a mesma tabela-pai por caminhos diferentes sem risco de ciclo — por isso um lado é `Cascade` e o outro `NoAction`. Isso é uma limitação técnica do banco, não uma escolha de negócio; na prática ambos os lados deveriam se comportar igual (uma amizade some se qualquer um dos dois usuários for excluído), mas hoje só o lado `RequesterId` limpa automaticamente.

4.4 Tabela por tabela

users

A entidade central. Guarda tanto identidade (`SteamId`, `Nickname`) quanto estatísticas agregadas (`KD`, `WinRate`, `TotalKills` ...) diretamente na linha do usuário — **não há uma tabela separada de "stats por partida" agregada em views; as estatísticas mostradas em telas como Perfil ou Ranking vêm direto dessas colunas**, que hoje são preenchidas apenas pelo `SeedData.cs` (dados de demonstração) porque não existe (ainda) nenhum processo que recalcule essas colunas a partir de partidas reais — ver [Capítulo 21](#). Índice único em `SteamId` (um usuário por conta Steam) e índice não-único em `Nickname` (para busca).

teams

Índice único em `Tag` (a sigla do time, ex. "NAVI", nunca duplicada no sistema todo). `CaptainId` é redundante com `User.TeamId + TeamRole.Captain` — o dono "de verdade" é sempre quem tem `TeamRole.Captain` entre os membros; `Team.CaptainId` é mantido em sincronia manualmente toda vez que a propriedade muda (transferência, saída do dono) mas não tem constraint de banco garantindo essa consistência. Isso é um ponto de atenção: um bug futuro que esqueça de atualizar `Team.CaptainId` num fluxo novo criaria uma divergência silenciosa entre "quem o time diz que é dono" e "quem realmente tem o cargo".

teaminvitations

Convite unidirecional criado pelo dono para um jogador específico. `Status` é o enum `TeamInvitationStatus` (`Pending/Accepted/Declined/Cancelled`). Índice composto (`TeamId`, `InvitedUserId`) acelera a checagem de "já existe convite pendente desse time para essa pessoa?".

teamjoinrequests

O espelho inverso do convite: o jogador pede para entrar, o dono aceita/recusa. Mesmos quatro status mais `Expired` (`JoinRequestStatus`, definido em `Models/Competition.cs`) — embora hoje nada no código automaticamente marque uma solicitação como `Expired`; esse valor existe no enum mas não tem produtor

ainda.

friendships

Uma linha por par de usuários, direção fixada em `RequesterId / AddresseeId` mesmo depois de aceita — index único `(RequesterId, AddresseeId)` impede duplicata **nessa ordem específica**, mas a query de relação (`GET /api/friends/relation`) sempre checa os dois sentidos (`(Requester=A,Addressee=B)` OR `(Requester=B,Addressee=A)`) porque uma amizade entre A e B pode ter sido criada por qualquer um dos dois. `Status = Blocked` reaproveita a mesma tabela e o mesmo enum (`FriendshipStatus`) em vez de criar uma tabela de bloqueio separada — ver [Capítulo 14](#) para os detalhes dessa decisão.

tournaments

A configuração completa de um campeonato — nome, formato, datas, regras de série (MD1/MD3/MD5), map pool como `MapPoolCsv` (uma string `"Mirage, Inferno, Nuke"`, nunca uma tabela separada de mapas — ver 4.5). Não tem chave estrangeira nenhuma (é a raiz da árvore de campeonato).

tournamentteams

A tabela de junção entre `tournaments` e `teams` — mas carrega muito mais que uma junção simples: `Seed` (posição no sorteio), `IsEliminated`, `FinalPosition`, o status de `CheckIn`, e `CaptainUserId` (o capitão *da escalação*, um conceito diferente do dono do time — ver [Capítulo 17](#)). Índice único `(TournamentId, TeamId)` — um time só pode se inscrever uma vez no mesmo campeonato.

tournamentlineuplayers

Junção simples entre `tournamentteams` e `users`: quem está na escalação daquele time, naquele campeonato específico. Índice único `(TournamentTeamId, UserId)`.

bracketrounds / bracketmatches

Uma rodada (`bracketrounds`) tem N partidas (`bracketmatches`). `RoundNumber` não é necessariamente sequencial visualmente — é usado como namespace para diferenciar Upper (1, 2, 3...), Lower (101, 102...) e Grande Final (200) dentro do mesmo campeonato, todos ordenáveis por esse número. `Side` (adicionado depois, via `ALTER TABLE`) diz a qual das três subchaves a rodada pertence. `BracketMatch.MatchId` é o link opcional para a tabela `matches` (só existe depois que o veto termina e a sala é criada). Ver [Capítulo 18](#) para a lógica completa de geração.

matches / matchplayers

`matches` é tanto a "sala pré-jogo" (com `ServerIp / ServerPassword / ProvisionState` enquanto o servidor ainda está subindo) quanto o "resultado pós-jogo" (`ScoreA / ScoreB`, `Status = Finished`) — o mesmo registro serve às duas fases da vida de uma partida, sem tabela separada para cada uma. `matchplayers` guarda o scoreboard por jogador (kills, deaths, HS%, rating, MVP), com índice único `(MatchId, UserId)`.

badges / userbadges

`badges` é o catálogo (fixo, vem do `SeedData`); `userbadges` é quais o usuário desbloqueou e quando. Hoje `userbadges` só é populado manualmente pelo seed — não existe lógica de "conceder badge quando X acontece" (ver [Capítulo 21](#)).

vetosessions / vetosteps

`vetosessions`: uma por `BracketMatch` (índice único em `BracketMatchId` — só pode haver um veto por partida). `vetosteps`: cada ban/pick/decider individual, ordenado por `Order`, índice único (`SessionId`, `Order`) — a sequência nunca pode ter dois passos com o mesmo número dentro da mesma sessão. Ver [Capítulo 19](#).

auditlogs

Log de auditoria, sem chave estrangeira para nada — grava ids como texto solto (`ActorUserId`, `TargetUserId`, `TeamId`, `TournamentId` são todos `varchar / longtext` sem `FOREIGN KEY`). Essa é uma escolha deliberada: um log de auditoria deve sobreviver mesmo que a entidade referenciada seja excluída depois (você quer conseguir ver "fulano excluiu o time X" mesmo que o time X já não exista mais no banco) — se fossem FKs de verdade, isso quebraria ou exigiria `ON DELETE SET NULL` em tudo.

poolservers

Sem nenhuma FK — é conceitualmente independente do resto do domínio (representa infraestrutura, não dado de produto). `CurrentMatchId` é apenas um id de texto solto pelo mesmo motivo do log de auditoria: um servidor de pool deve continuar existindo e sendo rastreável mesmo que a partida associada já tenha terminado e potencialmente sido limpa.

4.5 Um padrão recorrente: listas guardadas como CSV, não como tabela filha

Repare em dois campos: `Tournament.MapPoolCsv` (`"Mirage, Inferno, Nuke, Ancient..."`) e `VetoSession.MapPoolCsv`. Em vez de uma tabela `tournament_maps` com uma linha por mapa, o pool inteiro é guardado como uma única string separada por vírgula, e convertido para `List<string>` em uma propriedade computada (ignorada pelo EF Core, ver §4.3.2):

```
public List<string> MapPool => string.IsNullOrEmpty(MapPoolCsv)
    ? new List<string>()
    : MapPoolCsv.Split(',', StringSplitOptions.RemoveEmptyEntries | StringSplitOptions.TrimEntries).ToList();
```

Essa é uma simplificação deliberada: o map pool nunca precisa ser consultado individualmente (`WHERE algum_mapa = 'Mirage'` nunca acontece em lugar nenhum do código) — ele só é lido inteiro, sempre junto, então uma tabela filha normalizada adicionaria uma junção sem nenhum benefício de consulta. Se um dia o produto precisar, por exemplo, filtrar campeonatos por "contém o mapa X", essa modelagem precisaria mudar para uma tabela de fato.

4.6 Cuidado ao mexer em `enum`s persistidos

Como os enums são gravados como `int` (a posição do membro, não o nome), **a ordem dos membros importa e nunca deve ser reordenada** depois que existem dados gravados. Por exemplo:

```
public enum TeamRole { Member = 0, ViceCaptain = 1, Captain = 2 }
```

Se alguém reordenasse para `{ Captain = 0, ViceCaptain = 1, Member = 2 }`, todo `TeamRole` já gravado no banco passaria a significar outra coisa (um dono viraria membro comum silenciosamente, sem nenhum erro visível). A prática segura ao adicionar um novo valor é sempre **acrescentar no fim** (ou atribuir o número explicitamente, como já é feito em todos os enums deste projeto — repare que todo `enum` do `Models/` já declara os valores numéricos explicitamente, ex. `Pending = 0, Accepted = 1, ...`, exatamente para tornar esse risco visível e intencional em vez de implícito).

4.7 Como recriar o banco do zero (fluxo de dev)

```
mysql -u root -e "CREATE DATABASE summit CHARACTER SET utf8mb4"
mysql -u root summit < database/schema.sql
```

Isso aplica o dump de referência. Na prática, durante desenvolvimento normal, basta apagar o banco (`DROP DATABASE summit;`) e deixar a própria API recriá-lo via `EnsureCreated()` na próxima subida — o `schema.sql` é útil principalmente como **documentação legível** do estado do schema e como forma de recriar rapidamente sem subir a API primeiro. Ele não é regenerado automaticamente quando o `ApiDbContext` muda — se você adicionar uma tabela/coluna, o `schema.sql` do repositório fica desatualizado até alguém rodar um novo `mysqldump --no-data` e substituir o arquivo manualmente.

Capítulo 5 — MVVM na Prática

O Capítulo 3 já apresentou o esqueleto de `BaseViewModel` e `RelayCommand`. Este capítulo aprofunda como esses blocos se combinam no dia a dia, com o ciclo de vida completo de um ViewModel típico.

5.1 O ciclo de vida padrão de um ViewModel

Quase todo ViewModel do projeto segue esta receita:

1. Campos privados com estado (`_isLoading`, `_team`, etc.), expostos como propriedades públicas via `SetProperty`.
1. Um construtor que:
 - a. Instancia os `RelayCommand`s.
 - b. Dispara um carregamento assíncrono **sem esperar** (`_ = LoadAsync()`).
1. Um método privado `LoadAsync()` (ou nome equivalente) que marca `IsLoading = true`, busca dados via `App.XyzService` /repositório, popula as propriedades, e marca `IsLoading = false`.

Exemplo completo e representativo (`ViewModels/TeamProfileViewModel.cs`):

```
public class TeamProfileViewModel : BaseViewModel
{
    private Team? _team;
    private bool _isLoading = true;

    public Team? Team { get => _team; set { SetProperty(ref _team, value);
        OnPropertyChanged(nameof(HasTeam)); } }
    public bool IsLoading { get => _isLoading; set => SetProperty(ref _isLoading, value); }
    public bool HasTeam => Team != null;

    public RelayCommand OpenPlayerCommand { get; }

    public TeamProfileViewModel(string teamId)
    {
        OpenPlayerCommand = new RelayCommand(p => { if (p is string id) App.Navigation.NavigateTo(new
        PlayerProfileViewModel(id)); });
        _ = LoadAsync(teamId);
    }

    private async Task LoadAsync(string teamId)
    {
        IsLoading = true;
        Team = await App.TeamService.GetTeamAsync(teamId);
        IsLoading = false;
    }
}
```

5.1.1 Por que `_ = LoadAsync();` e não `await`?

O construtor de uma classe C# não pode ser `async`. Como toda tela precisa buscar dados da API antes de ter algo para mostrar, o padrão do projeto é: o construtor **dispara** a tarefa assíncrona e a descarta explicitamente (`_ = ...`, satisfazendo o analisador estático que avisaria sobre uma `Task` não aguardada), e a UI simplesmente começa mostrando o estado "vazio"/"carregando" até a tarefa completar e disparar `PropertyChanged` para as propriedades que ela populou. Isso é o que dá aos ViewModels a sensação de "a tela aparece na hora e os dados chegam logo depois", sem nenhum spinner bloqueante de tela cheia — cada tela decide como mostrar `IsLoading` (ou nem mostra, e só aparece vazio por um instante).

Consequência importante para quem for escrever um ViewModel novo: se um construtor recebe parâmetros (como `TeamProfileViewModel(string teamId)`), o `LoadAsync` precisa recebê-los também como parâmetro do método (não pode depender de os campos já estarem populados, porque o `LoadAsync` roda de forma concorrente com o resto do construtor terminando).

5.2 Notificação manual de propriedades derivadas

Como não há nenhum framework reativo por trás, toda vez que uma propriedade "resumo" depende de outra, o código precisa **disparar `OnPropertyChanged` da derivada manualmente** dentro do setter da propriedade base. Isso aparece em praticamente toda tela com contagem ou rótulo calculado:

```
// ViewModels/LineupViewModel.cs
public int SelectedCount => Members.Count(m => m.IsSelected);
public string SelectedCountLabel => $"{SelectedCount}/{RequiredCount} SELECCIONADOS";

private void NotifyCount()
{
    OnPropertyChanged(nameof(SelectedCount));
    OnPropertyChanged(nameof(SelectedCountLabel));
}
```

`NotifyCount()` é chamado manualmente depois de qualquer ação que altera a seleção (`ToggleSelect`, `LoadAsync`). Esquecer de chamá-lo depois de uma mudança de estado é o bug de UI mais comum e mais fácil de cometer neste padrão — o valor em memória muda corretamente, mas a tela simplesmente não atualiza até algo mais disparar um refresh geral.

5.3 Itens de lista com estado próprio: `BaseViewModel` também é usado para itens

Quando uma lista precisa que **cada item** tenha estado interativo próprio (selecionado, expandido, etc.), o item também vira uma classe que herda `BaseViewModel` — não é só um DTO estático:

```
// ViewModels/LineupViewModel.cs
public class LineupMemberItem : BaseViewModel
{
    public User User { get; init; } = null!;
    private bool _isSelected;
    private bool _isCaptainChoice;
    public bool IsSelected { get => _isSelected; set => SetProperty(ref _isSelected, value); }
    public bool IsCaptainChoice { get => _isCaptainChoice; set => SetProperty(ref _isCaptainChoice, value); }
}
```

Isso permite que o XAML faça binding direto a `IsSelected` de um item individual dentro de um `ItemsControl` / `ListBox`, e que clicar num item dispare `PropertyChanged` só daquele item — sem precisar recriar a lista inteira (`Members = new List<...>(...)`) toda vez que uma seleção muda. O mesmo padrão aparece em `SidebarItem` (`MainShellViewModel.cs`, cada item do menu lateral tem seu próprio `IsSelected`) e em `FilterItem` (`TournamentsViewModel.cs`, cada chip de filtro).

Exemplo genérico do porquê isso importa (não é código do projeto): se `LineupMemberItem` fosse uma `class` comum sem `INotifyPropertyChanged`, marcar `item.IsSelected = true` não avisaria a UI de nada — a única forma de atualizar a tela seria reatribuir a coleção inteira (`Members = Members.ToList()`), que é mais caro e mais fácil de esquecer.

5.4 Reatividade a eventos globais (troca de usuário, navegação)

Alguns ViewModels persistentes (o shell principal, por exemplo) precisam reagir a mudanças que acontecem "por fora" deles — o usuário atual mudou (login/logout), ou a navegação foi para outra tela. Isso é feito assinando eventos expostos pelos services globais:

```
// ViewModels/MainShellViewModel.cs
App.UserService.CurrentUserChanged += (_, _) =>
{
    OnPropertyChanged(nameof(UserNickname));
    OnPropertyChanged(nameof(UserRank));
    OnPropertyChanged(nameof(UserLevel));
    OnPropertyChanged(nameof(UserAvatarUrl));
    OnPropertyChanged(nameof(HasAvatar));
};

App.Navigation.CurrentViewChanged += (_, vm) =>
{
    if (vm == null) return;
    var title = vm switch
    {
        TournamentDetailsViewModel => "CAMPEONATO",
        PlayerProfileViewModel      => "JOGADOR",
        // ...
        _ => PageTitle
    };
    CurrentView = vm;
    PageTitle   = title;
};
```

Esse segundo bloco é também o mecanismo real de troca de título da barra superior: sempre que qualquer parte do app navega para um novo ViewModel, o `MainShellViewModel` decide o título via um `switch` de tipo sobre o ViewModel de destino. Adicionar uma tela nova que precisa de um título próprio na barra superior significa adicionar um `case` aqui — é um dos poucos lugares "centrais" que precisa ser tocado ao criar uma tela nova (junto com o `DataTemplate` em `App.xaml`, ver [Capítulo 6](#)).

5.5 Estado de edição "inline" (padrão editar/salvar/cancelar)

Várias telas (Perfil, Time) implementam edição inline sem modal: um booleano `IsEditing` alterna entre modo leitura e modo formulário, com campos "de rascunho" separados dos campos "confirmados" até o salvamento:

```
// ViewModels/ProfileViewModel.cs (resumido)
private bool _isEditing;
private string _editBio = string.Empty;

public bool IsEditing { get => _isEditing; set { SetProperty(ref _isEditing, value);
OnPropertyChanged(nameof(IsNotEditing)); } }
public string EditBio { get => _editBio; set => SetProperty(ref _editBio, value); }

private void StartEdit()
{
    EditBio = Bio;           // copia o valor atual pro campo de rascunho
    IsEditing = true;
}

private async Task SaveAsync()
{
    await App.UserService.UpdateBioAsync(EditBio);
    IsEditing = false;
    OnPropertyChanged(nameof(Bio)); // Bio é computado a partir de App.UserService.CurrentUser
}

private void CancelEdit() => IsEditing = false; // simplesmente descarta EditBio, nunca foi salvo
```

O truque é que `Bio` (modo leitura) e `EditBio` (modo formulário) são propriedades **diferentes** — cancelar a edição não precisa "desfazer" nada porque o valor real (`Bio` , lido de `App.UserService.CurrentUser`) nunca foi tocado até `SaveAsync` de fato chamar a API. O mesmo padrão, com mais campos, aparece em `TeamViewModel` para editar o time (`EditTeamName` / `EditTeamDescription` /...) e para o fluxo de confirmação de exclusão (`ConfirmingDelete` , um booleano simples que alterna um painel de "tem certeza?").

5.6 Timers para simular tempo real (polling)

Como o sistema não tem WebSocket/SignalR (ver [Capítulo 2](#)), qualquer tela que precisa parecer "ao vivo" usa um `DispatcherTimer` do WPF chamando o mesmo método de recarga em intervalos curtos. O exemplo mais claro é a sala de partida durante o veto:

```
// ViewModels/MatchRoomViewModel.cs
_timer = new DispatcherTimer { Interval = TimeSpan.FromSeconds(3) };
_timer.Tick += async (_, _) => await RefreshAsync();
_timer.Start();
_ = RefreshAsync();
```

`DispatcherTimer` (em vez de um `System.Threading.Timer` comum) é usado especificamente porque seu callback já roda na *dispatcher thread* da UI do WPF — ou seja, pode atualizar propriedades ligadas ao XAML diretamente, sem precisar de `Dispatcher.Invoke` manual. O timer é parado explicitamente quando o usuário sai da tela (`BackCommand` chama `_timer.Stop()`) ou quando o fluxo termina (assim que a sala fica

pronta, `MatchRoomViewModel` chama `_timer.Stop()` dentro do próprio `RefreshAsync()` — esquecer de parar um `DispatcherTimer` ao navegar para longe da tela deixaria ele rodando (e fazendo requisições HTTP) indefinidamente em segundo plano, então esse `Stop()` é um detalhe de correção, não só de performance.

5.7 Views: o quanto de código-behind é aceitável

O `.xaml.cs` de toda `View` é, por convenção do projeto, praticamente vazio:

```
// Views/AuditLogView.xaml.cs (padrão típico, se repete em quase toda View)
public partial class AuditLogView : UserControl
{
    public AuditLogView() => InitializeComponent();
}
```

A única exceção regular é `Components/LevelBadge.xaml.cs`, que **não é uma View de tela** — é um `UserControl` reutilizável com `DependencyProperty`s próprias (`Level`, `Size`, `ShowTier`) e lógica de desenho (escolher cor/nome de "tier" a partir do nível numérico). Isso é aceitável porque é lógica de **apresentação pura de um componente visual reutilizável**, sem nenhuma regra de negócio ou chamada de rede — a distinção que o projeto mantém é: code-behind pode conter lógica de desenho/layout de um controle puramente visual, mas nunca lógica de aplicação (chamadas HTTP, regra de permissão, navegação) — isso sempre mora no `ViewModel`.

Capítulo 6 — Navegação e Comunicação com a API

6.1 `NavigationService` — uma pilha, não um `Frame`

WPF tem um controle de navegação nativo (`Frame` / `Page`), mas o Summit não o usa. Em vez disso, `Services/NavigationService.cs` implementa uma pilha de histórico manual sobre `BaseViewModel`:

```
public class NavigationService
{
    private BaseViewModel? _currentView;
    private readonly Stack<BaseViewModel> _history = new();

    public BaseViewModel? CurrentView
    {
        get => _currentView;
        private set { _currentView = value; CurrentViewChanged?.Invoke(this, value); }
    }

    public bool CanGoBack => _history.Count > 0;
    public event EventHandler<BaseViewModel?>? CurrentViewChanged;

    public void NavigateTo(BaseViewModel viewModel)
    {
        if (_currentView != null) _history.Push(_currentView);
        CurrentView = viewModel;
    }

    public void GoBack()
    {
        if (_history.Count == 0) return;
        CurrentView = _history.Pop();
    }
}
```

Navegar para a frente empilha o ViewModel atual antes de trocar; voltar desempilha. Não existe navegação "para frente" depois de voltar (diferente de um browser) — uma vez que você chama `GoBack()`, o ViewModel que você tinha "avançado" antes é descartado, não fica disponível para um "avançar" futuro. Isso é suficiente para o produto porque toda a navegação do Summit é estritamente hierárquica (lista → detalhe → sub-detalhe), sem necessidade de navegação lateral complexa.

Ponto de atenção: `NavigateTo` sempre cria uma **instância nova** do ViewModel de destino (`new TeamProfileViewModel(id)`), nunca reaproveita uma existente. Isso significa que toda navegação recarrega os dados do zero (o construtor do novo ViewModel dispara seu próprio `LoadAsync`, ver [Capítulo 5](#)) — o que é o comportamento certo aqui (você quer ver dados frescos ao entrar numa tela de novo), mas também significa que **nenhum estado de tela sobrevive a uma navegação embora a instância antiga**

ainda exista na pilha `_history` até você voltar para ela (nesse momento ela reaparece com qualquer estado que tinha antes de você sair, porque o objeto C# nunca foi descartado — só não estava sendo mostrado).

6.2 De ViewModel para tela: `DataTemplate` implícito

Quando `NavigationService.CurrentView` muda, quem realmente decide "o que desenhar na tela" é o WPF, através da tabela de `DataTemplate`s registrada globalmente em `App.xaml`:

```
<DataTemplate DataType="{x:Type vm:TeamProfileViewModel}">
  <views:TeamProfileView/>
</DataTemplate>
```

O shell principal (`MainShellView.xaml`) tem um `ContentControl` cujo `Content` está ligado a `MainShellViewModel.CurrentView`. Quando esse `Content` é, por exemplo, uma instância de `TeamProfileViewModel`, o WPF procura na tabela de `DataTemplate`s um template cujo `DataType` seja esse tipo exato, encontra `TeamProfileView`, instancia e a exibe com `DataContext = <a instância de TeamProfileViewModel>` automaticamente.

Checklist para adicionar uma tela nova ao projeto (o "cerimonial" mínimo, sempre os mesmos três passos):

1. Criar o ViewModel em `ViewModels/NovaTelaViewModel.cs` (herdando `BaseViewModel`).
2. Criar a View em `Views/NovaTelaView.xaml` + `.xaml.cs` (o `.xaml.cs` só chama

`InitializeComponent()`).

1. Registrar o par em `App.xaml`:

```
` xml    &lt;DataTemplate    DataType="{x:Type    vm:NovaTelaViewModel}"&gt;    &lt;views:NovaTelaView/&gt;
&lt;/DataTemplate&gt; `
```

Se você esquecer o passo 3, a navegação não vai lançar exceção — o WPF simplesmente não encontra template e renderiza o `ToString()` do ViewModel como texto puro. Esse é o sintoma mais comum de "esqueci de registrar o `DataTemplate`": uma tela que deveria aparecer com UI vira uma linha de texto crua (o nome completo da classe) na tela.

Opcionalmente, um quarto passo: se a tela precisa de um título específico na barra superior, adicionar um `case` no `switch` de `MainShellViewModel` (ver §5.4).

6.3 `ApiClient` — referência completa dos métodos

`Services/ApiClient.cs` é o único ponto de saída HTTP do client inteiro — nenhum repositório ou service chama `HttpClient` diretamente, todos passam por aqui. Um único `HttpClient` estático é compartilhado por toda a aplicação (a prática recomendada pela Microsoft, para evitar esgotamento de sockets que aconteceria criando um `HttpClient` novo por requisição):


```
public static readonly string BaseUrl =
    Environment.GetEnvironmentVariable("SUMMIT_API_URL")?.TrimEnd('/') ?? "http://localhost:5180";

private static readonly HttpClient Http = new()
{
    BaseAddress = new Uri(BaseUrl),
    Timeout = TimeSpan.FromSeconds(15)
};
```

Tabela de referência de todos os métodos e quando usar cada um (a lógica de design já foi explicada em §3.6; aqui está o catálogo completo):

Método	Verbo	Em falha/erro devolve	Uso típico
<code>GetAsync<T>(path)</code>	GET	<code>default(T)</code> (ex. <code>null</code> , ou lista deve ser tratada com <code>?? new()</code> por quem chama)	Buscar qualquer recurso de leitura
<code>PostAsync<T>(path, body)</code>	POST	<code>default(T)</code>	Criar algo e usar o objeto criado de volta (ex. convite)
<code>PostRequiredAsync<T>(path, body)</code>	POST	lança <code>InvalidOperationException</code>	Login — falha precisa parar o fluxo e ser mostrada
<code>PostBoolAsync(path, body?)</code>	POST	<code>false</code>	Ações que só importam sucesso/falha (aceitar convite, promover, etc.)
<code>PutAsync<T>(path, body)</code>	PUT	<code>default(T)</code>	Atualizar e receber o objeto atualizado (ex. editar time)
<code>PutWithMessageAsync(path, body)</code>	PUT	(<code>false</code> , mensagem da API ou "Erro de conexão.")	Atualizar quando a API pode recusar com um motivo específico (ex. escalção)
<code>DeleteBoolAsync(path)</code>	DELETE	<code>false</code>	Excluir/remover (time, amizade)

Todos usam a mesma configuração de serialização:

```
private static readonly JsonSerializerOptions Json = new(JsonSerializerDefaults.Web)
{
    ReferenceHandler = ReferenceHandler.IgnoreCycles
};
```

`ReferenceHandler.IgnoreCycles` existe porque os Models compartilhados têm referências bidirecionais em memória (`Team.Members` → `User.Team` → de volta ao mesmo `Team`) — sem essa opção, a serialização JSON entraria em loop infinito na primeira entidade com referência circular. A API usa a mesma configuração do lado dela (`builder.Services.ConfigureHttpJsonOptions(o => o.SerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles)` em `Program.cs`) — os dois lados precisam concordar nessa opção, ou o formato do JSON trocado diverge.

6.3.1 Exemplo genérico: como escrever um novo método de repositório

Se você precisar adicionar uma chamada nova (não é código real do projeto, é um exemplo de referência para seguir o padrão):

```
// Um repositório fictício de "convites de campeonato" seguindo o padrão do projeto:
public class TournamentInviteRepository
{
    public Task<bool> SendAsync(string tournamentId, string teamId)
        => ApiClient.PostBoolAsync($"api/tournaments/{tournamentId}/invite", new { teamId });

    public async Task<List<TournamentInvite>> GetPendingAsync(string teamId)
        => await ApiClient.GetAsync<List<TournamentInvite>>($"api/teams/{teamId}/tournament-invites") ??
        new();
}
```

Note o `?? new()` no segundo método — essa é a convenção do projeto para todo método que retorna uma `List<T>`: nunca deixar `GetAsync` devolver `null` para quem chama, sempre normalizar para lista vazia ali mesmo no repositório (evita que todo consumidor precise checar `null` de novo).

6.4 Autenticação: `SteamAuthService`, `SteamConfig`, `SteamWebApiClient`

6.4.1 O fluxo OpenID (login real via Steam)

O Summit não usa OAuth — usa **Steam OpenID 2.0**, o protocolo legado que a Steam ainda suporta para "Login com Steam" sem precisar registrar uma aplicação OAuth. O fluxo, implementado em `Services/SteamAuthService.LoginWithSteamAsync()`:

1. Abre um `HttpListener` local numa porta livre aleatória (`GetFreePort()`), escutando em

`http://127.0.0.1:{port}/auth/steam/callback/`.

1. Monta a URL de autenticação da Steam (`BuildAuthUrl`) com esse endereço de callback e abre no

navegador padrão do usuário (`Process.Start` com `UseShellExecute = true`).

1. O usuário faz login no site da Steam normalmente (fora do app — na aba do navegador).
2. A Steam redireciona o navegador de volta para `http://127.0.0.1:{port}/...`, que o

`HttpListener` local está esperando — ao receber essa requisição, o app já tem a resposta (com um timeout de 2 minutos, `LoginTimeout`, caso o usuário nunca complete o login).

1. Os parâmetros `openid.*` da query string são **revalidados diretamente com a Steam**

(`ValidateWithSteamAsync`, um POST de volta para `steamcommunity.com/openid/login` com `openid.mode=check_authentication`) — isso é essencial, porque sem essa segunda chamada qualquer um poderia forjar um redirecionamento local alegando ser um SteamID arbitrário.

1. Extraí o SteamID64 da URL `claimed_id` via regex, busca nome/avatar reais

(`SteamWebApiClient.GetPlayerSummaryAsync`), e faz upsert do usuário na API (`UserRepository.UpsertFromSteamAsync`).

1. Salva a sessão localmente (`SessionStore.Save`) para restaurar automaticamente da próxima vez.

```
var claimedId = query["openid.claimed_id"];
var match = SteamIdRegex.Match(claimedId); // ^https?://steamcommunity\.com/openid/id/(\d{17})$
var steamId64 = match.Groups[1].Value;
if (!await ValidateWithSteamAsync(query)) return null;
```

6.4.2 `SteamWebApiClient` — nome e avatar reais, com fallback sem chave de API

Buscar nome de exibição e avatar reais da Steam precisa de uma API Key oficial (`ISteamUser/GetPlayerSummaries`). Como nem todo ambiente de desenvolvimento tem uma configurada, `SteamWebApiClient.GetPlayerSummaryAsync` tenta dois caminhos em cascata:

```
public async Task<SteamPlayerSummary?> GetPlayerSummaryAsync(string steamId64)
=> await GetViaWebApiAsync(steamId64)
?? await GetViaCommunityXmlAsync(steamId64);
```

1. **Via Web API oficial** (`GetViaWebApiAsync`) — só tenta se `SteamConfig.GetApiKey()` achar

uma chave configurada; devolve `null` de propósito se não achar (não é erro, é "pula essa opção").

1. **Fallback via XML público do perfil** (`GetViaCommunityXmlAsync`) —

`steamcommunity.com/profiles/{id}?xml=1`, que não precisa de chave nenhuma, mas só funciona se o perfil da Steam do usuário for público.

Se os dois falharem (perfil privado + sem API Key), o login ainda funciona — só usa um nome genérico gerado localmente: `$"Player_{steamId64[^4..]}"` (os 4 últimos dígitos do SteamID64), visto em `SteamAuthService.LoginWithSteamAsync`.

6.4.3 `SteamConfig` — onde a chave de API é lida

```
public static string? GetApiKey()
{
    var env = Environment.GetEnvironmentVariable("SUMMIT_STEAM_API_KEY");
    if (!string.IsNullOrEmpty(env)) return env.Trim();
    // fallback: arquivo %LOCALAPPDATA%\Summit\steam.config
}
```

Mesma filosofia do resto do projeto: variável de ambiente primeiro; se ausente, um arquivo local opcional (`steam.config`, texto puro com a chave) — útil para configurar a chave uma vez na máquina de desenvolvimento sem precisar redefinir a variável de ambiente em toda sessão de terminal nova.

6.4.4 `SessionStore` — persistência local da sessão

Um JSON simples em `%LOCALAPPDATA%\Summit\session.json`, guardando só o `SteamId`:

```
public static void Save(string steamId) { /* grava { SteamId, SavedAt } */ }
public static string? Load() { /* lê e devolve SteamId, ou null se não existir/corrompido */ }
public static void Clear() { /* apaga o arquivo (usado no logout) */ }
```

No próximo início do app, `SteamAuthService.TryRestoreSessionAsync()` lê esse `SteamId`, busca o `User` correspondente na API (`UserRepository.GetBySteamIdAsync`) e — se achar — já loga automaticamente sem passar pelo fluxo OpenID de novo. Repare que essa restauração também dispara `RefreshSummaryAsync` **sem aguardar** (fire-and-forget), que busca nick/avatar atualizados da Steam em segundo plano e só grava de volta na API se algo realmente mudou — a sessão restaura instantaneamente com os dados já em cache, e se atualiza silenciosamente depois.

6.4.5 Modo demo

`SteamAuthService.LoginDemoAsync()` existe para testar o app sem depender da Steam nem de internet: cria/atualiza um usuário fixo (`SteamId` = `"76561198000000000"`, nick `xGhostFrag`) com estatísticas hardcoded, e loga com ele. É o botão "entrar em modo demo" da tela de login (`LoginViewModel.LoginDemoCommand`).

Capítulo 7 — Modelos do Client

O [Capítulo 4](#) já cobriu os `Models/*.cs` do ponto de vista do banco de dados (o que é persistido, como). Este capítulo olha para os mesmos arquivos do ponto de vista de quem consome no client — e cataloga também uma segunda categoria de tipo que **não é compartilhada com a API**: pequenos DTOs de apresentação que vivem dentro dos próprios arquivos de ViewModel.

7.1 Duas categorias de "modelo" no client

1. **Modelos compartilhados** (`Models/*.cs`): vêm de/vão para a API via JSON, têm o mesmo formato nos dois lados (ver [§2.1.1](#)). Exemplos: `User`, `Team`, `Tournament`, `Match`, `VetoSession`.

1. **DTOs de apresentação locais**: classes simples definidas dentro do próprio arquivo de ViewModel que as usa, existem só em memória no client, nunca trafegam como tal pela rede — são *derivadas* de um modelo compartilhado, moldadas especificamente para o que aquela tela precisa mostrar. Exemplos: `MatchListItem`, `ScoreboardRow`, `VetoMapItem`, `BracketColumnViewModel`.

Entender essa distinção evita um erro comum ao explorar o código: procurar `MatchListItem` em `Models/` (não está lá — está em `ViewModels/HomeViewModel.cs` e `ViewModels/MatchesViewModel.cs`, cada arquivo com sua própria cópia da classe, porque as duas telas a usam de forma ligeiramente diferente e nenhuma delas depende da outra).

7.2 Por que existem DTOs de apresentação separados dos modelos compartilhados

Pegue `Match` / `MatchPlayer` (compartilhado) versus `MatchListItem` (local, em `HomeViewModel.cs` e `MatchesViewModel.cs`):

```
// Models/Match.cs — a entidade completa, como vem da API
public class Match
{
    public string TeamAId { get; set; } = string.Empty;
    public List<MatchPlayer> Players { get; set; } = new();
    // ... 15+ campos
}
```

```
// ViewModels/MatchesViewModel.cs – só o que a lista de partidas precisa mostrar
public class MatchListItem
{
    public string Id { get; set; } = string.Empty;
    public string Map { get; set; } = string.Empty;
    public bool Won { get; set; }
    public int Kills { get; set; }
    public string KDA => $"{Kills} / {Deaths} / {Assists}";
    public string WonLabel => Won ? "VITÓRIA" : "DERROTA";
}
```

`MatchListItem` já vem com `Won` calculado (comparando o `TeamSide` do jogador atual com quem venceu) e rótulos textuais prontos (`WonLabel`, `DateLabel`) — o ViewModel faz essa transformação uma vez, ao carregar (`raw.Select(m => new MatchListItem { ... })`), para que o XAML da lista só precise fazer *binding* direto a texto pronto, sem nenhuma lógica ou conversor no XAML. Essa é a razão de existir: **separar "o formato que a API devolve" de "o formato que esta tela específica quer desenhar"**, para que mudanças puramente visuais numa tela não tenham nenhum efeito sobre o modelo de rede.

7.3 Catálogo dos DTOs de apresentação locais

Classe	Onde vive	Constrói a partir de	Usado por
<code>MatchListItem</code>	<code>HomeViewModel.cs</code> e <code>MatchesViewModel.cs</code> (duas cópias independentes)	<code>Match</code> + <code>MatchPlayer</code> do usuário atual	Lista de partidas recentes
<code>ScoreboardRow</code>	<code>MatchDetailsViewModel.cs</code>	<code>MatchPlayer</code>	Placar detalhado pós-partida (dois lados A/B)
<code>VetoMapItem</code>	<code>MatchRoomViewModel.cs</code>	<code>VetoSession</code> / <code>VetoStep</code> / <code>VetoState</code>	Grade de mapas clicável durante o veto
<code>BracketSlotViewModel</code> / <code>BracketColumnViewModel</code>	<code>ViewModels/BracketLayout.cs</code>	<code>BracketRound</code> / <code>BracketMatch</code>	Renderização da chave (ver Capítulo 18)
<code>LineupMemberItem</code>	<code>LineupViewModel.cs</code>	<code>User</code> (membro do time)	Seleção da escalação (ver Capítulo 17)
<code>SidebarItem</code>	<code>MainShellViewModel.cs</code>	estático (ícone/label/comando)	Itens do menu lateral
<code>FilterItem</code>	<code>TournamentsViewModel.cs</code>	estático (nome do filtro)	Chips de filtro de campeonatos

Todos os que precisam de interatividade própria (seleção, clique) herdam `BaseViewModel` (`LineupMemberItem`, `SidebarItem`, `FilterItem`, `BracketSlotViewModel` como exceção — este é `class` simples porque seus dados são só de leitura para desenho, nunca mudam depois de construído). Os que são só exibição de dados já prontos (`MatchListItem`, `ScoreboardRow`, `VetoMapItem`) são `class` comuns — não precisam notificar mudança porque são recriados do zero a cada carregamento, nunca mutados individualmente depois de existir.

7.4 Como um modelo compartilhado carrega lógica de apresentação sem virar DTO local

Nem toda necessidade de apresentação gera um DTO local — muita coisa fica direto no modelo compartilhado como propriedade computada, quando faz sentido em qualquer tela que use aquele modelo (não é específico de uma tela). `Models/Tournament.cs` é o exemplo mais denso disso:

```
public string StatusLabel => Status switch
{
    TournamentStatus.Open      => "INSCRIÇÕES ABERTAS",
    TournamentStatus.InProgress => "EM ANDAMENTO",
    TournamentStatus.Finished   => "ENCERRADO",
    TournamentStatus.Upcoming   => "EM BREVE",
    -                            => ""
};

public string CountdownLabel { get { /* "EM 3 DIAS" / "EM 2H 15M" / "AO VIVO AGORA" / ... */ } }
public string TeamsCountText => $"{RegisteredTeams}/{MaxTeams}";
public double SlotsFillPercent => MaxTeams > 0 ? (double)RegisteredTeams / MaxTeams : 0;
```

Regra prática para decidir onde colocar uma nova propriedade de exibição: se o cálculo só usa campos que já existem no modelo compartilhado e faria sentido em *qualquer* tela que mostre aquele objeto (ex. "está com inscrições abertas?" é útil tanto na lista de campeonatos quanto na tela de detalhes), coloque como propriedade computada direto no `Models/`. Se o cálculo depende de *contexto específico de uma tela* (ex. "esse item de partida foi vitória para o usuário logado agora" — depende de quem está olhando, não é uma verdade universal do objeto `Match`), isso vai num DTO local daquela tela, como visto em `MatchListItem.Won`.

Lembre que toda propriedade computada adicionada a um `Models/*.cs` que é uma entidade EF Core precisa do `.Ignore(...)` correspondente em `ApiDbContext.OnModelCreating` (ver §4.3.2) — esse passo é fácil de esquecer justamente porque, do lado do client, tudo parece funcionar sem ele (o client não sabe nada sobre EF Core); o erro só aparece do lado da API na próxima vez que o schema for recriado do zero.

7.5 `IsRegistered`: um campo "calculado à mão" pelo Service, não pela API

Vale destacar um padrão específico de `Tournament.IsRegistered` porque foge da regra geral acima — ele **não** é uma propriedade computada a partir de outros campos do próprio objeto (não tem como saber, olhando só para um `Tournament`, se o time do usuário atual está inscrito nele). Em vez disso, é um campo mutável comum (`public bool IsRegistered { get; set; }`), com o comentário "set by service when loading for current user" — e de fato é o `TournamentService` do client quem o preenche depois de buscar a lista, com uma chamada extra por campeonato:

```
// Services/TournamentService.cs
private async Task MarkRegisteredAsync(List<Tournament> list)
{
    var teamId = App.UserService.CurrentUser?.TeamId;
    if (string.IsNullOrEmpty(teamId)) return;
    foreach (var t in list)
        t.IsRegistered = await _repo.IsTeamRegisteredAsync(t.Id, teamId);
}
```

Isso é um bom exemplo de que nem toda informação relevante para a tela é um campo "puro" do domínio — `IsRegistered` é fundamentalmente uma pergunta relativa ("registrado *em relação a qual time?*"), então ela é resolvida na camada de Service do client, sob demanda, e anexada ao objeto antes de chegar ao ViewModel. O trade-off explícito aqui é uma chamada HTTP extra por campeonato listado (`N+1`) em troca de manter a API simples (um único endpoint de listagem, sem precisar saber "para qual usuário" a listagem é). Para o volume de dados deste produto (dezenas de campeonatos, não milhares), esse custo é aceitável; seria o primeiro ponto a otimizar se a lista de campeonatos crescesse muito.

Capítulo 8 — Services e Repositórios do Client

Este capítulo cataloga cada repositório (`Data/*.cs`) e service (`Services/*.cs`) do client, descrevendo a responsabilidade de cada método. O padrão geral (repositório = tradução HTTP fina, service = regra de aplicação por cima) já foi explicado em §3.2-3.3; aqui o foco é o "o quê", não o "por quê".

8.1 Repositórios (`Data/`)

UserRepository

Método	Endpoint	Notas
<code>GetBySteamIdAsync(steamId)</code>	<code>GET /api/users/by-steam/{steamId}</code>	usado no login/restauração de sessão
<code>UpsertFromSteamAsync(steamId, nickname, avatarUrl)</code>	<code>POST /api/users/steam-login</code>	cria ou atualiza; usa <code>PostRequiredAsync</code> (falha é fatal para o login)
<code>UpdateAsync(user)</code>	<code>PUT /api/users/{id}</code>	manda o <code>User</code> inteiro — a API sobrescreve campo a campo
<code>SearchAsync(query)</code>	<code>GET /api/users/search?q=</code>	busca por substring de nickname, usado em convites/amizades
<code>GetByIdAsync(userId)</code>	<code>GET /api/users/{id}</code>	
<code>GetByNicknameAsync(nickname)</code>	<code>GET /api/users/by-nickname/{nickname}</code>	usado para convidar/adicionar por nome exato

TeamRepository

Cobre criação, convite, entrada por solicitação, cargos e edição/exclusão — é o repositório mais extenso do projeto porque `Team` é o agregado com mais operações administrativas (ver [Capítulo 13](#) para o fluxo completo de cada uma). Métodos notáveis:

- `CreateAsync` / `InviteAsync` / `AcceptInvitationAsync` / `DeclineInvitationAsync` / `LeaveTeamAsync`
- `GetJoinRequestsAsync` / `CreateJoinRequestAsync` / `AcceptJoinRequestAsync` / `DeclineJoinRequestAsync`
- `PromoteAsync` / `DemoteAsync` / `TransferOwnershipAsync`
- `UpdateAsync` / `DeleteAsync` / `KickAsync`

Todos os métodos que mudam algo recebem um `byUserId` / `ownerId` explícito — o repositório não sabe "quem é o usuário atual", isso é responsabilidade de quem chama (o `TeamService`, que lê `App.UserService.CurrentUser`).

TournamentRepository

Método	Endpoint
<code>GetAllAsync()</code>	<code>GET /api/tournaments</code>
<code>GetByIdAsync(id)</code>	<code>GET /api/tournaments/{id}</code>
<code>RegisterTeamAsync(tournamentId, teamId)</code>	<code>POST /api/tournaments/{id}/register</code>
<code>IsTeamRegisteredAsync(tournamentId, teamId)</code>	<code>GET /api/tournaments/{id}/registered/{teamId}</code>
<code>CheckInAsync(tournamentId, teamId, byUserId)</code>	<code>POST /api/tournaments/{id}/checkin</code>
<code>UpdateLineupAsync(tournamentId, teamId, byUserId, playerIds, captainUserId)</code>	<code>PUT /api/tournaments/{id}/lineup</code> (via <code>PutWithMessageAsync</code> , ver Capítulo 17)

FriendshipRepository

Além do CRUD óbvio de amizade, define um enum local só para representar a relação entre dois usuários vista do client:

```
public enum RelationStatus { None, Friends, OutgoingPending, IncomingPending, Blocked }

public async Task<RelationStatus> GetRelationAsync(string viewerId, string otherId)
{
    var raw = await ApiClient.GetAsync<string>($" /api/friends/relation?viewerId={viewerId}&otherId={otherId}");
    return Enum.TryParse<RelationStatus>(raw, out var status) ? status : RelationStatus.None;
}
```

Repare que a API devolve a relação como **string simples** (`"Blocked"`, `"Friends"`, etc. — ver `GET /api/friends/relation` em `Program.cs`), e o client faz `Enum.TryParse` sobre ela. Esse é um dos poucos lugares do sistema onde o contrato entre client e API é uma string "mágica" em vez de um tipo compartilhado — funciona porque os nomes dos valores são idênticos nos dois lados por convenção, mas não há nada no compilador garantindo isso (se alguém renomeasse um valor só do lado da API, o `TryParse` falharia silenciosamente e cairia no `RelationStatus.None` default, sem erro nenhum). `BlockAsync` / `UnblockAsync` completam o conjunto (`UnblockAsync` reaproveita o mesmo `DELETE /api/friends` do "remover amizade" — ver [Capítulo 14](#)).

MatchRepository, VetoRepository, AuditRepository, BadgeRepository

Repositórios pequenos e diretos — cada um cobre um único domínio de leitura (mais `VetoRepository.ActAsync` para agir no veto). Sem lógica além da tradução para URL.

8.2 Services (`Services/`)

TeamService (implementa `ITeamService`)

A camada de regra por cima de `TeamRepository`. Quase todo método começa checando `App.UserService.CurrentUser` e seu cargo antes de delegar ao repositório — ver o exemplo completo em [§3.3](#). Um detalhe que se repete bastante: vários métodos terminam chamando `ReloadCurrentUserAsync()` :

```
private async Task ReloadCurrentUserAsync()
{
    var me = App.UserService.CurrentUser;
    if (me == null) return;
    var fresh = await _userRepo.GetByIdAsync(me.Id);
    if (fresh != null) App.UserService.SetCurrentUser(fresh);
}
```

Isso existe porque `App.UserService.CurrentUser` é um **snapshot em memória** do usuário — depois de uma ação que muda algo sobre o próprio usuário atual do ponto de vista do servidor (entrar em um time, ser promovido, sair de um time, o time ser excluído), esse snapshot local fica desatualizado até algo buscar o `User` de novo na API e substituir a referência com `SetCurrentUser`. Qualquer método novo em `TeamService` que altere `TeamId` / `TeamRole` do usuário atual **precisa** terminar chamando isso (ou equivalente), senão a UI continua mostrando o estado antigo (ex. ainda mostrando o botão "criar time" depois de já ter entrado em um).

`TournamentService` (implementa `ITournamentService`)

Pequeno — orquestra `TournamentRepository` e adiciona o preenchimento de `IsRegistered` via `MarkRegisteredAsync` (ver §7.5).

`UserService` (implementa `IUserService`)

O "estado de sessão" do app inteiro — guarda `CurrentUser` e expõe o evento `CurrentUserChanged` que o resto da UI escuta (ver §5.4). Os métodos `UpdateBioAsync` / `UpdateRoleAsync` / `UpdateAvatarUrlAsync` / `UpdateCountryAsync` seguem todos o mesmo padrão: mutam `CurrentUser` localmente **primeiro** (otimista) e só depois chamam `App.UserRepository.UpdateAsync(CurrentUser)` para persistir — não há tratamento de erro nesse caminho (se o `PUT` falhar silenciosamente, o client já mostra o valor novo mesmo assim, porque `ApiClient.PutAsync` não lança exceção). Isso é aceitável para campos de perfil de baixo risco, mas é bom saber que esse `Update*Async` **não confirma que a API de fato salvou** antes de a UI já considerar a mudança como feita.

`StatsService` (implementa `IStatsService`)

Combina dados de dois repositórios (`UserRepository` + `MatchRepository`) para montar um `PlayerStats` — é o único service que faz uma "junção" client-side não trivial de duas fontes:

```
public async Task<PlayerStats> GetStatsAsync(string userId)
{
    var user = await _userRepo.GetByIdAsync(userId);
    var matches = await _matchRepo.GetRecentForUserAsync(userId, 12);
    var recent = matches.Select(m => { /* extrai o MatchPlayer do userId dentro de cada Match */ }).ToList();
    return new PlayerStats { /* campos agregados de `user` + RecentPerformance = recent */ };
}
```

`BadgeService` (implementa `IBadgeService`), `RankingService`

Pequenos, sem lógica de negócio própria além de decidir qual endpoint chamar (`GetAllForCurrentUserAsync` chama a variante "com estado" quando há usuário logado, senão a variante "catálogo puro"). `RankingService` não tem interface (ver nota em §3.3 sobre a inconsistência de quais services têm interface).

8.3 Por que algumas interfaces existem e outras não — o que isso significa na prática

As interfaces em `Services/Interfaces/` (`ITeamService`, `ITournamentService`, `IUserService`, `IStatsService`, `IBadgeService`) **não são usadas para injeção de dependência** (não há container de DI no client, ver §3.4). Elas existem hoje como documentação de contrato — e, notavelmente, **estão incompletas**: por exemplo, `ITeamService` declara só `GetTeamAsync` e `CreateTeamAsync`, mas a classe `TeamService` implementa mais de uma dezena de outros métodos públicos (`PromoteAsync`, `KickMemberAsync`, etc.) que não estão na interface. Isso funciona porque em todo o código do projeto, o acesso é sempre via `App.TeamService` (tipo concreto `TeamService`), nunca via uma variável do tipo `ITeamService` — então o compilador nunca reclama dos métodos "extras" que a interface não lista. Se algum dia o projeto passar a usar as interfaces de fato (por exemplo, para permitir testes com mock), esse descompasso precisaria ser resolvido primeiro.

Capítulo 9 — Program.cs e o Bootstrap da API

9.1 A sequência de inicialização

`Summit.Api/Program.cs` segue a estrutura padrão de um app ASP.NET Core moderno (top-level statements, sem `Main` explícito), na ordem:

```
var builder = WebApplication.CreateBuilder(args);

// 1. Escolhe o provedor de banco (MySQL via env, senão SQLite local)
var mysql = Environment.GetEnvironmentVariable("SUMMIT_DB") ??
builder.Configuration.GetConnectionString("MySql");
builder.Services.AddDbContext<ApiDbContext>(o => { /* ... */ });

// 2. Configura serialização JSON (mesma opção do client, ver Capítulo 6)
builder.Services.ConfigureJsonOptions(o => o.SerializerOptions.ReferenceHandler =
ReferenceHandler.IgnoreCycles);

// 3. Registra os três BackgroundServices + o MatchServerService singleton
builder.Services.AddHostedService<LifecycleWorker>();
builder.Services.AddSingleton<MatchServerService>();
builder.Services.AddHostedService<ServerProvisionPoller>();
builder.Services.AddHostedService<PoolManagerService>();

var app = builder.Build();

// 4. Cria o schema (se não existir) e semeia dados de demonstração (se o banco estiver vazio)
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<ApiDbContext>();
    db.Database.EnsureCreated();
    await SeedData.EnsureSeededAsync(db);
}

// 5. Centenas de linhas de app.MapGet/MapPost/MapPut/MapDelete ...

app.MapCompetitionEndpoints(); // 6. registra o segundo arquivo de rotas

app.Run("http://localhost:5180"); // 7. porta fixa, sempre a mesma
```

A porta é **fixa e hardcoded** (`5180`) — não vem de configuração nem variável de ambiente. Isso é consistente com o `ApiClient` do client, cujo padrão (`http://localhost:5180`) é exatamente esse valor; se algum dia a porta precisar ser configurável, os dois lados (aqui e `Services/ApiClient.cs`) precisam mudar juntos, ou usar `SUMMIT_API_URL` do lado do client para apontar para uma porta diferente sem tocar aqui.

9.2 `MatchServerService` como `Singleton`, os workers como `HostedService`

```
builder.Services.AddSingleton<MatchServerService>();
builder.Services.AddHostedService<ServerProvisionPoller>();
builder.Services.AddHostedService<PoolManagerService>();
```

`MatchServerService` é registrado como `Singleton` porque ele não guarda nenhum estado próprio entre chamadas (não tem campo mutável de instância) — é seguro compartilhar a mesma instância entre requisições HTTP concorrentes e os `BackgroundServices`. Os workers (`LifecycleWorker`, `ServerProvisionPoller`, `PoolManagerService`) são registrados como `AddHostedService`, o mecanismo padrão do ASP.NET Core para "processos de fundo que rodam durante toda a vida do app" — o framework os inicia automaticamente no `app.Build()` / início do host e os para de forma ordenada no shutdown.

Um detalhe de DI importante: como o `ApiDbContext` é *scoped* (uma instância por requisição HTTP, o padrão do EF Core), e os workers vivem fora do ciclo de uma requisição, cada um precisa **criar seu próprio escopo manualmente** a cada iteração do loop, em vez de receber o `ApiDbContext` injetado direto no construtor:

```
// padrão repetido nos três workers
using var scope = _scopes.CreateScope();
var db = scope.ServiceProvider.GetRequiredService<ApiDbContext>();
```

Isso é resolvido injetando `IServiceScopeFactory` (não `ApiDbContext`) no construtor do worker — ver [Capítulo 11](#) para o padrão completo.

9.3 Criação do schema e seed — o que acontece na primeira subida

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<ApiDbContext>();
    db.Database.EnsureCreated();
    await SeedData.EnsureSeededAsync(db);
}
```

Esse bloco roda **toda vez que a API sobe**, não só na primeira vez — mas ambas as chamadas são idempotentes por design: `EnsureCreated()` não faz nada se as tabelas já existem (ver §4.2), e `SeedData.EnsureSeededAsync` começa checando se já existe pelo menos um usuário:

```
public static async Task EnsureSeededAsync(ApiDbContext db)
{
    bool hasUsers = await db.Users.AnyAsync();
    if (hasUsers) return;
    // ... só semeia se o banco estiver completamente vazio
}
```

Isso significa: subir a API contra um banco que já tem dados reais de uso **nunca reintroduz os dados de demonstração** — o seed só roda mesmo na primeiríssima vez que aquele banco é usado. Para forçar o seed de novo (por exemplo, depois de limpar o banco de dev para testar do zero), é preciso apagar todas as linhas de `users` (ou o banco inteiro) antes de subir a API.

9.4 Os endpoints de diagnóstico (`/api/debug/*`)

Antes mesmo dos endpoints de domínio (`/api/users` , `/api/teams` , etc.), `Program.cs` define uma dezena de rotas `/api/debug/*` — esse é o padrão já descrito em §3.10. Referência rápida do que cada uma faz (todas fora do escopo de configuração AWS deste livro, mas relevantes para entender a superfície da API):

Endpoint	Método	Propósito
<code>/api/debug/ami-status</code>	GET	confere se a AMI configurada terminou de "empacotar" na AWS
<code>/api/debug/instances</code>	GET	lista instâncias EC2 taguadas pelo Summit (<code>summit:matchId</code> / <code>summit:pool</code> / <code>summit:manual</code>)
<code>/api/debug/instances/{id}/terminate</code>	POST	termina uma instância manualmente
<code>/api/debug/instances/{id}/stop</code>	POST	para (sem terminar) uma instância
<code>/api/debug/rcon</code>	POST	manda um comando RCON ad-hoc a um IP (usa senha do pool se conhecida, senão a do corpo)
<code>/api/debug/security-group</code>	GET	mostra as regras inbound/outbound do Security Group configurado
<code>/api/debug/generate-bracket/{tournamentId}</code>	POST	gera a chave imediatamente, sem esperar os horários automáticos (ver Capítulo 18)
<code>/api/debug/launch-bare-instance</code>	POST	sobe uma EC2 pura da AMI, sem User Data — para configuração manual via SSH
<code>/api/debug/volumes</code>	GET	lista volumes EBS, sinalizando órfãos (<code>Orphan = Attachments.Count == 0</code>)
<code>/api/debug/snapshots</code>	GET	lista snapshots EBS próprios
<code>/api/debug/pool</code>	GET	estado atual do pool de servidores CS2 (ver Capítulo 20)

Nenhum desses endpoints tem autenticação — são ferramentas internas de operação, não superfície de produto (ver a ressalva de segurança em §3.10).

9.5 Onde cada domínio de endpoint "de produto" mora

`Program.cs` organiza os endpoints de domínio em blocos comentados com separadores visuais (`// == USERS ==` , etc.), todos no mesmo arquivo, na ordem: **Users** → **Teams** → **Tournaments** → **Matches** → **Friends** → **Badges** → **Ranking**. O segundo arquivo, `CompetitionEndpoints.cs` , cobre um conjunto diferente e mais elaborado de regras (solicitações de entrada, cargos, check-in, escalação, veto, auditoria) — a distinção entre "vai em `Program.cs` " versus "vai em `CompetitionEndpoints.cs` " não é por domínio de dado, mas por **origem**: tudo que veio diretamente das especificações funcionais (`docs/espec-times.md` , `docs/espec-campeonatos.md`) foi implementado em `CompetitionEndpoints.cs` , cujo comentário de topo deixa isso explícito:

```
/// Endpoints das especificações funcionais (docs/espec-campeonatos.md e docs/espec-times.md).  
/// Regra central de segurança (§43): toda permissão é validada aqui no backend.  
public static class CompetitionEndpoints  
{  
    public static void MapCompetitionEndpoints(this WebApplication app) { /* ... */ }  
}
```

O **Capítulo 10** cataloga cada rota dos dois arquivos.

Capítulo 10 — Endpoints por Domínio

Este capítulo é referência de consulta rápida: toda rota HTTP exposta pela API, o que ela faz e qualquer regra de negócio não óbvia embutida nela. Para o *fluxo de produto* completo de cada área (não só a rota isolada), veja o capítulo de feature correspondente na Parte V.

10.1 Users (`Program.cs`)

Rota	Regra notável
<code>POST /api/users/steam-login</code>	upsert por <code>SteamId</code> ; se não existe, cria com <code>Level=1</code> , <code>Rank="Unranked"</code> ; se existe, só atualiza nick/avatar se vierem não-vazios e sempre atualiza <code>LastLoginAt</code>
<code>GET /api/users/{id}</code>	inclui <code>Team</code> + <code>Team.Members</code>
<code>GET /api/users/by-steam/{steamId}</code>	idem, usado no login
<code>GET /api/users/by-nickname/{nickname}</code>	comparação case-insensitive (<code>.ToLower()</code>)
<code>GET /api/users/search?q=</code>	substring case-insensitive no nickname, limita a 20 resultados
<code>PUT /api/users/{id}</code>	sobrescreve todos os campos com o que veio no corpo (não é um PATCH parcial — o client precisa mandar o objeto completo)

10.2 Teams (`Program.cs`)

Rota	Regra notável
<code>GET /api/teams</code>	ordenado por <code>Elo</code> decrescente
<code>GET /api/teams/{id}</code> / <code>GET /api/teams/by-tag/{tag}</code>	
<code>POST /api/teams</code>	cria o time e já promove o <code>captainId</code> a <code>TeamRole.Captain</code> na mesma transação
<code>PUT /api/teams/{id}</code>	exige <code>IsOwner</code> ; audita <code>team_edited</code>
<code>DELETE /api/teams/{id}</code>	exige <code>IsOwner</code> ; libera todos os membros (<code>TeamId=null</code>) antes de remover o time; audita <code>team_deleted</code>
<code>POST /api/teams/{teamId}/kick</code>	exige <code>IsOwner</code> ; recusa se <code>UserId == ByUserId</code> ("o dono não pode remover a si mesmo"); audita <code>member_kicked</code>
<code>GET /api/teams/invitations/{userId}</code>	só convites <code>Pending</code> , inclui <code>Team.Members</code> e <code>InvitedBy</code>
<code>POST /api/teams/{teamId}/invite</code>	exige que quem convida seja <code>Captain</code> do próprio time; recusa se o alvo já tem time; se já existe convite pendente idêntico, devolve o existente em vez de duplicar
<code>POST /api/teams/invitations/{id}/accept</code>	ao aceitar, cancela automaticamente qualquer outro convite pendente que o mesmo jogador tinha de outros times

Rota	Regra notável
POST <code>/api/teams/invitations/{id}/decline</code>	
POST <code>/api/teams/leave/{userId}</code>	ver lógica completa de transferência automática em §10.2.1

10.2.1 Saída do dono: a rota mais elaborada do domínio de Times

POST `/api/teams/leave/{userId}` implementa a regra de "o time nunca fica sem dono" (`docs/espec-times.md §12-13` , ver **Capítulo 13**):

```
if (user.TeamRole == TeamRole.Captain)
{
    var others = await db.Users.Where(u => u.TeamId == teamId && u.Id != userId).ToListAsync();
    if (others.Count == 0)
    {
        // último membro: o time inteiro é excluído
    }
    else
    {
        // ordem: sublíder mais antigo → membro mais antigo → id (desempate determinístico)
        var newOwner = others
            .OrderByDescending(u => u.TeamRole == TeamRole.ViceCaptain)
            .ThenBy(u => u.TeamJoinedAt ?? DateTime.MaxValue)
            .ThenBy(u => u.Id)
            .First();
        newOwner.TeamRole = TeamRole.Captain;
        team.CaptainId = newOwner.Id;
    }
}
```

O `.ThenBy(u => u.Id)` final existe puramente como **desempate determinístico** — se por algum motivo dois candidatos empatarem em cargo e data de entrada, a ordenação por `Id` (string) garante que a mesma entrada sempre produz o mesmo resultado, em vez de depender da ordem não-garantida que o banco devolveria sem um `ORDER BY` completo.

10.3 Tournaments (`Program.cs`)

Rota	Regra notável
GET <code>/api/tournaments</code>	ordenado por <code>Status</code> , depois <code>StartDate</code> ; inclui times inscritos + membros de cada time + toda a chave
GET <code>/api/tournaments/{id}</code>	mesma inclusão completa
POST <code>/api/tournaments/{id}/register</code>	ver §10.3.1
GET <code>/api/tournaments/{id}/registered/{teamId}</code>	usado por <code>TournamentService.MarkRegisteredAsync</code> (ver §7.5)

10.3.1 Inscrição: a rota mais densa de regras do projeto

POST `/api/tournaments/{id}/register` encadeia, em ordem, todas estas checagens antes de gravar qualquer coisa:

1. Idempotência: se o time já está inscrito, devolve `true` sem fazer nada de novo.

2. Campeonato existe.

3. Inscrições ainda abertas: `DateTime.UtcNow < t.RegistrationClosesAt` (fecha automaticamente 12h antes do início — `docs/espec-campeonatos.md §3`) e `Status == Open`.

1. Ainda há vaga (`count < t.MaxTeams`).

2. Time existe.

3. Quem está inscrevendo é dono ou sublíder do time (`IsOwnerOrSub`) — **só se `ByUserId` for informado** (deixa margem para chamadas internas/de teste sem esse parâmetro).

1. Monta a escalação: usa os `PlayerIds` explícitos do corpo, ou — se nenhum vier — cai no fallback automático dos "5 membros mais antigos do time" (`OrderBy(TeamJoinedAt).Take(5)`).

1. Valida a escalação inteira (`CompetitionEndpoints.ValidateLineupAsync` , ver

Capítulo 17).

1. Só então grava `TournamentTeam` + `TournamentLineupPlayer` s + audita `team_registered` .

Esse fallback do passo 7 é o que faz a inscrição funcionar mesmo sem nenhuma UI dedicada de "escolher escalação na hora de inscrever" — o client de hoje sempre chama `register` sem `PlayerIds` explícitos, deixando a API montar a escalação padrão; o ajuste fino de quem realmente joga acontece depois, na tela de Escalação (ver Capítulo 17), até o check-in abrir.

10.4 Matches (`Program.cs`)

Só leitura — não há criação/edição manual de partida via API pública (partidas nascem internamente quando um veto termina, ver Capítulo 19).

Rota	Notas
<code>GET /api/matches/recent?userId=&take=</code>	filtra por <code>Players.Any(p => p.UserId == userId)</code>
<code>GET /api/matches/team/{teamId}?take=</code>	filtra por <code>TeamAId</code> ou <code>TeamBId</code>
<code>GET /api/matches/{id}</code>	inclui <code>Players.User</code> (para nick/avatar/level no scoreboard)

10.5 Friends (`Program.cs`)

Rota	Regra notável
<code>GET /api/friends/{userId}</code>	união de "sou requester e foi aceito" com "sou addressee e foi aceito" — a direção original não importa mais depois de aceito
<code>GET /api/friends/{userId}/incoming / /outgoing</code>	só <code>Pending</code> , na direção certa

Rota	Regra notável
GET /api/friends/relation	devolve string ("None" / "Friends" / "OutgoingPending" / "IncomingPending" / "Blocked"), checando os dois sentidos do par
POST /api/friends/block	se já existe uma relação (de qualquer status), sobrescreve para Blocked ; senão cria uma nova linha já Blocked
POST /api/friends/request	recusa se já existe qualquer relação entre os dois (em qualquer sentido)
POST /api/friends/{id}/accept / /decline	exige que quem responde seja o AddresseeId da linha
DELETE /api/friends	remove a linha inteira — usado tanto para "desfazer amizade" quanto para "desbloquear"

10.6 Badges e Ranking (**Program.cs**)

Rota	Notas
GET /api/badges	catálogo completo
GET /api/badges/user/{userId}	só as desbloqueadas, via join explícito UserBadges ⇌ Badges
GET /api/badges/user/{userId}/all	catálogo completo, com IsUnlocked / UnlockedAt preenchidos onde aplicável
GET /api/ranking/players	top 50 por Elo , monta RankingPlayer (DTO de ranking, não o User completo)
GET /api/ranking/teams	top 50 por Elo , monta RankingTeam

10.7 **CompetitionEndpoints.cs** — Solicitações de Entrada

Rota	Regra notável
POST /api/teams/{teamId}/join-requests	recusa se o jogador já tem time, ou já existe solicitação pendente dele para esse time
GET /api/teams/{teamId}/join-requests?ownerId=	exige IsOwner , senão Forbid
POST /api/teams/join-requests/{id}/accept	exige IsOwner ; ao aceitar, cancela outras solicitações pendentes do mesmo jogador para outros times
POST /api/teams/join-requests/{id}/decline	exige IsOwner
POST /api/teams/join-requests/{id}/cancel	exige que quem cancela seja o próprio autor da solicitação (req.UserId == body.ByUserId) — não usado hoje pelo client (não existe botão "cancelar minha solicitação" na UI atual), mas a rota existe e funciona

10.8 CompetitionEndpoints.cs — Cargos

Rota	Regra notável
POST /api/teams/{teamId}/promote	exige <code>IsOwner</code> ; só promove quem é <code>Member</code> (recusa se já é <code>ViceCaptain</code> / <code>Captain</code>)
POST /api/teams/{teamId}/demote	exige <code>IsOwner</code> ; só rebaixa quem é <code>ViceCaptain</code>
POST /api/teams/{teamId}/transfer-ownership	exige <code>IsOwner</code> ; o antigo dono vira <code>ViceCaptain</code> (nunca <code>Member</code>) automaticamente

10.9 CompetitionEndpoints.cs — Check-in e Escalação

Ver [Capítulo 16](#) e [Capítulo 17](#) para o fluxo completo; referência de rota:

Rota	Regra notável
POST /api/tournaments/{id}/checkin	exige que a janela esteja aberta (<code>CheckInOpensAt <= now & StartDate</code>); exige que quem confirma seja dono/sublíder do time ou o capitão da escalação daquele campeonato especificamente; revalida os 5 da escalação antes de confirmar
POST /api/tournaments/{id}/close-checkin	remove (<code>NoShow</code> + <code>IsEliminated=true</code>) todo time que não confirmou; chamado hoje pelo <code>LifecycleWorker</code> no T-30min, não diretamente pelo client
PUT /api/tournaments/{id}/lineup	bloqueado a partir de <code>CheckInOpensAt</code> ; exige dono/sublíder; roda <code>ValidateLineupAsync</code> completo antes de substituir a escalação

10.10 CompetitionEndpoints.cs — Veto

Ver [Capítulo 19](#) para o fluxo completo.

Rota	Regra notável
POST /api/veto/{bracketMatchId}/start	idempotente (devolve a sessão existente se já houver); decide <code>Series</code> vs <code>FinalSeries</code> checando se o nome da rodada contém "FINAL"
GET /api/veto/{bracketMatchId}	devolve sessão + mapas restantes + próxima ação esperada (<code>next</code>)
POST /api/veto/{bracketMatchId}/action	valida turno exato, valida que o mapa está disponível; ao completar a sequência, cria o "decider" automaticamente e dispara a criação da sala (<code>Match</code>) + tentativa de atribuição via pool
GET /api/matches/by-bracket/{bracketMatchId}	busca a sala pelo <code>BracketMatchId</code>

10.11 CompetitionEndpoints.cs — Auditoria

Rota	Notas
GET /api/audit?teamId=&tournamentId=&take=	filtro opcional por qualquer um dos dois; ordenado por <code>CreatedAt</code> decrescente

10.12 Padrão de resposta HTTP usado nas rotas de ação

Três estilos coexistem deliberadamente, dependendo de quão informativa a falha precisa ser para o usuário (ver a tabela de métodos do `ApiClient` em §6.3 para o espelho do lado client):

- `Results.Ok(true/false)` — quando "não deu" é um resultado normal do domínio, sem motivo

específico a comunicar (ex. `RegisterTeamRequest` recusado por falta de vaga).

- `Results.BadRequest(string)` — quando existe um motivo específico que vale a pena mostrar

ao usuário (ex. `"A escalação precisa de exatamente 5 jogadores."`). O client só consegue ler esse texto através de `PutWithMessageAsync`; se um endpoint novo precisar comunicar um motivo assim, ele deve ser chamado do client via esse método, não `PostBoolAsync` / `GetAsync`.

- `Results.Forbid()` — quando a falha é de permissão (não é dono, não é quem deveria agir).

Vira um `403` HTTP puro, sem corpo — do lado do client, isso hoje aparece simplesmente como "falso"/falha genérica, porque nenhum dos métodos de `ApiClient` distingue um `403` de outro erro qualquer de forma especial.

Capítulo 11 — Services e Background Workers

Este capítulo explica **como** os processos de fundo da API são construídos (a mecânica de código). O **porquê de produto** de cada um (que problema resolve) está nos capítulos de feature correspondentes: [Capítulo 18](#) para o `LifecycleWorker` e [Capítulo 20](#) para `MatchServerService` / `PoolManagerService` / `RconClient` / `ServerProvisionPoller`.

11.1 O molde comum de um `BackgroundService`

Os três workers (`LifecycleWorker`, `ServerProvisionPoller`, `PoolManagerService`) herdam `Microsoft.Extensions.Hosting.BackgroundService` e seguem exatamente o mesmo esqueleto:

```
public class NomeDoWorker : BackgroundService
{
    private readonly IServiceScopeFactory _scopes;
    private readonly ILogger<NomeDoWorker> _log;

    public NomeDoWorker(IServiceScopeFactory scopes, ILogger<NomeDoWorker> log)
    {
        _scopes = scopes;
        _log = log;
    }

    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            try
            {
                using var scope = _scopes.CreateScope();
                var db = scope.ServiceProvider.GetRequiredService<ApiDbContext>();
                await TickAsync(db);
            }
            catch (Exception ex) { _log.LogError(ex, "Erro no NomeDoWorker"); }
            await Task.Delay(TimeSpan.FromSeconds(N), ct);
        }
    }
}
```

Três decisões de design se repetem nos três e valem a pena destacar:

1. `IServiceScopeFactory`, **não** `ApiDbContext` **direto** — porque `BackgroundService` é

registrado como singleton de longa duração (roda a vida inteira do processo), enquanto `ApiDbContext` é *scoped* (uma vida curta, tipicamente por requisição). Injetar o `ApiDbContext` direto no construtor do worker o prenderia à mesma instância para sempre — o que é errado para um `DbContext` (ele não foi desenhado para ser reusado indefinidamente através de muitos ciclos, é mais seguro pegar um novo por operação). Por isso todo tick cria seu próprio escopo (`_scopes.CreateScope()`) e pega um `ApiDbContext` novo, e o descarta (`using`) ao fim daquele tick.

1. `try/catch` genérico ao redor de tudo — nenhuma exceção deve nunca escapar do

`ExecuteAsync` de um `BackgroundService`, porque se escapar, o host do ASP.NET Core considera o serviço "parado" e ele nunca mais roda de novo (nem tenta) pelo resto da vida do processo — um único erro transitório (ex. timeout de rede na AWS) mataria o worker inteiro para sempre. Capturar e logar, deixando o `while` continuar para o próximo tick, é o que torna esses workers resilientes a falhas pontuais.

1. `Task.Delay(..., ct)` — o `CancellationToken` é passado para o `Delay`, não só checado no

`while`; isso garante que, quando o host pede para desligar (`ct` é cancelado), o worker acorda imediatamente do delay em vez de esperar o intervalo inteiro antes de perceber que devia parar.

11.2 `LifecycleWorker` — estrutura de código

Tick de 20 segundos. `ExecuteAsync` delega para `TickAsync(db)` (método `static`, o que facilita testá-lo isoladamente se um dia houver testes automatizados — recebe tudo que precisa como parâmetro, sem depender de campos de instância). A cada tick, ele:

1. Busca todos os campeonatos `Open` ou `Upcoming` (com toda a árvore de times/chave já incluída,

para evitar N+1 dentro do loop).

1. Para cada um, checa duas condições de tempo independentes (`now >= t.CheckInClosesAt` e

`now >= t.StartDate`) — **ambas podem disparar no mesmo tick** se o processo ficou parado por tempo suficiente para pular os dois marcos de uma vez (o que de fato aconteceu várias vezes durante o desenvolvimento deste projeto, dado os intervalos longos entre sessões).

1. Ao fim, chama `AutoVetoBotsAsync` — uma rotina separada que faz contas demo/bot jogarem o veto

sozinhas (identificadas por `team.CaptainId.Length <= 12`, os ids curtos como `usr_ghost` do `SeedData`, versus os ids longos gerados por `Guid.NewGuid():N` para contas reais).

A lógica de negócio de cada uma dessas etapas (o que significa "fechar check-in", "gerar a chave", "iniciar o campeonato") está detalhada no [Capítulo 16](#) e [Capítulo 18](#).

11.2.1 Por que `AutoVetoBotsAsync` chama a própria API via HTTP em vez de manipular o banco direto

```
private static readonly HttpClient Http = new() { BaseAddress = new Uri("http://localhost:5180") };
// ...
await Http.PostAsJsonAsync($"/api/veto/{s.BacketMatchId}/action", new { teamTag = tag, map });
```

Isso parece redundante (o worker já tem acesso direto ao `ApiDbContext` — por que passar por HTTP para si mesmo?), mas é uma escolha deliberada: a lógica de "qual é a próxima ação válida e o que acontece ao executá-la" já existe inteira e testada dentro do endpoint `POST /api/veto/{id}/action` (validação de turno, avanço de `StepIndex`, criação da sala ao terminar). Reimplementar essa mesma lógica diretamente contra o banco dentro do worker duplicaria uma regra inteira em dois lugares, com risco real de os dois

divergirem no futuro. Chamar o próprio endpoint HTTP é mais lento, mas garante que o bot segue **exatamente** o mesmo caminho de código que um jogador humano clicando no cliente — "dogfooding" da própria API.

11.3 `MatchServerService` — a peça central da lógica de servidor

Registrado como `Singleton`. Não guarda estado próprio; todo estado (qual instância, qual `PoolServer`, qual `Match`) vive no banco, lido/escrito a cada chamada através de um escopo criado na hora (`_scopes.CreateScope()`).

Os métodos se dividem em três grupos:

- **Provisionamento** (`ProvisionAsync`, `ProvisionPoolServerAsync`, `LaunchInstanceAsync`, `LaunchBareInstanceAsync`) — chamam a AWS SDK (`RunInstancesAsync`) para criar instâncias EC2. Fora do escopo deste livro em termos de *configuração* (ver `docs/plano-aws.md`), mas a *lógica* de quando cada um é chamado é produto e está no **Capítulo 20**.

- **Atribuição via RCON** (`TryAssignFromPoolAsync`, `ReleaseToPoolAsync`, `CheckPoolServerAliveAsync`, `GetHumanPlayerCountAsync`) — usam `RconClient` (ver 11.5) para trocar mapa/senha em um servidor já ligado, sem criar máquina nova.

- **Polling de estado AWS** (`PollAsync`, `PollPoolServerAsync`, `TerminateAsync`, `StopAsync`) — consultam `DescribeInstancesAsync` para saber se uma instância já tem IP público/está rodando.

11.3.1 `ToConsoleMapName` — um bug real, virado helper permanente

```
private static string ToConsoleMapName(string map)
{
    if (string.IsNullOrEmpty(map)) return "de_mirage";
    var trimmed = map.Trim().ToLowerInvariant();
    return trimmed.StartsWith("de_") ? trimmed : $"de_{trimmed}";
}
```

Vale a pena estudar este helper como estudo de caso de "por que um detalhe pequeno vira uma função nomeada": o pool de mapas do veto guarda nomes de exibição (`"Nuke"`, `"Ancient"`), mas o comando de console do CS2 (`changelevel / +map`) exige o nome real do arquivo do mapa (`de_nuke`, `de_ancient`). Enviar `changelevel Nuke` sem o prefixo não gera erro nenhum visível — o console do CS2 simplesmente ecoa `int(0=0x0)` e continua no mapa atual, silenciosamente. Esse bug existiu nos dois caminhos (cold-boot e pool) até ser descoberto ao vivo; a correção virou este helper único, chamado nos dois lugares (`BuildUserData` e `TryAssignFromPoolAsync`), justamente para que a próxima pessoa que adicionar um terceiro caminho de troca de mapa não repita o mesmo erro — qualquer string de mapa que vá para dentro de um comando de console **precisa** passar por `ToConsoleMapName` primeiro.

11.4 `PoolManagerService` — três sub-rotinas por tick

Tick de 30 segundos, só roda de fato se `MatchServerService.IsConfigured` (AWS configurada) — caso contrário, o loop gira vazio indefinidamente sem custo. A cada tick que roda:

```
await TopUpAsync(db, server, ct);           // 1. repõe o pool até SUMMIT_POOL_SIZE
await ConfirmBootingAsync(db, server, ct); // 2. confirma via RCON que quem está "Booting" já responde
await ReleaseEmptyAsync(db, server, ct);    // 3. libera de volta ao pool quem ficou vazio
```

Essas três etapas rodam sempre em sequência, nunca em paralelo — isso é intencional: por exemplo, um servidor recém-criado no passo 1 só vai aparecer como candidato do passo 2 no *próximo* tick (30s depois), nunca no mesmo — dando tempo real para a instância AWS de fato existir antes de a API tentar falar com ela.

11.4.1 Por que "instância rodando na AWS" ≠ "pronto para uso"

```
if (await server.CheckPoolServerAliveAsync(p))
{
    p.State = PoolServerState.Idle;
    // ...
}
```

`ConfirmBootingAsync` só marca um `PoolServer` como `Idle` depois de uma resposta **RCON real** (`status`), não apenas depois de a AWS reportar `State.Name == Running` com um IP público. Isso existe por causa de um problema real encontrado durante o desenvolvimento (documentado em `docs/plano-aws.md`): uma instância pode estar "Running" segundo a AWS enquanto o processo do CS2 dentro dela ainda está inicializando (ou travado por algum motivo), e tratar "Running" como sinônimo de "pronto para receber jogadores" levaria a atribuir partidas reais a um servidor que na verdade não está pronto. RCON respondendo é a única confirmação de que o CS2 de fato está de pé e aceitando comandos.

11.4.2 A janela de graça antes de liberar um servidor `InUse`

```
private static readonly TimeSpan AssignGrace = TimeSpan.FromMinutes(3);
private static readonly TimeSpan HardReleaseCeiling = TimeSpan.FromHours(3);
// ...
if (elapsed < AssignGrace) continue;           // não checa logo de cara
if (elapsed > HardReleaseCeiling) { /* libera de qualquer jeito */ }
var humans = await server.GetHumanPlayerCountAsync(p);
if (humans == 0) { /* libera */ }
```

`AssignGrace` (3 minutos) evita liberar um servidor de volta ao pool só porque nenhum jogador entrou *ainda* — dá tempo para os dois times realmente se conectarem depois do fim do veto antes de começar a monitorar se está vazio. `HardReleaseCeiling` (3 horas) é uma rede de segurança: se o RCON parar de responder por qualquer motivo (travou, perdeu rede) e `GetHumanPlayerCountAsync` nunca mais conseguir confirmar "vazio" normalmente, o servidor ainda assim é liberado incondicionalmente depois de 3 horas — para nunca prender um servidor do pool "preso" para sempre por uma falha de leitura.

11.5 `RconClient` — implementação do protocolo Source RCON

Escrito à mão, sem NuGet, porque o protocolo é simples o suficiente (pacotes binários de tamanho fixo de cabeçalho sobre TCP) para não justificar uma dependência externa. Estrutura de um pacote Source RCON:

```
[4 bytes: Size (int32, little-endian, tamanho do resto do pacote)]
[4 bytes: Id (int32, id da requisição, ecoado na resposta)]
[4 bytes: Type (int32, SERVERDATA_AUTH=3 / SERVERDATA_EXECCOMMAND=2 / SERVERDATA_AUTH_RESPONSE=2)]
[N bytes: Body (string, terminada em \0)]
[1 byte: \0 extra de terminação do pacote]
```

`ConnectAndAuthAsync` manda um pacote tipo `3` (auth) com a senha como corpo, e então **lê até 3 pacotes de resposta**, procurando especificamente pelo tipo `SERVERDATA_AUTH_RESPONSE` (2) — isso porque o protocolo manda um `SERVERDATA_RESPONSE_VALUE` vazio *antes* da resposta de autenticação real, e um cliente ingênuo que olhasse só o primeiro pacote recebido concluiria erroneamente que a autenticação falhou (corpo vazio) mesmo quando a senha estava correta.

11.5.1 Os três bugs reais encontrados ao validar isso ao vivo

Documentados também em `docs/plano-aws.md`, vale registrar aqui porque são o tipo de erro sutil que pode voltar a acontecer se o código for reescrito sem atenção:

1. **Leitura de pacote desalinhada** — a versão inicial lia 4 bytes como todo o "cabeçalho"

(achando que só existia o campo `Size`) e depois `Size` bytes como "o resto", mas isso descartava os 4 bytes do campo `Id`, desalinhando a leitura de tudo dali para frente. A versão corrigida (a que está no código hoje) lê `Size` sozinho primeiro, e então exatamente `Size` bytes de uma vez (que já incluem `Id` + `Type` + corpo + os dois bytes nulos finais): ``
`csharp var sizeBytes = await ReadExactAsync(4, timeoutMs); var size = BitConverter.ToInt32(sizeBytes, 0); var rest = await ReadExactAsync(size, timeoutMs); var id = BitConverter.ToInt32(rest, 0); var type = BitConverter.ToInt32(rest, 4); ```

1. **Consumo duplo de `ValueTask`** — `NetworkStream.ReadAsync` moderno devolve `ValueTask<int>`,

que (ao contrário de `Task<T>`) **não pode ser aguardado (`await`) mais de uma vez**. Uma versão anterior chamava `.AsTask()` nele e depois dava `await` na `ValueTask` original de novo por engano, lançando `InvalidOperationException`. A correção guarda a conversão numa variável e reusa só essa:

```
``csharp var readTask = _stream!.ReadAsync(buf.AsMemory(offset, count - offset)).AsTask(); if (await Task.WhenAny(readTask, Task.Delay(timeoutMs)) != readTask) throw new TimeoutException(...); var n = await readTask; // reusa a MESMA Task já convertida, nunca a ValueTask original de novo ``
```

1. **Nome de mapa sem prefixo** — já coberto em §11.3.1.

11.6 `ServerProvisionPoller` — o irmão mais simples do `PoolManagerService`

Tick de 10 segundos (mais frequente que o pool, porque o cold-boot é um caminho de exceção onde cada segundo a menos de espera importa mais para a experiência do jogador). Só faz uma coisa: busca `Match` es com `ProvisionState == Booting` e chama `MatchServerService.PollAsync` para cada uma, que verifica se a instância já tem IP público via `DescribeInstancesAsync` e, se sim, grava `ServerIp / ProvisionState = Ready / Status = Live` na mesma passada.

11.7 `SeedData` — como os dados de demonstração são estruturados

`SeedData.EnsureSeededAsync` roda uma única vez (ver §9.3) e monta, em ordem (cada bloco depende do anterior já ter sido salvo, por causa das chaves estrangeiras):

1. 16 usuários mock (nomes inspirados em jogadores profissionais de CS, estatísticas variadas — de `usr_ghost` no topo do ranking até `usr_rookie` quase no fim).

1. 6 times, cada um com um `CaptainId` apontando para um dos usuários acima.
2. Atribuição de cada usuário a um time e cargo, via uma função local `Join(userId, teamId, role)`

que centraliza a repetição.

1. 4 campeonatos em estados diferentes (`Open` , `InProgress` , `Upcoming`) para que a tela de

Campeonatos mostre variedade logo de cara.

1. Inscrições (`TournamentTeam`) para dois desses campeonatos.
2. Uma chave pré-montada manualmente (não gerada pelo `LifecycleWorker`) para o campeonato "Cup #1" e outra, parcialmente com resultado, para o "Ranked Series" (uma partida já `Finished` , uma `Live`) — isso existe para que a tela de chave tenha algo interessante para mostrar sem precisar esperar o relógio do sistema atingir os marcos automáticos.

1. Partidas com scoreboard completo (`SeedMatchesAsync` / `BuildPlayerStat` , com números

pseudo-aleatórios gerados a partir de uma seed determinística por partida — `new Random(m.id.GetHashCode())` — para que os números sejam sempre os mesmos entre uma recriação do banco e outra, facilitando comparar telas antes/depois de uma mudança de UI).

1. 8 badges de catálogo + desbloqueios manuais para alguns usuários.
2. Amizades (algumas aceitas, algumas pendentes) para popular a tela de Amigos.

Nenhuma dessas linhas de seed passa pelas mesmas rotas HTTP que um uso real passaria — o `SeedData` escreve direto no `ApiDbContext` (`db.Users.AddRangeAsync(...)`), pulando toda validação de negócio que os endpoints aplicariam. Isso é aceitável exatamente porque é dado de demonstração controlado, não simula um usuário real interagindo com regras — mas serve de aviso: não se deve inferir "quais validações existem" olhando o `SeedData` , porque ele deliberadamente não passa por nenhuma.

Capítulo 12 — Conta, Login e Perfil

12.1 Visão de ponta a ponta

```
LoginView —clica "Entrar com Steam"—> SteamAuthService.LoginWithSteamAsync()
                                     | (navegador abre, usuário loga na Steam)
                                     v
                                POST /api/users/steam-login (upsert)
                                     |
                                     v
MainShellViewModel decide: tem User.Country vazio?
├─ sim —> OnboardingViewModel (bem-vindo, país + role)
└─ não —> HomeViewModel direto
```

O mecanismo técnico completo do login (OpenID, validação com a Steam, restauração de sessão, modo demo) já foi explicado em §6.4 — este capítulo foca na parte de produto: o que acontece **depois** do login (onboarding) e a tela de Perfil em si.

12.2 Onboarding — o gate do primeiro login

`MainShellViewModel`, ao construir, decide para onde navegar primeiro com uma única condição:

```
if (string.IsNullOrEmpty(App.UserService.CurrentUser?.Country))
    Navigate(new OnboardingViewModel(), "BEM-VINDO");
else
    Navigate(new HomeViewModel(), "HOME");
```

`Country` vazio é usado como o sinal de "usuário nunca completou o onboarding" — não existe um campo booleano dedicado tipo `HasCompletedOnboarding`. Isso é simples e funciona porque nenhum outro fluxo do sistema define `Country` sozinho antes do onboarding (nem o `SteamAuthService`, que só preenche `Nickname` / `AvatarUrl` a partir da Steam). A implicação prática: se algum dia alguém adicionar um jeito de definir `Country` em outro lugar do fluxo de criação de conta, esse gate de onboarding pararia de funcionar silenciosamente (todo mundo pularia direto para a Home).

`OnboardingViewModel` é propositalmente mínimo — só dois campos, país e função principal — e o botão continuar só habilita quando os dois estão preenchidos:

```
ContinueCommand = new RelayCommand(async _ => await ContinueAsync(),
    _ => !string.IsNullOrEmpty(Country) && !string.IsNullOrEmpty(Role));

private async Task ContinueAsync()
{
    await App.UserService.UpdateCountryAsync(Country.Trim());
    await App.UserService.UpdateRoleAsync(Role.Trim());
    App.Navigation.NavigateTo(new HomeViewModel());
}
```

Note que a navegação para a Home usa `NavigateTo` diretamente (não passa de novo pelo `MainShellViewModel`) — o onboarding, uma vez completo, simplesmente troca a tela atual, sem re-executar a checagem do construtor do shell.

12.3 A tela de Perfil (`ProfileViewModel`)

Segue o padrão de edição inline já descrito em §5.5 — campos de leitura (`Bio`, `PrimaryRole`, `AvatarUrl`, `Country`) espelhados por campos de rascunho (`EditBio`, `EditRole`, `EditAvatarUrl`, `EditCountry`) que só substituem os originais quando `SaveAsync` é confirmado. Os quatro campos editáveis viram quatro chamadas separadas ao `UserService`:

```
private async Task SaveAsync()
{
    await App.UserService.UpdateBioAsync(EditBio);
    await App.UserService.UpdateRoleAsync(EditRole);
    await App.UserService.UpdateAvatarUrlAsync(EditAvatarUrl);
    await App.UserService.UpdateCountryAsync(EditCountry);
    IsEditing = false;
    // ... notifica as propriedades de leitura que mudaram
}
```

Cada um desses quatro métodos do `UserService` faz seu próprio `PUT /api/users/{id}` completo (mandando o objeto `User` inteiro, não um PATCH parcial — ver §10.1) — ou seja, salvar o perfil dispara **quatro requisições HTTP sequenciais**, não uma só. Isso funciona, mas é um candidato natural de otimização futura (por exemplo, um único `UpdateProfileAsync(bio, role, avatarUrl, country)` que fizesse só uma chamada) se a tela crescer com mais campos editáveis.

As estatísticas mostradas na tela (`WinRateText`, `KDText`, `Matches`, `Wins`, `FavoriteMap`) são lidas direto de `App.UserService.CurrentUser` — são os mesmos campos agregados discutidos em §4.4 (tabela `users`), hoje só populados pelo `SeedData` (ver a ressalva sobre isso no Capítulo 21).

12.4 Perfil de outro jogador (`PlayerProfileViewModel`) — o mesmo dado, contexto diferente

`PlayerProfileViewModel` mostra o perfil de **qualquer** usuário (não necessariamente o atual), recebendo o `userId` alvo no construtor. A diferença central em relação a `ProfileViewModel` é que aqui existe uma **relação** a considerar entre "quem está olhando" (`App.UserService.CurrentUser`) e "quem está sendo visto" (`User`, carregado por id):

```
public bool IsSelf           => App.UserService.CurrentUser?.Id == _user?.Id;
public bool CanAddFriend     => !IsSelf && _relation == FriendshipRepository.RelationStatus.None;
public bool IsAlreadyFriend  => _relation == FriendshipRepository.RelationStatus.Friends;
public bool HasOutgoing      => _relation == FriendshipRepository.RelationStatus.OutgoingPending;
public bool HasIncoming      => _relation == FriendshipRepository.RelationStatus.IncomingPending;
public bool IsBlocked        => _relation == FriendshipRepository.RelationStatus.Blocked;
```

Essas propriedades controlam quais botões de ação aparecem (adicionar amigo, bloquear, desbloquear) — a lógica completa de amizade/bloqueio está no Capítulo 14; este capítulo só nota que o *Perfil* é a superfície de UI onde essas ações também ficam disponíveis (além da tela dedicada de Amigos).

Amigos em comum são calculados **inteiramente no client**, sem endpoint dedicado:

```
var myFriends = await _friendRepo.GetFriendsAsync(me.Id);
var theirFriends = await _friendRepo.GetFriendsAsync(User.Id);
var theirIds = theirFriends.Select(f => f.Id).ToHashSet();
MutualFriends = myFriends.Where(f => theirIds.Contains(f.Id)).ToList();
```

Isso significa duas chamadas HTTP (a lista de amigos de cada um) mais uma interseção em memória — simples e correto para o volume de amigos que um jogador tem tipicamente (dezenas, não milhares), mas seria o primeiro ponto a mover para um endpoint dedicado no servidor (`GET /api/friends/mutual?a=&b=`) se a lista de amigos crescesse a ponto de tornar duas transferências completas caras.

12.5 O que este capítulo não cobre

Bloqueio/desbloqueio e pedidos de amizade em si são detalhados no [Capítulo 14](#). Badges mostradas no perfil são só leitura hoje — a lógica de *quando* uma badge é concedida é um gap conhecido, coberto no [Capítulo 21](#).

Capítulo 13 — Times

Times são o agregado com mais regras de permissão do sistema. Este capítulo percorre cada sub-fluxo (criação, entrada, cargos, edição/exclusão) mostrando como client e API se encaixam para cada um. A tabela de rotas já está em §10.2; aqui o foco é o fluxo completo.

13.1 Cargos — o vocabulário

```
public enum TeamRole { Member = 0, ViceCaptain = 1, Captain = 2 }
```

Mapeamento para a linguagem de produto (docs/espec-times.md): `Captain` = **Dono** (único, obrigatório), `ViceCaptain` = **Sublíder** (N por time), `Member` = **Membro comum**. Isso é diferente do "capitão da escalação" (`TournamentTeam.CaptainUserId`), um papel *por campeonato* que qualquer um dos 5 jogadores selecionados pode assumir, dono ou não — ver [Capítulo 17](#). Confundir os dois é o erro conceitual mais fácil de cometer lendo este código pela primeira vez.

13.2 Criação e entrada

Criar um time (`TeamViewModel.CreateTeamAsync` → `TeamService.CreateTeamAsync` → `POST /api/teams`) promove automaticamente quem cria a `Captain` na mesma operação — não existe um estado intermediário de "time sem dono".

Depois disso, há **duas vias simétricas** para um jogador entrar em um time existente, ambas exigindo confirmação das duas partes (nunca automática):

Via	Quem inicia	Quem aprova	Tela
Convite	Dono do time	O jogador convidado	<code>FriendsViewModel</code> (aba "Convites de Time")
Solicitação de entrada	O jogador	Dono do time	<code>TeamProfileViewModel.RequestJoinCommand</code> → <code>JoinRequestsViewModel</code>

O convite só pode ser enviado pelo dono (`TeamService.InviteByNicknameAsync` checa `me.TeamRole != TeamRole.Captain && me.TeamRole != TeamRole.ViceCaptain` — na verdade a checagem do client permite dono ou sublíder tentar, mas o endpoint `POST /api/teams/{teamId}/invite` recusa se `inviter.TeamRole != TeamRole.Captain`, ou seja, **só o dono de fato consegue**, mesmo que o botão do client não distinga isso visualmente para o sublíder). Isso é um pequeno descompasso entre a UI (que parece permitir ao sublíder abrir o painel de convite, `CanInvite > TeamId != null && (IsCaptain || IsViceCaptain)`) e a regra real do backend (só `Captain`) — na prática o sublíder veria a mensagem de erro genérica "Não foi possível convidar: jogador não encontrado ou já tem time." mesmo com um nickname válido, porque o `BadRequest()` do endpoint não distingue os dois motivos de falha para o client.

Aceitar um convite ou solicitação **cancela automaticamente** qualquer outra pendência do mesmo tipo que o jogador tinha em aberto — impedindo o cenário de alguém aceitar dois convites de times diferentes e acabar "em dois lugares".

13.3 Promover, rebaixar, transferir propriedade

Todas as três exigem `IsOwner` no backend (ver §10.8). No client, a visibilidade de cada botão por linha de membro é controlada inteiramente em XAML, com `MultiDataTrigger` combinando o cargo de quem está olhando com o cargo do membro daquela linha — vale a pena estudar o trecho real (`Views/TeamView.xaml`) porque é o uso mais elaborado de `MultiDataTrigger` do projeto:

```
<!-- Botão PROMOVER: só aparece se eu sou o dono E o alvo não é dono nem sublíder -->
<Style TargetType="Button">
  <Setter Property="Visibility" Value="Collapsed"/>
  <Style.Triggers>
    <MultiDataTrigger>
      <MultiDataTrigger.Conditions>
        <Condition Binding="{Binding DataContext.IsMyTeamCaptain, RelativeSource={RelativeSource
AncestorType=UserControl}}" Value="True"/>
        <Condition Binding="{Binding IsCaptain}" Value="False"/>
        <Condition Binding="{Binding IsViceCaptain}" Value="False"/>
      </MultiDataTrigger.Conditions>
      <Setter Property="Visibility" Value="Visible"/>
    </MultiDataTrigger>
  </Style.Triggers>
</Style>
```

Note a mistura de dois contextos de binding na mesma condição: `IsMyTeamCaptain` vem do `DataContext` do `UserControl` inteiro (o `TeamViewModel`, acessado via `RelativeSource={RelativeSource AncestorType=UserControl}` porque dentro do `DataTemplate` de cada membro o `DataContext` local já é o próprio `User` da linha, não o `ViewModel` da tela) — mas `IsCaptain` / `IsViceCaptain` (sem esse `RelativeSource`) vêm do próprio `User` daquela linha específica. Essa combinação de "uma condição olha para o `ViewModel` da tela, a outra olha para o item da linha" é o que garante que os botões de ação **nunca aparecem na própria linha do dono** (porque a segunda condição, `IsCaptain == False`, já exclui a linha dele) e **nunca aparecem para quem não é dono** (a primeira condição). O botão REBAIXAR usa a mesma estrutura, trocando a segunda condição para `IsViceCaptain == True` (só existe o que rebaixar se o alvo já é sublíder).

Do lado do backend, cada ação valida o estado atual do alvo antes de agir:

```
// promover: só quem é Member vira ViceCaptain (recusa se já é ViceCaptain/Captain)
if (target == null || target.TeamRole != TeamRole.Member) return Results.BadRequest();

// rebaixar: só quem é ViceCaptain volta a Member
if (target == null || target.TeamRole != TeamRole.ViceCaptain) return Results.BadRequest();
```

Transferir propriedade é a única ação que muda **dois** usuários na mesma operação: o alvo vira `Captain`, e quem era dono antes vira `ViceCaptain` (nunca `Member` — ele não perde todo o privilégio, só desce um degrau):

```
newOwner.TeamRole = TeamRole.Captain;
oldOwner.TeamRole = TeamRole.ViceCaptain;
team.CaptainId = newOwner.Id;
```

13.4 Saída do dono — a regra "o time nunca fica sem dono"

Já detalhada em §10.2.1. Vale reforçar aqui a hierarquia de decisão completa (docs/espec-times.md §12-13), porque é a regra de negócio mais sofisticada de todo o domínio de Times:

1. Time tem outros membros? → o próximo dono é escolhido automaticamente, nesta ordem de

prioridade: **sublíder há mais tempo no cargo** → **membro mais antigo no time** → **desempate por id** (determinístico, nunca aleatório).

1. Time não tem mais ninguém além do dono saindo? → o time inteiro é excluído.

Isso acontece **inteiramente no servidor**, dentro de `POST /api/teams/leave/{userId}` — o client (`TeamViewModel.LeaveTeamAsync` → `TeamService.LeaveTeamAsync`) não participa dessa decisão, só chama o endpoint e recarrega o estado depois.

13.5 Editar e excluir o time

`PUT /api/teams/{id}` (editar) e `DELETE /api/teams/{id}` (excluir) são ambos owner-only e seguem o padrão de edição inline/confirmação em duas etapas já descrito em §5.5 para editar, e um padrão de **confirmação explícita separada** para excluir:

```
DeleteTeamCommand = new RelayCommand(_ => ConfirmingDelete = true, _ => IsMyTeamCaptain);
ConfirmDeleteTeamCommand = new RelayCommand(async _ => await DeleteTeamAsync());
CancelDeleteTeamCommand = new RelayCommand(_ => ConfirmingDelete = false);
```

O primeiro clique só liga um booleano (`ConfirmingDelete = true`), que revela um painel de aviso vermelho no XAML (`"EXCLUIR O TIME? ESSA AÇÃO NÃO PODE SER DESFEITA."`) com dois botões — só o segundo clique de fato chama a API. Essa é a única ação destrutiva do domínio de Times que tem essa confirmação de duas etapas na UI (comparado a "Sair do time" ou "Remover jogador", que disparam direto no primeiro clique) — uma escolha consciente de UX proporcional ao dano ("excluir o time" afeta todos os membros e todo o histórico associado; "sair" afeta só quem clicou).

A exclusão em si, do lado da API, é **simples e sem validação de estado do campeonato** — isto é uma decisão de escopo documentada (docs/pendencias.md): um time pode ser excluído mesmo que esteja inscrito ou disputando um campeonato ativo no momento. A especificação completa (docs/espec-times.md §33) previa bloquear a exclusão nesse caso ("não exclui com campeonato ativo/partida pendente"), mas essa validação não foi implementada — ficou marcada como simplificação consciente para não expandir o escopo no momento em que este recurso foi construído.

```
app.MapDelete("/api/teams/{id}", async (ApiDbContext db, string id, string byUserId) =>
{
    if (!await CompetitionEndpoints.IsOwner(db, id, byUserId)) return Results.Forbid();
    var team = await db.Teams.FirstOrDefaultAsync(t => t.Id == id);
    if (team == null) return Results.NotFound();

    var members = await db.Users.Where(u => u.TeamId == id).ToListAsync();
    foreach (var m in members) { m.TeamId = null; m.TeamRole = TeamRole.Member; m.TeamJoinedAt = null; }
    db.Teams.Remove(team);
    // ...
});
```

13.6 Remover jogador (kick)

`POST /api/teams/{teamId}/kick`, owner-only, com uma checagem explícita para impedir o dono de se auto-remover por esse caminho (ele precisa usar "sair do time" ou "transferir propriedade" primeiro):

```
if (body.UserId == body.ByUserId) return Results.BadRequest("O dono não pode remover a si mesmo.");
```

Esta rota substituiu um stub antigo que existia em `TeamService` (`RemoveMemberAsync` sempre devolvia `true` sem fazer nada de verdade) — o stub foi removido junto com a implementação real sendo adicionada, incluindo a declaração correspondente que havia ficado não-usada em `ITeamService`.

13.7 Histórico de auditoria do time

O botão "HISTÓRICO" em `TeamView.xaml` (visível para qualquer membro, não só o dono) navega para `AuditLogViewModel(teamId: Team.Id)`, que é somente leitura — ver [Capítulo 15](#) para o mecanismo completo de auditoria. Toda ação deste capítulo (promover, rebaixar, transferir, editar, excluir, kick, entrada por convite/solicitação) grava uma linha correspondente, com o `action` como uma string livre e legível (`"member_promoted"`, `"ownership_transferred"`, `"team_deleted"`, etc.) — ver a lista completa de ações auditadas no [Capítulo 15](#).

Capítulo 14 — Amizades

14.1 Uma tabela, um enum, quatro (na prática cinco) significados

Toda relação entre dois jogadores — pedido, aceite, recusa, bloqueio — vive na mesma tabela `friendships` e no mesmo enum:

```
public enum FriendshipStatus { Pending = 0, Accepted = 1, Declined = 2, Blocked = 3 }
```

Isso é uma decisão de modelagem deliberada: em vez de uma tabela separada `blockedusers`, o bloqueio **reaproveita** a mesma linha de amizade (se já existir uma) ou cria uma nova já diretamente com `Status = Blocked`:

```
// Summit.Api/Program.cs — POST /api/friends/block
var existing = await db.Friendships.FirstOrDefaultAsync(x => /* nos dois sentidos */);
if (existing != null)
{
    existing.Status = FriendshipStatus.Blocked;
    existing.RespondedAt = DateTime.UtcNow;
}
else
{
    db.Friendships.Add(new Friendship { /* ... Status = FriendshipStatus.Blocked */ });
}
```

A vantagem prática: bloquear alguém que já era seu amigo **substitui** a amizade existente automaticamente (não sobra uma linha "Accepted" e outra "Blocked" competindo) — é a mesma linha que muda de status. A desvantagem: a tabela não guarda *quem* bloqueou quem de forma inequívoca quando parte de um par pré-existente é reaproveitada — `RequesterId` / `AddresseeId` continuam refletindo quem pediu a amizade *originalmente*, não necessariamente quem decidiu bloquear. Isso não causa bug hoje porque nenhuma tela distingue "quem bloqueou" de "quem foi bloqueado" (o efeito do bloqueio é simétrico em tudo que o sistema faz hoje), mas seria uma limitação real se um dia o produto precisasse, por exemplo, mostrar "você bloqueou X" de forma diferente de "X te bloqueou".

14.2 A consulta de relação: sempre nos dois sentidos

Como uma amizade pode ter sido originada por qualquer um dos dois lados, toda consulta que pergunta "qual é a relação entre A e B" precisa checar as duas ordens possíveis:

```
var f = await db.Friendships.FirstOrDefaultAsync(x =>
    (x.RequesterId == viewerId && x.AddresseeId == otherId) ||
    (x.RequesterId == otherId && x.AddresseeId == viewerId));
```

Esse padrão se repete em `GET /api/friends/relation`, `POST /api/friends/block`, `POST /api/friends/request` (para recusar duplicata) e `DELETE /api/friends`. Se você for escrever uma consulta nova sobre `friendships`, replique esse padrão de "OR nos dois sentidos" — esquecê-lo é a fonte mais provável de um bug do tipo "amizade que devia aparecer bloqueada não aparece" dependendo de quem pediu a amizade originalmente.

`GET /api/friends/relation` devolve a direção certa olhando quem é o `viewerId`:

```
if (f.Status == FriendshipStatus.Pending)
    return Results.Ok(f.RequesterId == viewerId ? "OutgoingPending" : "IncomingPending");
```

Do lado do client, `FriendshipRepository.RelationStatus` (`None / Friends / OutgoingPending / IncomingPending / Blocked`) espelha exatamente essas cinco strings possíveis via `Enum.TryParse` — ver a ressalva sobre esse contrato "por nome de string" em §8.1.

14.3 Lista de amigos: união de dois sentidos, não um `JOIN` de tabela própria

```
app.MapGet("/api/friends/{userId}", async (ApiDbContext db, string userId) =>
{
    var asRequester = db.Friendships.Where(f => f.RequesterId == userId && f.Status ==
FriendshipStatus.Accepted).Select(f => f.Addressee!);
    var asAddressee = db.Friendships.Where(f => f.AddresseeId == userId && f.Status ==
FriendshipStatus.Accepted).Select(f => f.Requester!);
    return Results.Ok(await asRequester.Concat(asAddressee).OrderBy(u => u.Nickname).ToListAsync());
});
```

Duas subconsultas — "amizades aceitas onde eu sou quem pediu" (pega o `Addressee`, o outro lado) e "amizades aceitas onde eu sou quem recebeu o pedido" (pega o `Requester`, o outro lado) — unidas com `Concat`. Isso continua sendo traduzido para SQL pelo EF Core como uma única consulta (não são duas idas ao banco), graças ao provider LINQ-to-Entities.

14.4 Amigos em comum: cálculo client-side

Já descrito em §12.4: não existe endpoint dedicado — o `PlayerProfileViewModel` busca a lista completa de amigos dos dois usuários e faz a interseção em memória (`HashSet` + `.Where(Contains)`). Reforçando aqui o trade-off: simples de implementar, correto, mas transfere mais dados pela rede do que um endpoint dedicado faria (`GET /api/friends/mutual?a=&b=` que fizesse a interseção no banco).

14.5 O fluxo completo em `FriendsViewModel`

A tela de Amigos tem quatro abas (`ActiveTab` 0-3: Amigos, Recebidos, Enviados, Convites de Time), cada uma carregada de uma vez só no `LoadAsync()` inicial — não há carregamento sob demanda por aba, todas as quatro listas são buscadas de uma vez sempre que a tela abre ou uma ação é concluída:

```
private async Task LoadAsync()
{
    Friends      = await _friendRepo.GetFriendsAsync(me.Id);
    Incoming     = await _friendRepo.GetIncomingRequestsAsync(me.Id);
    Outgoing     = await _friendRepo.GetOutgoingRequestsAsync(me.Id);
    TeamInvites  = await _teamRepo.GetInvitationsForUserAsync(me.Id);
}
```

Note que a quarta aba (**Convites de Time**) não é uma "amizade" de forma nenhuma — é reaproveitar a mesma tela para também mostrar convites de time pendentes (`TeamInvitation`, ver [Capítulo 13](#)), porque ambos são "coisas que outra pessoa te mandou e você precisa aceitar/recusar" do ponto de vista de UX, mesmo sendo dois domínios de dado completamente diferentes por trás. `AcceptTeamInviteCommand` / `DeclineTeamInviteCommand` nessa tela chamam `App.TeamService`, não `FriendshipRepository`.

Buscar um amigo por nickname exato (`SendRequestAsync`) valida dois casos triviais antes de mandar o pedido — não pode adicionar a si mesmo, e o alvo precisa existir:

```
var target = await _userRepo.GetByNicknameAsync(SearchNickname.Trim());
if (target == null) { SearchMessage = "Jogador não encontrado."; return; }
if (target.Id == me.Id) { SearchMessage = "Você não pode adicionar a si mesmo."; return; }
```

A segunda checagem (`target.Id == me.Id`) só existe no client — o endpoint `POST /api/friends/request` também tem a mesma checagem (`if (req.RequesterId == req.AddresseeId) return Results.Ok(false)`), então mesmo que o client pulasse essa validação, o backend recusaria de qualquer forma (reforçando a regra de "nunca confiar só na validação do client", ver [§3.7](#)).

14.6 O que a especificação previa e não foi implementado

[docs/espec-times.md §22-26](#) menciona "denúncia de perfil" como parte do fluxo de amizades/perfis. Essa peça foi **deixada de fora conscientemente** — não existe fila de moderação/revisão administrativa no sistema hoje, e implementar denúncia sem ter para onde ela vai (um painel de admin, um processo de revisão) seria construir metade de uma feature. Isso está documentado como decisão de escopo em [docs/pendencias.md](#), não como um esquecimento.

Capítulo 15 — Auditoria

15.1 Propósito e forma

`AuditLog` (`Models/Competition.cs`) é um registro de texto livre, sem chave estrangeira real para nada (ver §4.4, seção `auditlogs`):

```
public class AuditLog
{
    public string Id { get; set; } = string.Empty;
    public string Action { get; set; } = string.Empty;
    public string? ActorUserId { get; set; }
    public string? TargetUserId { get; set; }
    public string? TeamId { get; set; }
    public string? TournamentId { get; set; }
    public string? OldValue { get; set; }
    public string? NewValue { get; set; }
    public string? Reason { get; set; }
    public DateTime CreatedAt { get; set; } = DateTime.UtcNow;
}
```

Essa forma "achatada" (todos os campos opcionais, preenchidos ou não conforme o tipo de ação) é deliberada: um único tipo de registro serve para qualquer ação auditável do sistema, sem precisar de uma tabela por tipo de evento. O preço dessa simplicidade é que **nada garante em nível de banco** que `TeamId` esteja preenchido quando `Action` é algo relacionado a time — essa consistência é responsabilidade de quem chama `Audit(...)` em cada endpoint, não do schema.

15.2 Catálogo de ações auditadas hoje

Toda chamada a `CompetitionEndpoints.Audit(...)` no código hoje, por área:

Área	Ações (<code>Action</code>)
Times	<code>team_edited</code> , <code>team_deleted</code> , <code>member_kicked</code> , <code>member_left</code> , <code>ownership_auto_transferred</code> , <code>ownership_transferred</code> , <code>member_promoted</code> , <code>member_demoted</code>
Solicitações de entrada	<code>join_request_created</code> , <code>join_request_accepted</code> , <code>join_request_declined</code>
Campeonatos	<code>team_registered</code> , <code>team_noshow_removed</code> , <code>bracket_generated</code> , <code>tournament_started</code> , <code>checkin_confirmed</code> , <code>lineup_changed</code>
Veto	<code>veto_completed</code>

Note que **nem toda ação do sistema é auditada** — por exemplo, aceitar/recusar convite de time (`POST /api/teams/invitations/{id}/accept / decline`) e as ações de amizade (Capítulo 14) não chamam `Audit`. Isso não é uma inconsistência acidental relatada como bug em nenhum lugar do projeto, mas é uma

observação útil: **a cobertura de auditoria é parcial**, concentrada nas ações que a especificação ([docs/espec-times.md §32](#)) explicitamente listou como merecedoras de log ("criação/edição do time, entradas/saídas/remoções, convites, promoções, troca de dono, inscrições, escalações, exclusão").

15.3 O helper `Audit` e por que ele não salva sozinho

```
public static Task Audit(ApiDbContext db, string action, string? actor, string? target,
    string? teamId, string? tournamentId, string? oldValue, string? newValue, string? reason)
{
    db.AuditLogs.Add(new AuditLog { Id = $"aud_{Guid.NewGuid():N}", Action = action, /* ... */ });
    return Task.CompletedTask;
}
```

Já explicado em §3.8: `Audit` só adiciona a entidade ao `DbContext`, e o `SaveChangesAsync()` seguinte (do próprio endpoint que chamou `Audit`) grava o log **na mesma transação** da mudança real. Isso garante atomicidade: se o `SaveChangesAsync` falhar por qualquer motivo, nem a mudança nem o log são persistidos — nunca existe um log de auditoria "órfão" descrevendo uma mudança que na verdade não aconteceu.

`Audit` retorna `Task` (não é realmente assíncrono, sempre `Task.CompletedTask`) só para manter a assinatura consistente com o resto do código `async` ao redor — pode ser chamado com `await` sem custo real, mantendo a leitura do código uniforme.

15.4 A tela: somente leitura, por design

`AuditLogViewModel` aceita filtros opcionais por `teamId` e/ou `tournamentId` no construtor, e o `AuditRepository` monta a query string condicionalmente:

```
public async Task<List<AuditLog>> GetAsync(string? teamId = null, string? tournamentId = null, int take = 50)
{
    var qs = $"?take={take}" + (teamId != null ? $"&teamId={teamId}" : "") + (tournamentId != null ?
    $"&tournamentId={tournamentId}" : "");
    return await ApiClient.GetAsync<List<AuditLog>>($"api/audit{qs}") ?? new();
}
```

A tela (`Views/AuditLogView.xaml`) não tem nenhum comando de ação além de "voltar" — é propositalmente **read-only**: lista `Action`, `CreatedAt`, e o par `OldValue` → `NewValue` quando presentes, mais `Reason` quando houver. Não há filtro de data, paginação além do `take` fixo (100, passado por `AuditLogViewModel`), ou busca por ator — é a versão mínima de uma tela de auditoria, suficiente para "o dono do time consegue ver o que aconteceu", mas sem nenhuma ferramenta de investigação mais profunda.

Hoje o único ponto de entrada para essa tela é o botão "HISTÓRICO" em `TeamView.xaml` (navegando com `teamId` preenchido). Não existe um ponto de entrada equivalente a partir de uma tela de campeonato (mesmo `AuditLogViewModel` já aceitando `tournamentId` desde o construtor) — seria um acréscimo pequeno e direto se o produto quisesse um "histórico do campeonato" visível para o organizador.

Capítulo 16 — Campeonatos: Inscrição e Check-in

16.1 A linha do tempo automática de um campeonato

Tudo em torno do ciclo de vida de um campeonato gira em torno de três datas **calculadas**, nunca digitadas por um organizador — são todas derivadas de `Tournament.StartDate` (`Models/Tournament.cs`):

```
public DateTime RegistrationClosesAt => StartDate.AddHours(-12);
public DateTime CheckInOpensAt      => StartDate.AddHours(-1);
public DateTime CheckInClosesAt     => StartDate.AddMinutes(-30);
```

Momento	O que acontece	Quem dispara
Publicação	Inscrições abrem	(implícito — o campeonato já nasce <code>Open</code>)
T-12h	Inscrições fecham	checagem em <code>POST /api/tournaments/{id}/register</code> (recusa se <code>now >= RegistrationClosesAt</code>)
T-1h	Check-in abre	checagem em <code>POST /api/tournaments/{id}/checkin</code> (recusa se <code>now < CheckInOpensAt</code>) — e também é o instante em que <code>CanEditLineup</code> vira <code>false</code> (ver Capítulo 17)
T-30min	Check-in fecha; ausentes removidos; chave gerada	<code>LifecycleWorker.TickAsync</code> , ramo <code>now >= t.CheckInClosesAt</code>
T-0 (<code>StartDate</code>)	Campeonato inicia; vetos da 1ª rodada abrem	<code>LifecycleWorker.TickAsync</code> , ramo <code>now >= t.StartDate</code>

Não existe nenhum job agendado externo (cron, Hangfire, Quartz) — tudo é resolvido pelo `LifecycleWorker` comparando `DateTime.UtcNow` contra essas propriedades computadas a cada tick de 20 segundos (ver §11.2). Isso significa que a precisão do sistema é de "até 20 segundos de atraso" em relação ao horário exato — aceitável para este domínio (ninguém precisa que o check-in feche no milissegundo exato).

16.2 Inscrição

A rota mais densa de validações do projeto já foi detalhada em §10.3.1. Do lado do client, `TournamentsViewModel.RegisterAsync` chama isso sem nenhuma tela de "escolher escalação agora" — o time do usuário atual é usado direto (`App.UserService.CurrentUser?.TeamId`), e a escalação é resolvida pelo fallback automático da API (os 5 membros mais antigos). Ajustar quem realmente joga acontece depois, na tela dedicada de Escalação, a qualquer momento até o check-in abrir — ver Capítulo 17.

Idempotência é tratada explicitamente: chamar `register` para um time já inscrito **não é um erro**, simplesmente devolve sucesso sem duplicar nada:

```
var exists = await db.TournamentTeams.AnyAsync(x => x.TournamentId == id && x.TeamId == req.TeamId);
if (exists) return Results.Ok(true);
```

16.3 Check-in

O check-in confirma presença de um time já inscrito, dentro da janela T-1h → T-0. Quem pode confirmar não é só "dono ou sublíder do time" de forma genérica — a regra é mais específica e vale a pena ler com atenção:

```
var canCheckIn = by != null &&
    ((by.TeamId == body.TeamId && (by.TeamRole == TeamRole.Captain || by.TeamRole == TeamRole.ViceCaptain))
    || by.Id == tt.CaptainUserId);
```

Ou seja: **dono ou sublíder do time em geral**, ou especificamente o **capitão da escalação daquele campeonato** (mesmo que essa pessoa seja um membro comum sem cargo administrativo no time) — reflete `docs/espec-campeonatos.md §4`: "(dono, sublíder ou capitão da escalação, se habilitado no campeonato)". Isso é a primeira vez neste livro em que os dois conceitos de "capitão" (dono do time vs. capitão da escalação, ver [§13.1](#)) se encontram na mesma regra de permissão — e é exatamente por isso que a distinção entre os dois importa tanto: um membro comum, sem nenhum cargo administrativo no time, pode legitimamente confirmar o check-in de todo o time se ele foi escolhido como capitão da escalação daquele campeonato específico.

Antes de confirmar, a rota **revalida os 5 da escalação inteira**:

```
var lineupIds = tt.Lineup.Select(l => l.UserId).ToList();
var stillValid = await db.Users.CountAsync(u => lineupIds.Contains(u.Id) && u.TeamId == body.TeamId);
if (lineupIds.Count != 5 || stillValid != 5)
    return Results.BadRequest("Escalação inválida: o time precisa de 5 jogadores elegíveis.");
```

Isso cobre o cenário em que um jogador da escalação saiu do time (ou foi removido) depois da inscrição mas antes do check-in — `stillValid` conta só quem ainda pertence ao time *agora*; se algum saiu, a contagem cai abaixo de 5 e o check-in é recusado até a escalação ser corrigida (reforça `docs/espec-times.md §18`: "jogador sai antes do bloqueio → removido da escalação, inscrição incompleta, notificar").

16.4 Fechamento automático do check-in (T-30min)

```
// Summit.Api/LifecycleWorker.cs
if (now >= t.CheckInClosesAt)
{
    var waiting = t.TournamentTeams.Where(x => x.CheckIn == CheckInStatus.Waiting &&
!x.IsEliminated).ToList();
    foreach (var w in waiting)
    {
        w.CheckIn = CheckInStatus.NoShow;
        w.IsEliminated = true;
        await CompetitionEndpoints.Audit(db, "team_noshow_removed", ...);
    }
    if (t.Bracket.Count == 0) { /* gera a chave só com os CONFIRMADOS */ }
}
```

Todo time que ainda está `CheckInStatus.Waiting` nesse momento vira `NoShow` e é marcado `IsEliminated = true` — automaticamente, sem intervenção humana. A geração da chave (ver [Capítulo 18](#)) só acontece depois disso, e só considera `x.CheckIn == CheckInStatus.Confirmed` — times ausentes nunca entram na chave.

Existe também um endpoint equivalente chamável manualmente, `POST /api/tournaments/{id}/close-checkin`, com a mesma lógica de remoção — mas ele não gera a chave sozinho (só o `LifecycleWorker` faz as duas coisas em sequência); esse endpoint existe principalmente como ferramenta administrativa/de teste, não como parte do fluxo automático normal.

16.5 O fallback do T-0: e se ninguém fez check-in?

```
if (now >= t.StartDate)
{
    if (t.Bracket.Count == 0)    // chave ainda não foi gerada (ex.: check-in nunca rodou, ou pulou)
    {
        var teams = t.TournamentTeams.Where(x => !x.IsEliminated && x.Team != null).OrderBy(x =>
x.Seed).ToList();
        if (teams.Count >= 2) GenerateBracket(db, t, teams);
    }
    if (t.Bracket.Count == 0) continue;    // ainda sem times suficientes: aguarda
    // ...
}
```

Esse é um caminho de recuperação: se por algum motivo a chave ainda não existe quando `StartDate` chega (por exemplo, o processo da API ficou parado durante toda a janela de check-in — algo que de fato aconteceu várias vezes durante este projeto, dado os intervalos longos entre sessões de desenvolvimento), o `LifecycleWorker` gera a chave na hora, usando **todos os inscritos não eliminados** em vez de só os confirmados (já que nenhum check-in rodou para filtrar quem faltou). Se mesmo assim não houver pelo menos 2 times, o campeonato simplesmente aguarda (`continue`) — ele não trava em erro, só permanece sem iniciar até algo mudar (por exemplo, mais uma equipe se inscrever, embora tecnicamente as inscrições já devessem estar fechadas há muito tempo nesse ponto).

16.6 O que falta neste domínio

A especificação (`docs/espec-campeonatos.md`) prevê ações administrativas — desclassificação, remarcação, W.O. por não comparecer *no servidor* (diferente do no-show do check-in, que já existe) — que não estão implementadas. Isso está detalhado como parte do gap maior no [Capítulo 21](#), já que a maioria dessas ações depende de haver primeiro um conceito de "resultado de partida" que hoje não existe.

Capítulo 17 — Escalação (Lineup)

17.1 O que é a escalação e por que é separada do elenco do time

`docs/espec-times.md §2` é explícito: **não há limite de 5 jogadores no elenco** de um time — um time pode ter 8, 10, 15 membros. O limite de 5 é uma regra *por campeonato*: a **escalação** (`TournamentTeam.Lineup`, uma lista de `TournamentLineupPlayer`) é quem, dentre o elenco todo, realmente representa o time naquela competição específica. Um jogador pode estar no elenco do time e nunca ter sido escalado para um campeonato específico; times diferentes campeonatos podem (em teoria, se o elenco for grande o suficiente) ter escalações completamente diferentes de jogadores do mesmo elenco.

Junto disso vem o **capitão da escalação** (`TournamentTeam.CaptainUserId`) — um papel *independente do cargo no time* (ver a distinção já estabelecida em §13.1). Qualquer um dos 5 selecionados pode ser o capitão da escalação, mesmo que seja um `Member` comum sem nenhum privilégio administrativo no time. Esse papel é quem, segundo `docs/espec-times.md §16`, responde pela equipe "na competição": notificações, presença, vetos, dados do servidor, contato com admin, confirmação de resultados — e, como visto em §16.3, tem autorização explícita para confirmar o check-in mesmo sem ser dono/sublíder.

17.2 Janela de edição: `CanEditLineup`

```
// Models/Tournament.cs
public bool CanEditLineup => IsRegistered && DateTime.UtcNow < CheckInOpensAt;
```

A escalação pode ser alterada livremente **a qualquer momento entre a inscrição e a abertura do check-in** (T-1h) — depois disso, fica bloqueada. `IsRegistered` (ver §7.5) precisa estar `true` para o botão de escalação sequer aparecer — não faz sentido editar a escalação de um campeonato em que o time nem está inscrito. O endpoint replica exatamente essa mesma janela de tempo do lado do servidor (nunca confia só no botão estar desabilitado no client):

```
// PUT /api/tournaments/{id}/lineup
if (DateTime.UtcNow >= t.CheckInOpensAt)
    return Results.BadRequest("Escalação bloqueada: o check-in já abriu.");
```

17.3 `ValidateLineupAsync` — a validação central, reusada em dois lugares

```
// Summit.Api/CompetitionEndpoints.cs
public static async Task<string?> ValidateLineupAsync(
    ApiDbContext db, string tournamentId, string teamId,
    List<string> playerIds, string? captainUserId, string? ignoreTournamentTeamId,
    int requiredCount = 5)
{
    var ids = playerIds.Distinct().ToList();
    if (ids.Count != requiredCount) return $"A escalação precisa de exatamente {requiredCount} jogadores.";

    var inTeam = await db.Users.CountAsync(u => ids.Contains(u.Id) && u.TeamId == teamId);
    if (inTeam != requiredCount) return "Todos os jogadores da escalação precisam pertencer ao time.";

    if (string.IsNullOrEmpty(captainUserId) || !ids.Contains(captainUserId))
        return "O capitão da escalação deve estar entre os 5 selecionados.";

    // nenhum jogador pode representar outro time no mesmo campeonato
    var conflict = await db.TournamentLineupPlayers
        .Include(lp => lp.TournamentTeam)
        .AnyAsync(lp => ids.Contains(lp.UserId)
            && lp.TournamentTeam!.TournamentId == tournamentId
            && lp.TournamentTeamId != ignoreTournamentTeamId);
    if (conflict) return "Um dos jogadores já está inscrito por outro time neste campeonato.";

    return null; // null = válido
}
```

Essa função é chamada de **dois lugares**: dentro de `POST /api/tournaments/{id}/register` (validando a escalação inicial, com `ignoreTournamentTeamId = null` porque ainda não existe `TournamentTeam` para esse time nesse campeonato) e dentro de `PUT /api/tournaments/{id}/lineup` (validando uma troca posterior, passando o `TournamentTeam.Id` atual como `ignoreTournamentTeamId` — para que a checagem de conflito não acuse o próprio time de estar "em conflito consigo mesmo" ao comparar a escalação nova contra a lista de jogadores já escalados *por qualquer time* naquele campeonato, incluindo os que o próprio time já tinha escalado antes).

A convenção de retorno (`string?` — `null` significa válido, qualquer outra coisa é a mensagem de erro a mostrar) é o padrão que o `PutWithMessageAsync` do client foi desenhado para consumir (ver §3.6) — a mensagem retornada aqui é literalmente o texto que aparece na tela do usuário.

17.3.1 O "modo alpha": 5 é o alvo, mas times pequenos usam o elenco inteiro

```
var required = Math.Min(5, team.Members.Count);
```

Times com menos de 5 membros no elenco (comum em ambientes de teste/desenvolvimento com poucos usuários cadastrados) não ficam impedidos de se inscrever — a exigência cai para "o elenco inteiro", não trava em "exatamente 5". Isso é chamado de "modo alpha" no comentário do código (`CompetitionEndpoints.ValidateLineupAsync`), reconhecendo que é uma acomodação temporária para o estágio atual de poucos usuários, não a regra final de produto (que é sempre 5, conforme CS2 padrão).

17.4 A tela: `LineupViewModel` / `LineupView`

Carrega o elenco completo do time e pré-marca a seleção/capitão a partir do que **já veio embutido** no `Tournament` carregado (`TournamentTeam.Lineup` / `CaptainUserId`) — não há um endpoint de leitura dedicado para "buscar a escalação atual", porque essa informação já faz parte da árvore que `GET /api/tournaments/{id}` devolve:

```
private async Task LoadAsync()
{
    var team = await _teamRepo.GetByIdAsync(_teamId);
    var tournament = await _tourRepo.GetByIdAsync(_tournamentId);
    var tt = tournament?.TournamentTeams.FirstOrDefault(x => x.TeamId == _teamId);
    var currentLineupIds = tt?.Lineup.Select(l => l.UserId).ToHashSet() ?? new HashSet<string>();

    RequiredCount = Math.Min(5, team?.Members.Count ?? 0);
    Members = (team?.Members ?? new()).Select(u => new LineupMemberItem
    {
        User = u,
        IsSelected = currentLineupIds.Contains(u.Id),
        IsCaptainChoice = tt?.CaptainUserId == u.Id
    }).ToList();
}
```

Seleção é por clique/toggle (`ToggleSelect`), limitada a `RequiredCount` — tentar selecionar um sexto jogador quando já há 5 simplesmente não faz nada (`if (!item.IsSelected && SelectedCount >= RequiredCount) return;`). Desmarcar um jogador que era o capitão escolhido automaticamente também limpa a escolha de capitão daquele jogador (`if (!item.IsSelected) item.IsCaptainChoice = false;`) — evitando o estado inconsistente de "capitão escolhido que não está mais na escalação". `SetCaptain` só permite escolher capitão entre quem já está selecionado (`if (item == null || !item.IsSelected) return;`).

Salvar (`SaveCommand`) faz uma validação de UI rápida antes mesmo de chamar a API (contagem exata, capitão escolhido) e só então chama `TournamentRepository.UpdateLineupAsync`, mostrando a mensagem de erro exata que a API devolver via `PutWithMessageAsync` se algo passar da validação local mas falhar no servidor (por exemplo, um conflito de "jogador já escalado por outro time" que o client não tem como prever sozinho, porque não tem visibilidade sobre a escalação de outros times).

17.5 Ligação com o resto do sistema

A escalação confirmada é o que a checagem de check-in revalida (ver §16.3) e é também, semanticamente, quem "representa o time" durante o veto e a partida (embora hoje o veto em si identifique os lados só pela `Tag` do time, não pela lista de jogadores da escalação — ver Capítulo 19). A escalação não tem nenhuma relação direta com a *geração* da chave (Capítulo 18) — a chave é montada por time (via `Seed`), não por jogador.

Capítulo 18 — Chave (Bracket)

Este é o capítulo mais matemático do livro — a geração e o desenho da chave envolvem fórmulas que vale a pena entender passo a passo, não só citar.

18.1 O vocabulário: `BracketRound`, `BracketMatch`, `BracketSide`

```
public enum BracketMatchStatus { Pending = 0, Live = 1, Finished = 2, Veto = 3, PreparingServer = 4 }
public enum BracketSide { Upper = 0, Lower = 1, GrandFinal = 2 }

public class BracketRound
{
    public int RoundNumber { get; set; }
    public string Name { get; set; } = string.Empty;
    public BracketSide Side { get; set; } = BracketSide.Upper;
    public List<BracketMatch> Matches { get; set; } = new();
}
```

`BracketSide` só importa para eliminação **dupla** — eliminação simples usa exclusivamente `Upper`. `RoundNumber` não é um índice visual sequencial: é usado como **namespace** para separar as três sub-chaves dentro do mesmo campeonato — Upper usa 1, 2, 3...; Lower usa 101, 102... (offset `100 + i`); Grande Final usa exatamente `200`. Isso permite que uma consulta simples (`ORDER BY RoundNumber`) sempre ordene corretamente dentro de cada sub-chave, sem precisar de uma segunda coluna de "ordem visual".

18.2 Geração — eliminação simples

`LifecycleWorker.GenerateSingleElimination` (privado, mas reusado tanto para uma chave puramente simples quanto como a metade "Upper" de uma chave dupla):

```
teams = teams.OrderBy(_ => Random.Shared.Next()).ToList(); // seed ALEATÓRIO
int n = teams.Count;
int totalRounds = (int)Math.Ceiling(Math.Log2(Math.Max(n, 2)));

for (int r = 0; r < totalRounds; r++)
{
    int matchesInRound = (int)Math.Ceiling(n / Math.Pow(2, r + 1));
    for (int p = 0; p < Math.Max(matchesInRound, 1); p++)
    {
        // só a rodada 0 recebe times de verdade; o resto nasce "TBD"
        if (r == 0)
        {
            var a = p * 2 < n ? teams[p * 2] : null;
            var b = p * 2 + 1 < n ? teams[p * 2 + 1] : null;
            bm.TeamATag = a?.Team?.Tag ?? "TBD";
            bm.TeamBTag = b?.Team?.Tag ?? "BYE";
        }
    }
}
```

Dois pontos centrais:

1. **O sorteio é aleatório** (docs/espec-campeonatos.md §5 permite manual/ranking/aleatório —

hoje só o modo aleatório está implementado; não há UI para o organizador escolher seed manual ou por ranking).

1. **Só a primeira rodada recebe nomes de times reais.** Todas as rodadas seguintes nascem com

`TeamATag = TeamBTag = "TBD"` — porque preencher quem avança depende de saber o resultado da partida anterior, e **isso ainda não existe no sistema** (ver [Capítulo 21](#)). Isso significa: **a chave, hoje, só é realmente jogável na primeira rodada**. As rodadas seguintes ficam visualmente montadas (a estrutura de colunas/partidas existe) mas travadas em "TBD" para sempre, até que alguém preencha manualmente ou até que o avanço automático seja implementado.

1. **BYE**, não "TBD", para o lado B quando o número de times é ímpar/não é potência de 2 exata:

`p * 2 + 1 < n ? teams[p*2+1] : null` seguido de `b?.Team?.Tag ?? "BYE"` — um time sem adversário na primeira rodada aparece com `"BYE"` no lado oposto, não com `"TBD"` (a diferença textual comunica corretamente "não existe adversário aqui", vs. "existe adversário, mas ainda não sabemos quem").

Nomeação de rodada (`RoundName`) é dinâmica, calculada de trás para frente a partir do total de rodadas: a última é sempre `"FINAL"`, a penúltima `"SEMIS"`, a antepenúltima `"QUARTAS"`, e qualquer coisa antes disso vira `"RODADA {n}"` genérico — o que garante nomes corretos independente de quantos times existem (8 times = 3 rodadas = Quartas/Semis/Final; 16 times = 4 rodadas = Oitavas seria "RODADA 1"/Quartas/Semis/Final, já que o código não tem um nome específico para oitavas).

18.3 Geração — eliminação dupla

`LifecycleWorker.GenerateDoubleElimination` monta três blocos em sequência:

```
int n = teams.Count;
int k = (int)Math.Ceiling(Math.Log2(Math.Max(n, 2)));

GenerateSingleElimination(db, t, teams, BracketSide.Upper); // reaproveita o algoritmo acima

int lowerRounds = Math.Max(0, 2 * (k - 1));
for (int i = 0; i < lowerRounds; i++)
{
    int matchesInRound = Math.Max(1, (int)(n / Math.Pow(2, Math.Floor(i / 2.0) + 2)));
    // rodada i: RoundNumber = 100+i+1, Name = "LOWER {i+1}" ou "LOWER FINAL" na última
    // TODAS as partidas nascem TBD/TBD (nem a "primeira rodada" da Lower tem time real -
    // não há como saber quem cai pra lá sem o avanço da Upper, que não existe ainda)
}

// Grande Final: 1 rodada, 1 partida, RoundNumber=200, Side=GrandFinal, times TBD
```

18.3.1 A fórmula de contagem de partidas por rodada da Lower

```
int matchesInRound = Math.Max(1, (int)(n / Math.Pow(2, Math.Floor(i / 2.0) + 2)));
```


Essa fórmula foi **verificada empiricamente ao vivo** (via o endpoint de debug `/api/debug/generate-bracket/{tournamentId}`, ver §9.4) contra dois casos concretos:

- **N=4 (k=2)**: `lowerRounds = 2*(2-1) = 2`. Rodada 0 (`i=0`): `4 / 2^(0+2) = 4/4 = 1` partida

("LOWER 1"). Rodada 1 (`i=1`, última): `4 / 2^(0+2) = 1` partida ("LOWER FINAL"). Total: Upper (2 partidas na rodada 1 + 1 na final = 3) + Lower (1+1=2) + Grande Final (1) = **6 partidas**, que bate exatamente com a fórmula geral de eliminação dupla `2N-2` para N=4 (`2*4-2=6`). ✓

- **N=7 (single elimination)**: testado separadamente, confirmando o cálculo correto de BYE

(um time sem adversário na primeira rodada) quando N não é potência de 2 exata.

Vale registrar isso porque a fórmula em si não é óbvia de derivar de cabeça — ela existe justamente para replicar a estrutura padrão de dupla eliminação (a Lower "absorve" os perdedores da Upper em um ritmo específico onde cada duas rodadas da Lower processam uma rodada de eliminados da Upper), e foi mantida como está desde que a verificação empírica confirmou que bate com `2N-2` para os casos testados.

18.3.2 Por que a Lower inteira nasce "TBD" (diferente da Upper, que ganha a rodada 1 preenchida)

Isso é intencional e consistente com o resto do sistema: preencher a Lower exigiria saber quem perdeu cada partida da Upper — informação que só existiria se houvesse avanço automático de resultado (o gap do Capítulo 21). Então, diferente da Upper (que pelo menos consegue preencher sua *primeira* rodada com o sorteio inicial), a Lower inteira — mesmo sua "primeira" rodada — fica travada em TBD até esse gap ser resolvido.

`Tournament.BraceReset` (um campo booleano já existente no modelo, pensado para a "partida de reset" da grande final quando o campeão vindo da Lower vence a primeira série) também não é usado ainda — a mesma razão: essa mecânica só faz sentido quando existe avanço de resultado de verdade.

18.3.3 Sistema Suíço: fora de escopo, documentado

```
internal static void GenerateBracket(ApiDbContext db, Tournament t, List<TournamentTeam> teams)
{
    if (t.FormatType == TournamentFormat.DoubleElimination)
        GenerateDoubleElimination(db, t, teams);
    else
        GenerateSingleElimination(db, t, teams, BracketSide.Upper);
}
```

Repare que não há nenhum `case` para `TournamentFormat.Swiss` — qualquer valor que não seja `DoubleElimination` cai no `else` de eliminação simples. Isso significa que, hoje, **configurar um campeonato como Suíço no banco produziria uma chave de eliminação simples por engano** (não um erro, um comportamento silenciosamente errado) — porque não existe nenhuma UI no client para criar um campeonato e escolher o formato (todos os campeonatos hoje vêm do `SeedData`, todos `SingleElimination` ou `DoubleElimination`), esse caso nunca foi de fato exercitado. É uma lacuna a ter em mente se um dia a criação de campeonatos ganhar UI própria com essa opção disponível.

18.4 BracketLayout — o algoritmo de desenho genérico

`ViewModels/BracketLayout.cs` resolve um problema puramente visual: como desenhar N colunas (rodadas) com um número decrescente de partidas por coluna, mantendo o alinhamento vertical correto (cada partida da rodada 2 centralizada entre as duas partidas da rodada 1 que a alimentam), **sem desenhar nenhuma linha de conexão** (decisão de design consciente — ver §18.4.2) e **para qualquer quantidade de times**.

```
public static List<BracketColumnViewModel> Build(IEnumerable<BracketRound> rounds, double unit = 96, double cardHeight = 80)
{
    var ordered = rounds.OrderBy(r => r.RoundNumber).ToList();
    var columns = new List<BracketColumnViewModel>();

    for (int r = 0; r < ordered.Count; r++)
    {
        var matches = ordered[r].Matches.OrderBy(m => m.Position).ToList();
        var spacing = unit * Math.Pow(2, r);
        var slots = matches.Select((m, i) => new BracketSlotViewModel
        {
            Match = m,
            Top = spacing * i + (spacing - unit) / 2
        }).ToList();

        var height = slots.Count == 0 ? cardHeight : slots.Max(s => s.Top) + unit;
        columns.Add(new BracketColumnViewModel { Header = ordered[r].Name, Slots = slots, Height = height });
    }
    return columns;
}
```

18.4.1 A matemática do espaçamento

`unit` (96px por padrão) é a altura "lógica" reservada para cada partida na primeira rodada (card de 80px + um respiro de 16px). Em cada rodada seguinte, o espaço vertical entre partidas **dobra** (`spacing = unit * 2r`) — porque cada partida da rodada `r` precisa ocupar verticalmente o mesmo espaço que as *duas* partidas da rodada `r-1` que a alimentam ocupavam juntas. O deslocamento `(spacing - unit) / 2` centraliza o card all width `unit` dentro desse espaço maior `spacing`, para que ele fique visualmente no meio das duas partidas anteriores — este é exatamente o efeito visual de uma chave eliminatória clássica, só que calculado matematicamente em vez de fixado à mão.

Exemplo genérico para tornar a fórmula concreta (rodada `r=1`, segunda coluna, `unit=96`): `spacing = 96 * 21 = 192`. Partida de índice `i=0`: `Top = 192*0 + (192-96)/2 = 48`. Partida `i=1`: `Top = 192*1 + 48 = 240`. Ou seja, a primeira partida da segunda rodada fica centralizada entre as posições `0` e `96` da primeira rodada (a média é `48` ✓), e a segunda partida da segunda rodada fica centralizada entre `192` e `288` (a média é `240` ✓).

18.4.2 Por que sem linhas conectoras

Essa foi uma escolha explícita, feita via `AskUserQuestion` durante o desenvolvimento: desenhar as linhas de conexão entre uma partida e a próxima exigiria calcular geometria de `Path` (pontos de entrada/saída, curvas) que varia com o número de times e a posição relativa de cada partida — uma complexidade bem

maior do que o espaçamento vertical sozinho. A alternativa escolhida ("colunas com espaçamento padrão de bracket, sem linhas conectoras") é visualmente mais simples mas generaliza para *qualquer* tamanho de chave com uma fração do código.

18.5 O XAML: de hardcoded para genérico

Antes desta refatoração, `TournamentDetailsView.xaml` tinha um bloco fixo — um `Canvas` de 840×384 pixels com posições `Canvas.Left` / `Canvas.Top` **literais** para 7 partidas (assumindo sempre exatamente 8 times, eliminação simples), mais uma `Path` desenhada à mão como conector entre elas. Hoje esse bloco inteiro foi substituído por um `DataTemplate` reusável:

```
<DataTemplate x:Key="BracketColumnTpl">
  <StackPanel Margin="0,0,40,0" Width="240">
    <TextBlock Text="{Binding Header}" .../>
    <ItemsControl ItemsSource="{Binding Slots}" Width="240" Height="{Binding Height}">
      <ItemsControl.ItemsPanel>
        <ItemsPanelTemplate><Canvas/></ItemsPanelTemplate>
      </ItemsControl.ItemsPanel>
      <ItemsControl.ItemContainerStyle>
        <Style TargetType="ContentPresenter">
          <Setter Property="Canvas.Top" Value="{Binding Top}" />
          <Setter Property="Canvas.Left" Value="0" />
        </Style>
      </ItemsControl.ItemContainerStyle>
      <ItemsControl.ItemTemplate>
        <DataTemplate>
          <Border Width="240" Height="80" Style="{StaticResource MatchCard}">
            <ContentControl Content="{Binding Match}" ContentTemplate="{StaticResource
MatchInnerTpl}" />
          </Border>
        </DataTemplate>
      </ItemsControl.ItemTemplate>
    </ItemsControl>
  </StackPanel>
</DataTemplate>
```

O truque técnico central aqui é `ItemsControl.ItemsPanel` trocado para `Canvas` (em vez do `StackPanel` padrão), combinado com `ItemContainerStyle` fazendo `Setter Property="Canvas.Top" Value="{Binding Top}"` — isso funciona porque `Canvas.Top` é uma *propriedade anexada* (*attached property*) que aceita um `double` direto via binding, sem precisar de nenhum `IValueConverter`. É esse binding que aplica, item por item, os valores calculados por `BracketLayout.Build` (a propriedade `Top` de cada `BracketSlotViewModel`).

Esse `DataTemplate` é reaproveitado **três vezes** na view (uma para Upper, uma para Lower, uma para Grande Final), cada uma ligada à propriedade computada correspondente do `TournamentDetailsViewModel`:

```
public List<BracketColumnViewModel> UpperColumns =>
    BracketLayout.Build(Tournament?.Bracket.Where(r => r.Side == BracketSide.Upper) ??
    Enumerable.Empty<BracketRound>());
public List<BracketColumnViewModel> LowerColumns =>
    BracketLayout.Build(Tournament?.Bracket.Where(r => r.Side == BracketSide.Lower) ??
    Enumerable.Empty<BracketRound>());
public bool HasLowerBracket => LowerColumns.Count > 0;
```

A seção "CHAVE INFERIOR" (Lower) e a Grande Final só ficam visíveis quando `HasLowerBracket` é verdadeiro — o que automaticamente cobre o caso de eliminação simples (sem nenhum `if` explícito de "é simples ou dupla?" no XAML: se não há rodadas `Lower`, a lista fica vazia, e a seção some sozinha por causa do binding de `Visibility`).

18.6 Como testar isto sem esperar os horários automáticos

O endpoint `POST /api/debug/generate-bracket/{tournamentId}` (ver §9.4) limpa qualquer chave existente daquele campeonato e chama `LifecycleWorker.GenerateBracket` diretamente — permitindo testar a geração (simples ou dupla, qualquer contagem de times inscritos) sem precisar que `CheckInClosesAt` chegue de verdade. Foi assim que os casos N=4 e N=7 citados em §18.3.1 foram verificados.

Capítulo 19 — Veto de Mapas e Sala da Partida

19.1 Quando um veto começa

`LifecycleWorker`, no ramo `now >= t.StartDate`, cria uma `VetoSession` para toda partida da **primeira rodada** que já tem os dois lados preenchidos (não é "TBD"/"BYE") e ainda não tem sessão:

```
var r1 = t.Basket.OrderBy(r => r.RoundNumber).First();
foreach (var bm in r1.Matches.Where(m => m.TeamATag != "TBD" && m.TeamBTag != "TBD" && m.Status ==
BasketMatchStatus.Pending))
{
    var hasVeto = await db.VetoSessions.AnyAsync(v => v.BasketMatchId == bm.Id);
    if (hasVeto) continue;
    db.VetoSessions.Add(new VetoSession { /* ... */ });
    bm.Status = BasketMatchStatus.Veto;
}
```

Consistente com o que foi discutido no [Capítulo 18](#): como só a primeira rodada tem times reais preenchidos, só a primeira rodada consegue automaticamente abrir vetos hoje. Existe também [POST /api/veto/{basketMatchId}/start](#), chamado pelo client (`MatchRoomViewModel.RefreshAsync`, ver [§19.5](#)) como um mecanismo idempotente de "garantir que a sessão existe" — se o `LifecycleWorker` já criou, ele só devolve a existente; senão, cria uma na hora.

19.2 A sequência de bans/picks — `BuildSequence`

`docs/espec-campeonatos.md §8` define a sequência por formato de série:

- **MD1**: 6 bans alternados (A, B, A, B, A, B), o mapa restante é jogado.
- **MD3**: 2 bans, 2 picks, 2 bans, restante = decider.
- **MD5**: 2 bans, 4 picks, restante = decider (menos bans, porque mais mapas são jogados).

O código generaliza isso com uma fórmula em vez de listar cada caso manualmente:

```
public static List<VetoActionType action, int side> BuildSequence(SeriesFormat series, int poolSize)
{
    int picks = series switch { SeriesFormat.MD3 => 2, SeriesFormat.MD5 => 4, _ => 0 };
    int totalSteps = Math.Max(poolSize - 1, 0); // sempre sobra 1 mapa (o decider)
    int bansBefore = Math.Min(2, Math.Max(totalSteps - picks, 0));
    int bansAfter = Math.Max(totalSteps - picks - bansBefore, 0);

    var steps = new List<VetoActionType, int>();
    int i = 0;
    for (int k = 0; k < bansBefore; k++) steps.Add((VetoActionType.Ban, i++ % 2));
    for (int k = 0; k < picks; k++) steps.Add((VetoActionType.Pick, i++ % 2));
    for (int k = 0; k < bansAfter; k++) steps.Add((VetoActionType.Ban, i++ % 2));
    return steps;
}
```

`side` alterna estritamente `0, 1, 0, 1, ...` ao longo de toda a sequência (`i++ % 2`) — `side 0` é sempre o Time A, `side 1` o Time B, e a alternância nunca reinicia entre a fase de bans e a fase de picks (ela continua de onde parou). `totalSteps = poolSize - 1` é a regra central: **sempre sobra exatamente 1 mapa** no fim, que vira o decider automaticamente — nunca é um pick explícito de ninguém.

O pool padrão do projeto tem 7 mapas (`docs/espec-campeonatos.md`: Mirage, Inferno, Nuke, Ancient, Anubis, Dust2, Train) — para MD1 isso dá `totalSteps = 6`, todos bans (`bansBefore = min(2, 6-0) = 2` ... espera, isso daria só 2 bans antes e o resto depois: com `picks=0`, `bansBefore = min(2, 6) = 2`, `bansAfter = 6-0-2 = 4`, total 6 bans — bate com a especificação "6 bans alternados" para MD1 com pool de 7). Para MD3 com pool de 7: `totalSteps=6`, `picks=2`, `bansBefore=min(2,4)=2`, `bansAfter=6-2-2=2` → 2 bans, 2 picks, 2 bans, exatamente como a especificação descreve.

19.3 Executando uma ação: `POST /api/veto/{bracketMatchId}/action`

Cada chamada:

1. Recalcula a sequência esperada (`BuildSequence`) e confere que `body.TeamTag` é exatamente quem deveria agir no passo atual (`s.StepIndex`) — recusa com mensagem específica se não for ("Não é a vez de X — vez de Y.").

1. Confere que o mapa pedido está de fato disponível (não foi banido/escolhido antes, e está no pool) via `RemainingMaps`.

1. Grava o `VetoStep`, incrementa `s.StepIndex`.
2. **Se a sequência terminou**, cria automaticamente o passo final `Decider` com o único mapa restante, marca `s.IsComplete = true`, e — na mesma operação — **cria a sala da partida** (`Match`).

```
public static List<string> RemainingMaps(VetoSession s)
{
    var used = s.Steps.Select(st => st.Map).ToHashSet(StringComparer.OrdinalIgnoreCase);
    return s.MapPool.Where(m => !used.Contains(m)).ToList();
}
```

19.4 O que acontece exatamente quando o veto termina

Este é um dos trechos mais densos do sistema — vale acompanhar linha a linha (`CompetitionEndpoints.cs`, dentro do `if (s.StepIndex >= seq.Count)`):

```

var playMaps = s.Steps.Where(x => x.Action != VetoActionType.Ban).OrderBy(x => x.Order).Select(x =>
x.Map).ToList();
var isAwsConfigured = MatchServerService.IsConfigured;
var room = new Match
{
    Id = $"m_{Guid.NewGuid():N}",
    Map = playMaps.First(),
    Status = MatchStatus.Scheduled,
    TeamATag = s.TeamATag, TeamBTag = s.TeamBTag,
    BracketMatchId = bm.Id,
    ServerIp = isAwsConfigured ? "" : $"sv{Random.Shared.Next(1,9)}.summit.gg:{27015 +
Random.Shared.Next(0,4)}",
    ServerPassword = $"smt_{Guid.NewGuid().ToString("N")[..8]}",
    ProvisionState = isAwsConfigured ? ServerProvisionState.Requesting : ServerProvisionState.Ready
};

```

Repare no `if (isAwsConfigured)`: se a AWS **não** está configurada neste ambiente (ex. desenvolvimento local, sem `AWS_ACCESS_KEY_ID` / `SUMMIT_AMI_ID` definidos), a sala nasce com um **IP simulado** (`sv3.summit.gg:27017`, nunca um servidor real) e `ProvisionState = Ready` imediatamente — isso existe puramente para que a UI da sala de partida tenha algo para mostrar e testar (o botão "entrar no servidor", o fluxo completo até aqui) sem exigir AWS configurada localmente. Quando a AWS está configurada, `ProvisionState = Requesting`, e o preenchimento do IP real acontece depois, de forma assíncrona (ver [Capítulo 20](#)).

Logo depois, ainda na mesma requisição HTTP (fora do bloco `if`, já com `SaveChangesAsync` feito):

```

var newRoom = await db.Matches.FirstOrDefaultAsync(m => m.BrainetMatchId == bracketMatchId);
if (newRoom != null && newRoom.ProvisionState == ServerProvisionState.Requesting)
{
    var assignedFromPool = await server.TryAssignFromPoolAsync(newRoom.Id, newRoom.Map,
newRoom.ServerPassword);
    if (!assignedFromPool)
        _ = server.ProvisionAsync(newRoom.Id); // fire-and-forget: cold boot como fallback
}

```

Isso é o ponto de integração exato entre o veto e o [Capítulo 20](#): assim que a sala nasce precisando de servidor real, a API **tenta primeiro** pegar um servidor já quente do pool (via RCON, leva segundos); só cai para o cold-boot de uma EC2 nova (minutos) se o pool não tiver nenhum servidor livre no momento. `_ = server.ProvisionAsync(...)` é fire-and-forget deliberado — a resposta HTTP do `/action` não espera o cold-boot terminar (que levaria minutos), ela devolve na hora e o `ServerProvisionPoller` (tick de 10s) acompanha o resto em segundo plano.

19.5 A tela: `MatchRoomViewModel`

Um hub estilo FACEIT que mistura três estados (elencos, veto ao vivo, sala pronta) na mesma tela, usando um `DispatcherTimer` de 3 segundos para simular tempo real (ver §5.6). A cada `RefreshAsync`:

```

var state = await _veto.GetStateAsync(_bracketMatchId) ?? await _veto.StartAsync(_bracketMatchId);

```

Note esse `??`: se ainda não existe sessão (`GET` devolve `null` /404), o client tenta **criar** uma via `StartAsync` — isso é o que permite ao jogador abrir a sala de uma partida mesmo que o `LifecycleWorker` ainda não tenha rodado o tick que criaria a sessão automaticamente (reduz a espera percebida de até 20 segundos para instantâneo, do ponto de vista do jogador).

`MyTurn` decide se os mapas ficam clicáveis, comparando a tag do time do usuário atual com quem o servidor diz que deve agir agora:

```
var next = state.Next;
MyTurn = !s.IsComplete && next != null && string.Equals(next.Team, myTag,
StringComparison.OrdinalIgnoreCase);
```

Cada mapa do pool vira um `VetoMapItem` (ver §7.3) com um rótulo textual calculado a partir do `VetoStep` correspondente, se houver:

```
var label = step?.Action switch
{
    VetoActionType.Ban    => $"BAN {step.TeamTag}",
    VetoActionType.Pick   => $"PICK {step.TeamTag}",
    VetoActionType.Decider => "DECIDER",
    _                     => ""
};
IsClickable = available && MyTurn;
```

Quando `s.IsComplete`, a tela para de esperar ação de veto e passa a buscar a sala (`_veto.GetRoomAsync`), mostrando `"PROVISIONANDO SERVIDOR NA AWS... ~90S"` enquanto `HasRoom` (que checa `!string.IsNullOrEmpty(_room.ServerIp)`) for falso, e parando o timer assim que o IP aparecer:

```
if (HasRoom) { StatusLine = "SERVIDOR PRONTO – BOA SORTE!"; _timer.Stop(); }
else         { StatusLine = "PROVISIONANDO SERVIDOR NA AWS... ~90S"; }
```

O botão "Entrar no servidor" (`ConnectCommand`, tanto aqui quanto em `MatchDetailsViewModel`) não abre nenhum processo do CS2 diretamente — ele copia o comando de conexão para a área de transferência, para o jogador colar no console do próprio jogo:

```
System.Windows.Clipboard.SetText($"connect {_room.ServerIp}; password {_room.ServerPassword}");
```

19.6 Bots de veto para contas de demonstração

Já mencionado em §11.2.1: `LifecycleWorker.AutoVetoBotsAsync` identifica contas "bot" pelo tamanho do id (`team.CaptainId.Length <= 12` — os ids curtos como `usr_ghost` do `SeedData`, contra os ids longos de 36+ caracteres gerados por `Guid.NewGuid():N` para contas reais criadas via login), e faz uma jogada de veto por tick para essas contas, escolhendo aleatoriamente entre os mapas disponíveis (`Random.Shared.Next(remaining.Count)`) — dando ao jogador humano do outro lado a sensação de estar vetando contra um adversário real que também está agindo, em vez de o veto ficar parado esperando indefinidamente porque "o outro lado" é só uma conta de seed sem ninguém logado como ela.

Capítulo 20 — Pool de Servidores CS2 (a lógica, não a infraestrutura)

Este capítulo explica **por que** o sistema de pool existe e **como as peças se encaixam do ponto de vista de produto**. A implementação mecânica de cada classe (`PoolManagerService` , `MatchServerService` , `RconClient`) já foi coberta no [Capítulo 11](#); configuração específica de AWS (Security Groups, AMI, chaves) fica fora deste livro — ver [docs/plano-aws.md](#) .

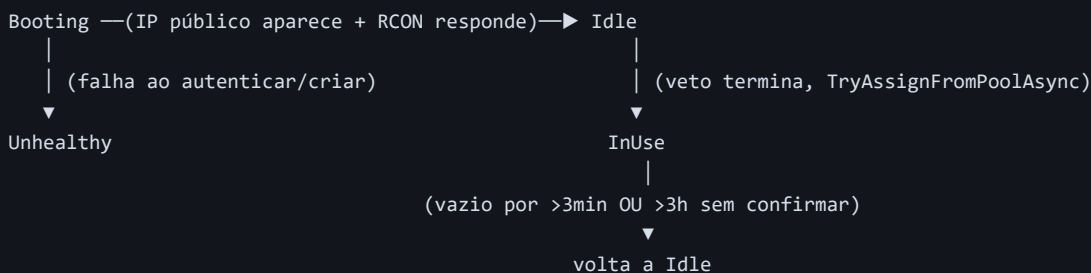
20.1 O problema que o pool resolve

Quando um veto termina (Capítulo 19), o jogador quer entrar no servidor **agora**. Criar uma instância EC2 do zero ("cold boot") — mesmo com a AMI já pronta com CS2/Metamod instalados — ainda leva de 60 a 120+ segundos: o boot da máquina virtual, a inicialização do sistema, e o próprio CS2 carregando. Para um veto de partida competitiva, esperar dois minutos parado numa tela de "preparando servidor" é uma experiência ruim.

A solução (o mesmo padrão usado por FACEIT/ESEA): manter um pequeno número de servidores **já ligados e com CS2 já rodando**, parados num mapa neutro sem ninguém jogando, esperando serem "chamados". Quando um veto termina, em vez de criar uma máquina nova, a API manda dois comandos via **RCON** (protocolo de administração remota do próprio motor Source) para um desses servidores ociosos: trocar o mapa e definir a senha da partida. Isso leva segundos, não minutos.

20.2 O ciclo de vida de um `PoolServer`

```
public enum PoolServerState { Booting = 0, Idle = 1, InUse = 2, Unhealthy = 3 }
```



- **Booting** : acabou de pedir a criação da instância EC2; ainda não tem IP público, ou tem IP mas o CS2 ainda não respondeu por RCON.
- **Idle** : confirmado utilizável — tem IP, e um comando RCON (`status`) foi respondido com sucesso. Só neste estado um servidor é elegível para receber uma partida.
- **InUse** : atribuído a uma partida específica (`CurrentMatchId` , `AssignedAt` preenchidos).

- **Unhealthy** : falhou em algum ponto (RCON não autenticou, instância terminou inesperadamente).

Não participa mais do ciclo normal — conta como "faltando" para o **TopUpAsync** repor o pool.

Todo esse ciclo é orquestrado pelo **PoolManagerService** a cada 30 segundos (mecânica completa em §11.4).

20.3 Por que "instância rodando" não é o mesmo que "pronta"

Este é o ponto mais importante de todo este capítulo, e vale repetir com ênfase de produto (a mecânica de código já está em §11.4.1): a AWS reportar que uma instância está **Running** só significa que o sistema operacional terminou de inicializar — não diz nada sobre se o processo do CS2 dentro dela terminou de carregar, autenticou na Steam, e está de fato aceitando conexões e comandos. Um servidor marcado **Idle** prematuramente (só por estar "Running" na AWS) poderia receber uma partida atribuída e falhar silenciosamente, deixando os dois times sem conseguir entrar. Por isso o único critério aceito para marcar **Idle** é uma resposta RCON real e bem-sucedida — é a diferença entre "a máquina ligou" e "o jogo está pronto para ser jogado".

20.4 O fallback: cold-boot como rede de segurança, não como caminho normal

```
var assignedFromPool = await server.TryAssignFromPoolAsync(newRoom.Id, newRoom.Map, newRoom.ServerPassword);
if (!assignedFromPool)
    _ = server.ProvisionAsync(newRoom.Id);
```

Se **nenhum** **PoolServer** estiver **Idle** no momento exato em que um veto termina (por exemplo, **SUMMIT_POOL_SIZE=1** e esse único servidor já está **InUse** numa outra partida simultânea), o sistema não falha — ele automaticamente recorre ao caminho antigo de criar uma EC2 nova do zero. O jogador nesse caso vê o aviso honesto "PREPARANDO SERVIDOR NA AWS... ~90S" em vez do "SERVIDOR PRONTO" quase instantâneo — **nenhuma mudança de UI foi necessária** para suportar esse caso de transbordo, porque a tela já lidava com o estado "aguardando provisionamento" desde antes de o pool existir (ver §19.5).

Isso significa que **SUMMIT_POOL_SIZE** é, na prática, uma escolha de trade-off entre custo (servidores ociosos ligados 24/7 custam dinheiro continuamente, mesmo sem ninguém jogando neles) e capacidade de absorver picos sem cair para o caminho lento — quanto maior o valor, menos vezes o fallback de cold-boot é acionado, ao custo de mais instâncias sempre ligadas.

20.5 Liberação automática — como o sistema sabe que uma partida "acabou"

Aqui está um ponto sutil e importante: **o sistema não tem um evento de "partida terminou"** (ver Capítulo 21 — esse é o gap central do produto). Então como o **PoolManagerService** sabe quando pode liberar um servidor **InUse** de volta ao pool?

A resposta é: **não sabe, de verdade** — ele **infere** por ausência de jogadores, não por um sinal explícito de fim de partida:

```
var humans = await server.GetHumanPlayerCountAsync(p); // RCON "status", regex sobre a contagem
if (humans == 0) await server.ReleaseToPoolAsync(db, p);
```

Isso é uma aproximação razoável (se ninguém está conectado, é seguro assumir que a partida acabou ou nunca começou de verdade), mas é imperfeita por construção: um servidor entre o fim do veto e a entrada efetiva dos 10 jogadores também mostraria zero humanos — por isso existe a `AssignGrace` de 3 minutos antes de sequer começar a checar (dando tempo real para todos entrarem). E o teto de segurança de 3 horas (`HardReleaseCeiling`) existe para o caso em que o RCON simplesmente pare de responder (falha de rede, processo travado) e a contagem de humanos nunca mais possa ser confirmada — sem esse teto, um servidor nessas condições ficaria "preso" para sempre, nunca voltando ao pool mesmo que a partida real já tivesse terminado há muito tempo.

20.6 Onde este capítulo termina e o próximo começa

Note que nada neste capítulo depende de saber *quem venceu* a partida — o pool só cuida do ciclo de vida da *infraestrutura* (servidor ligado/atribuído/liberado). O que acontece com o **resultado** da partida (placar, estatísticas, quem avança na chave) é uma responsabilidade completamente separada — e é exatamente a responsabilidade que **não existe ainda** no sistema, coberta em detalhe no [Capítulo 21](#).

Capítulo 21 — Pós-Partida: o Que Ainda Não Existe

Todo capítulo anterior da Parte V terminou apontando para este — este é, de longe, o maior buraco conhecido do sistema hoje ([docs/pendencias.md §7](#) o chama literalmente de "MAIOR BURACO DO SISTEMA"). Entender exatamente onde a corrente para é tão importante quanto entender o que já funciona, porque explica por que várias telas do produto mostram dados que nunca mudam sozinhos.

21.1 O ponto exato onde tudo para

O sistema sabe fazer tudo até "o servidor está pronto, aqui está o IP" (fim do [Capítulo 19](#)). A partir do momento em que os jogadores entram e jogam, **não existe nenhum caminho de volta da partida real para a plataforma:**

```
// Isto NÃO existe em lugar nenhum do código:  
app.MapPost("/api/matches/{id}/result", ...)
```

[docs/plano-aws.md](#) já havia planejado esse endpoint desde o início (Fase 4: "Endpoint `POST /api/matches/{id}/result` (webhook MatchZy): placar + stats por player → vencedor avança na chave → `TerminateInstances`") — o MatchZy (o plugin de partida instalado e validado com sucesso no servidor CS2, ver [docs/plano-aws.md](#)) já é capaz de, tecnicamente, mandar um webhook de fim de partida. O que falta é o lado que **recebe** esse webhook na Summit.Api e faz algo com ele.

21.2 As cinco consequências concretas dessa lacuna

21.2.1 Nada atualiza `MatchStatus.Finished` de verdade

Uma `Match` criada pelo veto nasce `Status = Scheduled` (ou, com a AWS não configurada, tecnicamente pronta mas sem ninguém nunca marcando o fim). Nenhum processo do sistema muda esse status para `Finished`, preenche `ScoreA` / `ScoreB`, ou grava `MatchPlayer`s com stats reais. Toda `Match` com `Status = Finished` e scoreboard completo que existe no banco hoje **veio do `SeedData`** (ver [§11.7](#)) — é dado de demonstração fixo, não histórico real de uso.

21.2.2 A chave trava depois da primeira rodada

Já visto em [§18.2](#): rodadas além da primeira nascem `TeamATag = TeamBTag = "TBD"` e **permanecem assim para sempre**, porque nada preenche "quem avançou" a partir do resultado de uma partida anterior. Isso significa que, na prática, um campeonato de verdade jogado neste sistema hoje só teria sua primeira rodada realmente disputável — a chave visualmente existe até a final, mas travada.

21.2.3 Sem monitoramento de no-show/W.O. dentro da partida

Existe um no-show do **check-in** (times que não confirmam presença 30min antes do início são removidos automaticamente, ver §16.4) — mas isso é diferente do no-show **dentro do servidor** ([docs/espec-campeonatos.md §10-11](#): "sistema monitora conectados/mínimo/tempo... após tempo configurado → W.O."). Não há nada que observe quantos jogadores realmente entraram no servidor depois que a sala foi criada.

21.2.4 Badges nunca são concedidas por desempenho real

```
// Existe a tela (BadgesView) e a API de leitura (GET /api/badges/user/{userId})...  
// ...mas nenhuma lógica em lugar nenhum do código chama "conceder badge pra fulano".
```

As únicas linhas em `userbadges` vêm de `SeedData.EnsureSeededAsync` (uma lista fixa de `(userId, badgeId)` escrita à mão). Não existe, por exemplo, um processo que detecte "5 kills num round" e conceda automaticamente a badge "ACE!" — mesmo essa badge já existindo no catálogo.

21.2.5 Campeonato nunca "encerra" sozinho

`TournamentStatus.Finished` existe como valor de enum, mas nada no `LifecycleWorker` (ou em qualquer outro lugar) faz a transição `InProgress → Finished`. [docs/espec-campeonatos.md §17](#) descreve o encerramento como: registrar campeão/vice/3ºs, salvar histórico permanente, atualizar ranking, distribuir premiação — nenhuma dessas etapas tem implementação hoje.

21.3 Por que isso não foi implementado ainda (não é um esquecimento)

Esse gap não é acidental — é uma sequência de decisões de escopo deliberadas ao longo do desenvolvimento (documentadas explicitamente em [docs/pendencias.md](#)), priorizando primeiro "conseguir chegar a uma partida real jogável de ponta a ponta" (login → time → inscrição → escalação → check-in → chave → veto → servidor real com CS2+MatchZy) antes de fechar o círculo de volta. Cada peça dessa cadeia foi testada ao vivo nesse processo — o [Capítulo 19](#) e [Capítulo 20](#) descrevem infraestrutura genuinamente validada em produção (uma partida real conectando a um servidor CS2 real na AWS). O que falta é especificamente o **retorno** dessa informação — um problema de escopo isolado e bem compreendido, não um problema de arquitetura.

21.4 O que precisaria existir (visão de implementação futura, não código atual)

Para deixar claro o tamanho real do trabalho pendente, uma lista concreta do que a próxima fase precisaria construir — isto é análise, **não** uma descrição de código existente:

1. `POST /api/matches/{id}/result` — endpoint que recebe placar final + stats por jogador

(formato ainda a definir, mas o webhook do MatchZy é o candidato natural de origem). Precisa validar que veio de uma fonte confiável (hoje nenhum endpoint da API tem autenticação/API key — isso também precisaria ser resolvido para um webhook exposto à internet, diferente dos endpoints de debug que só rodam localmente).

1. **Avanço de chave** — ao receber um resultado, localizar a próxima `BracketMatch` que deveria

receber o vencedor (isso exige uma função que, dado um `BracketMatch` da Upper, saiba determinar sua "próxima" partida — hoje não existe nenhum link explícito entre partidas consecutivas de rodadas diferentes; a estrutura atual só sabe agrupar por `RoundId / Position`, não "esta partida alimenta aquela outra"). Para eliminação dupla, isso é ainda mais elaborado: o perdedor da Upper precisa ser roteado para a posição certa da Lower, seguindo o padrão específico de "descida" que a fórmula de §18.3.1 já antecipa em quantidade, mas não em *roteamento* individual.

1. **Atualização de estatísticas do usuário** — recalculando (ou incrementando)

`User.TotalKills/TotalDeaths/KD/WinRate/...` a partir do resultado — hoje esses campos só existem como valores estáticos do seed.

1. **Motor de badges** — um conjunto de regras ("5 kills num round → ACE", "MVP em 10 partidas → MVP", etc.) avaliadas a cada resultado recebido.

1. **Encerramento de campeonato** — quando a Grande Final (ou a final da eliminação simples) tem resultado, marcar `TournamentStatus.Finished`, gravar posições finais (`TournamentTeam.FinalPosition`).

1. **Monitoramento de conexão/no-show em servidor** — provavelmente via RCON `status` periódico

(o mesmo mecanismo já usado por `MatchServerService.GetHumanPlayerCountAsync`, ver §20.5), comparando contra os jogadores esperados da escalação de cada lado.

Nenhum desses itens é conceitualmente difícil isoladamente — a razão de listá-los juntos aqui é mostrar que "fechar esse buraco" é, na prática, o tamanho de um novo capítulo de produto inteiro, não um ajuste pequeno.

21.5 Como isso se conecta com o que os capítulos anteriores já entregaram

Vale fechar este capítulo — e o livro — lembrando o que **já** funciona de ponta a ponta, porque é fácil, depois de uma lista de gaps, perder de vista que a maior parte do produto está genuinamente pronta e testada:

- Conta, onboarding e perfil completo (Capítulo 12).
- Time de ponta a ponta: criar, convidar, solicitar entrada, promover, rebaixar, transferir, remover, editar, excluir (Capítulo 13).
- Amizades e bloqueio (Capítulo 14).
- Auditoria de leitura (Capítulo 15).
- Inscrição com escalação, check-in automático com remoção de ausentes

(Capítulo 16, Capítulo 17).

- Geração de chave flexível (qualquer tamanho, simples ou dupla) e renderização genérica

(Capítulo 18).

- Veto de mapas completo (MD1/MD3/MD5) com criação de sala

(Capítulo 19).

- Pool de servidores CS2 reais na AWS, com fallback de cold-boot

(Capítulo 20).

O que resta é especificamente **o que acontece depois de "os jogadores entraram no servidor"** — um escopo isolado, bem delimitado, e agora, espera-se, bem compreendido por quem terminar de ler este livro.

Capítulo 22 — Referência: Classes do Client

Dicionário de consulta rápida de toda classe do projeto `Summit.csproj`. Para o *porquê* de cada uma, veja o capítulo de feature ou de fundação linkado na coluna "Ver também".

22.1 Models (`Models/*.cs`) — compartilhados com a API

Classe/Enum	Arquivo	Papel	Ver também
<code>TeamRole</code> , <code>FriendshipStatus</code> , <code>TeamInvitationStatus</code> , <code>MatchStatus</code>	<code>Enums.cs</code>	Enums de estado usados em vários agregados	Cap. 4 , Cap. 13
<code>User</code>	<code>User.cs</code>	Conta do jogador + estatísticas agregadas + vínculo com time	Cap. 4
<code>Team</code> , <code>TeamInvitation</code>	<code>Team.cs</code>	Time e convites pendentes	Cap. 13
<code>TournamentFormat</code> , <code>SeriesFormat</code> , <code>CheckInStatus</code> , <code>VetoActionType</code> , <code>VetoSession</code> , <code>VetoStep</code> , <code>JoinRequestStatus</code> , <code>TeamJoinRequest</code> , <code>TournamentLineupPlayer</code> , <code>AuditLog</code> , <code>VetoState</code> , <code>VetoNext</code>	<code>Competition.cs</code>	O arquivo mais denso de enums/DTOs de regra — veto, escalação, solicitações, auditoria	Cap. 15 , Cap. 17 , Cap. 19
<code>TournamentStatus</code> , <code>Tournament</code> , <code>TournamentTeam</code> , <code>TournamentTeamEntry</code>	<code>Tournament.cs</code>	Configuração de campeonato, inscrição de time, DTO de listagem por time	Cap. 16
<code>BracketMatchStatus</code> , <code>BracketSide</code> , <code>BracketRound</code> , <code>BracketMatch</code> , <code>TournamentTeamEntry</code>	<code>Bracket.cs</code>	Estrutura da chave	Cap. 18
<code>ServerProvisionState</code> , <code>Match</code> , <code>MatchPlayer</code>	<code>Match.cs</code>	Sala/resultado de partida + scoreboard	Cap. 19 , Cap. 20
<code>Friendship</code> , <code>UserBadge</code>	<code>Friendship.cs</code>	Amizade/bloqueio; junção usuário↔badge	Cap. 14
<code>Badge</code>	<code>Badge.cs</code>	Catálogo de conquistas	Cap. 21
<code>PlayerStats</code> , <code>RecentPerformance</code>	<code>PlayerStats.cs</code>	DTO agregado (não é tabela própria — montado a partir de <code>User</code> + <code>Match</code>)	Cap. 8
<code>RankingPlayer</code> , <code>RankingTeam</code>	<code>RankingEntry.cs</code>	DTOs de ranking (não são <code>User</code> / <code>Team</code> completos)	§10.6
<code>PoolServerState</code> , <code>PoolServer</code>	<code>PoolServer.cs</code>	Estado de um servidor CS2 do pool quente	Cap. 20

22.2 Commands / Helpers / Components

Classe	Arquivo	Papel
<code>RelayCommand</code>	<code>Commands/RelayCommand.cs</code>	Implementação de <code>ICommand</code> para binding de botões/itens
<code>BoolToVisibilityConverter</code> , <code>InverseBoolToVisibilityConverter</code> , <code>NullOrEmptyToVisibilityConverter</code> , <code>BoolToWinLossConverter</code> , <code>BoolToBrushConverter</code>	<code>Helpers/BoolToVisibilityConverter.cs</code>	<code>IValueConverter</code> s usados em bindings XAML
<code>LevelBadge</code> (+ <code>Tier</code> privado)	<code>Components/LevelBadge.xaml.cs</code>	<code>UserControl</code> reutilizável: anel colorido + rótulo de "tier" a partir do nível numérico

22.3 Services (`Services/*.cs`)

Classe	Interface	Papel	Ver também
<code>ApiClient</code> (estático)	—	Único ponto de saída HTTP do client	§6.3
<code>NavigationService</code>	—	Pilha de histórico de ViewModels	§6.1
<code>SessionStore</code> (estático)	—	Persiste/restaura o <code>SteamId</code> da sessão em disco	§6.4.4
<code>SteamAuthService</code>	—	Fluxo de login OpenID, restauração de sessão, modo demo	§6.4.1
<code>SteamConfig</code> (estático)	—	Onde a chave da Steam Web API é lida (env var ou arquivo local)	§6.4.3
<code>SteamWebApiClient</code>	—	Busca nick/avatar reais da Steam, com fallback sem chave	§6.4.2
<code>TeamService</code>	<code>ITeamService</code>	Regra de aplicação sobre <code>TeamRepository</code>	Cap. 13, §8.2
<code>TournamentService</code>	<code>ITournamentService</code>	Regra sobre <code>TournamentRepository</code> + preenche <code>IsRegistered</code>	§7.5
<code>UserService</code>	<code>IUserService</code>	Estado do usuário atual (<code>CurrentUser</code>) + evento de mudança	§5.4
<code>StatsService</code>	<code>IStatsService</code>	Monta <code>PlayerStats</code> combinando <code>User</code> + <code>Match</code>	§8.2
<code>BadgeService</code>	<code>IBadgeService</code>	Catálogo de badges com/sem estado do usuário	§8.2
<code>RankingService</code>	— (sem interface)	Top players/teams	§8.2

22.4 Data (Data/*.cs) — Repositórios

UserRepository, TeamRepository, TournamentRepository, FriendshipRepository, MatchRepository, VetoRepository, AuditRepository, BadgeRepository — catalogados método a método em §8.1.

22.5 ViewModels (ViewModels/*.cs)

Classe	Tela	Notas
BaseViewModel	(base)	INotifyPropertyChanged + SetProperty — ver §3.1
SidebarItem, MainShellViewModel	Shell principal	Menu lateral + título dinâmico + minimizar/maximizar/fechar janela
LoginViewModel	Login	Steam real ou demo
OnboardingViewModel	Onboarding	País + role no primeiro login — §12.2
HomeViewModel	Home	Partidas recentes + campeonatos em destaque
ProfileViewModel	Meu Perfil	Edição inline — §12.3
PlayerProfileViewModel	Perfil de outro jogador	Relação de amizade, amigos em comum — §12.4
SettingsViewModel	Configurações	Só logout hoje
FilterItem, TournamentsViewModel	Lista de campeonatos	Filtro por status
TournamentDetailsViewModel	Detalhes do campeonato	Chave (UpperColumns / LowerColumns / GrandFinalColumns) — Cap. 18
BracketSlotViewModel, BracketColumnViewModel, BracketLayout (estático)	(auxiliar da tela acima)	Algoritmo de espaçamento — §18.4
LineupMemberItem, LineupViewModel	Escalação	Cap. 17
TeamViewModel	Meu Time	O ViewModel mais extenso — Cap. 13
TeamProfileViewModel	Perfil de time (de outro)	Solicitar entrada
JoinRequestsViewModel	Solicitações de entrada	Só para o dono
FriendsViewModel	Amigos	4 abas: amigos/recebidos/enviados/convites de time — §14.5
MatchListItem, MatchesViewModel	Minhas partidas	
ScoreboardRow, MatchDetailsViewModel	Detalhes de partida	Placar + scoreboard dois lados
VetoMapItem, MatchRoomViewModel	Sala da partida	Elenco + veto ao vivo + servidor — §19.5
AuditLogViewModel	Histórico	Somente leitura — Cap. 15
BadgesViewModel	Badges	Catálogo + desbloqueadas

Classe	Tela	Notas
<code>RankingViewModel</code>	Ranking	Pódio + resto, jogadores/times
<code>StatsViewModel</code>	Estatísticas	Gráficos normalizados (<code>KDNormalized</code> , <code>ADRNORMALIZED</code>)

22.6 Views (`Views/*.xaml` + `.xaml.cs`)

Uma View por ViewModel da lista acima (mesmo nome, sufixo `View` em vez de `ViewModel`), todas registradas em `App.xaml` como `DataTemplate` (ver §6.2). Todo `.xaml.cs` é vazio além de `InitializeComponent()` , exceto onde já observado neste livro.

22.7 `App.xaml.cs` — o composition root

`App` expõe os services "globais" como propriedades estáticas, instanciadas uma única vez em `OnStartup` — o "container de DI manual" do client (ver §3.4). Também registra um handler global de exceção não tratada (`DispatcherUnhandledException`) que mostra um `MessageBox` com tipo/mensagem/stack trace em vez de deixar o processo travar silenciosamente — a rede de segurança de último recurso do client inteiro.

Capítulo 23 — Referência: Classes da API

Dicionário de consulta rápida de toda classe do projeto `Summit.Api/Summit.Api.csproj`. Os `Models/*.cs` são os mesmos catalogados em [§22.1](#) — não repetidos aqui.

23.1 Arquivos e responsabilidades

Arquivo	Papel	Ver também
<code>Program.cs</code>	Bootstrap (DI, DbContext, workers) + endpoints de Users/Teams/Tournaments/Matches/Friends/Badges/Ranking + todos os <code>/api/debug/*</code>	Cap. 9 , §10.1-10.6
<code>ApiDbContext.cs</code>	Mapeamento EF Core completo — enums→int, <code>.Ignore()</code> de computed properties, <code>DeleteBehavior</code> por relacionamento	Cap. 4
<code>CompetitionEndpoints.cs</code>	Endpoints das especificações funcionais: solicitações de entrada, cargos, check-in, escalção, veto, auditoria + helpers de regra (<code>IsOwner</code> , <code>ValidateLineupAsync</code> , <code>BuildSequence</code> , <code>Audit</code>)	§10.7-10.11 , Caps. 13 , 16 , 17 , 19
<code>LifecycleWorker.cs</code>	Motor do ciclo de vida do campeonato (tick 20s): fecha check-in, gera chave, inicia campeonato, abre vetos, roda bots	§11.2 , Caps. 16 , 18 , 19
<code>MatchServerService.cs</code>	Provisionamento AWS (cold-boot + pool) + atribuição via RCON de alto nível	§11.3 , Cap. 20
<code>PoolManagerService.cs</code>	Mantém o pool de servidores quentes (tick 30s): repõe, confirma, libera	§11.4 , Cap. 20
<code>RconClient.cs</code>	Cliente do protocolo Source RCON escrito à mão	§11.5
<code>ServerProvisionPoller.cs</code>	Acompanha instâncias de cold-boot (tick 10s)	§11.6
<code>SeedData.cs</code>	Dados de demonstração (usuários, times, campeonatos, chave, partidas, badges, amizades)	§11.7

23.2 `ApiDbContext` — `DbSet` S

```
DbSet<User> Users; DbSet<Team> Teams; DbSet<TeamInvitation> TeamInvitations;  
DbSet<Friendship> Friendships; DbSet<Tournament> Tournaments; DbSet<TournamentTeam> TournamentTeams;  
DbSet<BracketRound> BracketRounds; DbSet<BracketMatch> BracketMatches;  
DbSet<Match> Matches; DbSet<MatchPlayer> MatchPlayers;  
DbSet<Badge> Badges; DbSet<UserBadge> UserBadges; DbSet<TeamJoinRequest> TeamJoinRequests;  
DbSet<TournamentLineupPlayer> TournamentLineupPlayers;  
DbSet<VetoSession> VetoSessions; DbSet<VetoStep> VetoSteps;  
DbSet<AuditLog> AuditLogs; DbSet<PoolServer> PoolServers;
```

Praticamente um-para-um com as tabelas do [Capítulo 4](#): o dump `schema.sql` tem 17, e o único `DbSet` sem tabela correspondente nesse dump é `PoolServers` — criada depois da última exportação do arquivo, via `CREATE TABLE` manual direto no MySQL (o mesmo motivo, e o mesmo risco de desatualização, descrito na ressalva em [§4.7](#) sobre esse arquivo poder ficar desatualizado em relação ao `ApiDbContext` real).

23.3 Helpers estáticos de `CompetitionEndpoints` — referência rápida

Método	Assinatura resumida	Uso
<code>IsOwner</code>	<code>(db, teamId, userId) → bool</code>	Confere se <code>userId</code> é <code>Captain</code> do <code>teamId</code>
<code>IsOwnerOrSub</code>	<code>(db, teamId, userId) → bool</code>	Confere se <code>userId</code> é <code>Captain</code> ou <code>ViceCaptain</code>
<code>ValidateLineupAsync</code>	<code>(db, tournamentId, teamId, playerIds, captainUserId, ignoreTournamentTeamId, requiredCount) → string?</code>	Valida escalação; <code>null</code> = válida — Cap. 17
<code>BuildSequence</code>	<code>(series, poolSize) → List<(VetoActionType, int side)>;</code>	Sequência de bans/picks por formato — Cap. 19
<code>RemainingMaps</code>	<code>(session) → List<string>;</code>	Mapas ainda não usados na sessão de veto
<code>Audit</code>	<code>(db, action, actor, target, teamId, tournamentId, oldValue, newValue, reason) → Task</code>	Grava log de auditoria (não salva sozinho) — §3.8

23.4 Métodos públicos de `MatchServerService` — referência rápida

Método	Categoria	Uso
<code>ProvisionAsync(matchId)</code>	Provisionamento cold-boot	Fallback quando o pool não tem servidor livre
<code>ProvisionPoolServerAsync()</code>	Provisionamento pool	Chamado por <code>PoolManagerService.TopUpAsync</code>
<code>LaunchBareInstanceAsync()</code>	Provisionamento manual	Sem User Data — para configuração manual via SSH (debug)
<code>TryAssignFromPoolAsync(matchId, map, password)</code>	RCON	Tenta atribuir servidor <code>Idle</code> ; <code>false</code> se não houver nenhum
<code>ReleaseToPoolAsync(db, poolServer)</code>	RCON	Reseta mapa/senha, marca <code>Idle</code>
<code>CheckPoolServerAliveAsync(poolServer)</code>	RCON	Confirma que o CS2 responde (<code>status</code>)
<code>GetHumanPlayerCountAsync(poolServer)</code>	RCON	Extrai contagem de humanos via regex sobre <code>status</code>
<code>PollAsync(db, match)</code>	Polling AWS	Grava IP quando cold-boot fica <code>Running</code>
<code>PollPoolServerAsync(poolServer)</code>	Polling AWS	Idem, para instância do pool
<code>TerminateAsync(id) / StopAsync(id)</code>	Controle AWS	Termina/para uma instância manualmente

23.5 Rotas — índice completo por prefixo

Para a tabela detalhada de cada rota (regra de negócio embutida), ver [Capítulo 10](#). Índice de prefixos, para localizar rapidamente onde procurar:

Prefixo	Arquivo	Seção
/api/users/*	Program.cs	§10.1
/api/teams/* (CRUD básico, convites, saída)	Program.cs	§10.2
/api/teams/*/join-requests , /promote , /demote , /transfer-ownership	CompetitionEndpoints.cs	§10.7, §10.8
/api/tournaments/* (listagem, registro)	Program.cs	§10.3
/api/tournaments/*/checkin , /close-checkin , /lineup	CompetitionEndpoints.cs	§10.9
/api/matches/* (leitura)	Program.cs	§10.4
/api/matches/by-bracket/*	CompetitionEndpoints.cs	§10.10
/api/friends/*	Program.cs	§10.5
/api/badges/* , /api/ranking/*	Program.cs	§10.6
/api/veto/*	CompetitionEndpoints.cs	§10.10
/api/audit	CompetitionEndpoints.cs	§10.11
/api/debug/*	Program.cs	§9.4

Apêndices

Apêndice A — Glossário

Termo	Significado neste projeto
Summit	Nome do produto (a pasta no disco ainda se chama <code>Wallbang</code> , o nome anterior — ver §1.2)
Dono (<code>TeamRole.Captain</code>)	Cargo único e obrigatório de um time — não confundir com "capitão da escalação"
Sublíder (<code>TeamRole.ViceCaptain</code>)	Cargo intermediário de um time, promovido/rebaixado pelo dono
Capitão da escalação (<code>TournamentTeam.CaptainUserId</code>)	Papel <i>por campeonato</i> , entre os 5 escalados — pode ser qualquer um deles, dono ou não
Escalação / Lineup	Os 5 jogadores que representam o time num campeonato específico (Cap. 17)
Elenco	Todos os membros do time, sem limite de 5 — distinto da escalação
Chave / Bracket	A estrutura eliminatória de um campeonato (Cap. 18)
Upper / Lower / Grande Final	As três sub-chaves de uma eliminação dupla (<code>BracketSide</code>)
Veto	Sequência de bans/picks de mapa antes de uma partida (Cap. 19)
Decider	O mapa final de uma série, decidido por eliminação (nunca escolhido diretamente)
Pool (de servidores)	Servidores CS2 mantidos sempre ligados/prontos para atribuição rápida via RCON (Cap. 20)
Cold-boot	Criar uma instância EC2 nova do zero (60-120s+) — o caminho lento, usado como fallback do pool
RCON	Protocolo de administração remota do motor Source, usado para trocar mapa/senha sem recriar a instância
GSLT	Game Server Login Token — credencial exigida pela Steam para um servidor dedicado autenticar
AMI	Amazon Machine Image — o "molde" de disco usado para criar instâncias EC2 já com CS2 instalado
MatchZy	Plugin de partida (via CounterStrikeSharp) instalado no servidor CS2 — warmup, placar, webhook de resultado
<code>EnsureCreated</code>	Método do EF Core que cria o schema se não existir, sem suporte a migrations incrementais (§4.2)
DTO de apresentação local	Classe auxiliar de uma tela específica (<code>MatchListItem</code> , <code>ScoreboardRow</code> , etc.), não compartilhada com a API (§7.2)

Apêndice B — Convenções de Código

- **Ids:** sempre `string`, gerados como `($"{prefixo}_{Guid.NewGuid():N}"` (ex. `usr_`, `team_`, `trn_`, `bm_`, `rnd_`, `m_`, `mp_`, `fr_`, `inv_`, `jrj_`, `lp_`, `veto_`, `vst_`, `aud_`, `pool_`). Nunca `int IDENTITY`.
- **Enums:** sempre com valores numéricos explícitos (`Pending = 0`, `Accepted = 1`, ...) e nunca reordenados depois de existir dado gravado (ver §4.6).
- **Repositórios do client:** um método por endpoint, sem lógica de negócio — só tradução para URL/corpo (ver §3.2).
- **Services do client:** onde mora a checagem "isso faz sentido para o usuário atual?" antes de delegar ao repositório.
- **ViewModels:** sempre herdam `BaseViewModel`; construtor monta `RelayCommand`s e dispara `_ = LoadAsync()` sem esperar.
- **Endpoints da API:** `record` posicional para o corpo de requisição; `Results.Ok` / `BadRequest(string)` / `Forbid()` conforme o tipo de falha (ver §10.12).
- **Validação:** sempre repetida no backend, mesmo quando o client já checkou (ver §3.7) — a única fonte de verdade é a API.
- **Auditoria:** toda ação administrativa relevante termina com `await CompetitionEndpoints.Audit(...)`, antes do `SaveChangesAsync()` final (não chama `SaveChangesAsync` sozinha).
- **Background workers:** sempre `IServiceScopeFactory` (nunca `ApiDbContext` direto no construtor), sempre `try/catch` ao redor do corpo do tick, sempre `Task.Delay(..., ct)`.
- **Comentários no código:** curtos, no topo de um método ou classe, explicando o *porquê* não óbvio (uma decisão de escopo, uma armadilha já encontrada) — não o *o quê* (o nome do método já diz isso).

Apêndice C — Roadmap Consolidado

Resumo do estado de cada área, espelhando `docs/pendencias.md` no momento em que este livro foi escrito:

Completo e testado

- Login Steam real + sessão persistida; onboarding mínimo; edição de perfil completa.
- Time: criar, convidar, aceitar/recusar convite, sair (com transferência automática de propriedade), solicitação de entrada, promover/rebaixar/transferir, editar, excluir, kick.
- Amizades: pedido/aceite/recusa/remoção, bloqueio/desbloqueio, amigos em comum.

- Auditoria (somente leitura).
- Campeonato: inscrição com escalação, check-in automático com remoção de ausentes.
- Escalação: seleção de 5 + capitão, editável até o check-in abrir.
- Chave: geração flexível (qualquer tamanho de time), eliminação simples e dupla, renderização

genérica sem linhas conectoras.

- Veto: MD1/MD3/MD5 completos, criação automática de sala.
- Servidor CS2 real (AWS): cold-boot funcional, pool de servidores quentes com RCON, fallback automático, CounterStrikeSharp + MatchZy validados ao vivo.

Decisões conscientes de escopo (não é falta, é opção)

- Sistema Suíço: não implementado (só enum existe).
- Denúncia de perfil: fora de escopo (exigiria fila de moderação administrativa).
- Exclusão de time sem validar campeonato ativo: simplificação deliberada.
- "Modo alpha" da escalação (aceita elenco < 5): acomodação temporária para poucos usuários de teste, não a regra final de produto.

O maior gap conhecido — pós-partida (Capítulo 21)

- Sem endpoint de resultado de partida.
- Chave não avança automaticamente além da primeira rodada.
- Sem no-show/W.O. monitorado dentro do servidor.
- Badges nunca concedidas automaticamente.
- Campeonato nunca encerra sozinho (sem campeão/vice registrados automaticamente).
- Notificações: não existe nenhum sistema de notificação (in-app ou outro) — toda mudança de estado só aparece se o usuário recarregar a tela manualmente.

Fim do livro. Para o estado mais atual e vivo do roadmap, sempre prefira `docs/pendencias.md` no repositório — este livro descreve a arquitetura e a lógica de forma duradoura, mas a lista de pendências muda com o ritmo de desenvolvimento.